

В.А. Калитвин

ЧИСЛЕННЫЕ МЕТОДЫ. ИСПОЛЬЗОВАНИЕ PYTHON

Учебное пособие

Липецк - 2024

Министерство просвещения РФ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Липецкий государственный педагогический университет
имени П.П. Семенова-Тян-Шанского»

В. А. Калитвин

ЧИСЛЕННЫЕ МЕТОДЫ. ИСПОЛЬЗОВАНИЕ PYTHON

Учебное пособие

Липецк - 2024

УДК 517
ББК 22.161.6
К 172

Печатается по решению кафедры
математики и физики
ФГБОУ ВО «ЛГПУ имени
П.П. Семенова-Тян-Шанского».
Протокол №8 от 09.04.2024 г.

Калитвин, В.А. Численные методы. Использование Python: учебное пособие /
В.А. Калитвин. – Липецк: ЛГПУ имени П.П. Семенова-Тян-Шанского, 2024. – 193 с.

ISBN 978-5-907792-62-3

Учебное пособие содержит разработку практической части курса «Численные методы», ориентированного на использование языка программирования Python и операционной системы GNU/Linux.

В пособии приведены десять лабораторных работ, охватывающих основные разделы курса. Рассмотрена задача численного решения интегральных уравнений Вольтерра-Фредгольма с частными интегралами. Каждая лабораторная работа содержит краткие теоретические сведения, задания для самостоятельной работы и вопросы к защите лабораторной работы. Разработка содержит большое количество примеров и список литературы, содержащий 62 наименования.

Учебное пособие предназначено для студентов физико-математических факультетов и представляет интерес для аспирантов, преподавателей, учителей математики и информатики. Рекомендовано для слушателей программ дополнительного профессионального образования соответствующего направления подготовки.

УДК 517
ББК 22.161.6
К 172

Рецензенты:

Ю.А. Кондрашин, кандидат технических наук, доцент кафедры информатики, информационных технологий и защиты информации института естественных, математических и технических наук ЛГПУ имени П.П. Семенова-Тян-Шанского;

А.А. Никитина, кандидат физико-математических наук, доцент Липецкого филиала РАНХиГС.

ISBN 978-5-907792-62-3

© ФГБОУ ВО «Липецкий государственный педагогический университет имени П.П. Семенова-Тян-Шанского», 2024
© В.А. Калитвин, 2024

Оглавление

Предисловие	6
1. Введение в программирование на Python	7
1.1. Простейшая программа на python	9
1.2. Типы данных в python	10
1.3. Операторы	10
1.4. Переменные	11
1.5. Ввод и вывод данных	14
1.6. Форматированный вывод	16
1.7. Математические функции (math)	19
1.8. Ветвление в программе	20
1.9. Циклы	21
1.10. Последовательности	23
1.11. Функции	31
1.12. Множества	36
1.13. Словари	37
1.14. Модули и пакеты	38
1.15. Файлы	41
1.16. Документирование программного кода	42
1.17. Математические пакеты numpy и scipy	44
1.18. Визуализация данных с использованием matplotlib	48
1.19. Объектно-ориентированное программирование	58
1.20. Задания для самостоятельной работы	67
2. Теория погрешностей и машинная арифметика	70
2.1. Основные понятия теории приближенных вычислений	70
2.2. Определение количества верных цифр по относительной погрешности приближенного числа	71
2.3. Формулы для вычисления погрешностей арифметических действий	72
2.4. Вычисление погрешностей значений элементарных функций	72
2.5. Задания для самостоятельной работы	73

3. Решение нелинейных уравнений	77
3.1. Локализация корней	77
3.2. Решение нелинейных уравнений с помощью библиотечной функции fsolve	77
3.3. Метод деления отрезка пополам	78
3.4. Теорема Банаха	78
3.5. Метод простых итераций	82
3.6. Метод Ньютона	86
3.7. Модифицированный метод Ньютона	87
3.8. Метод хорд	88
3.9. Задания для самостоятельной работы	88
4. Прямые методы решения систем линейных алгебраических уравнений	91
4.1. Метод обратной матрицы	91
4.2. Метод Гаусса	91
4.3. Метод прогонки решения систем с трехдиагональными матрицами коэффициентов	93
4.4. Задания для самостоятельной работы	93
5. Итерационные методы решения систем уравнений	98
5.1. Итерационные методы решения систем линейных алгебраических уравнений	98
5.2. Решение систем нелинейных уравнений	101
5.3. Задания для самостоятельной работы	105
6. Приближение функций	113
6.1. Постановка задачи	113
6.2. Встроенные функции SciPy	113
6.3. Интерполяционный многочлен Лагранжа	115
6.4. Интерполяционные многочлены Ньютона для равноотстоящих узлов	117
6.5. Интерполяция сплайнами	120
6.6. Метод наименьших квадратов	123
6.7. Задания для самостоятельной работы	125

7. Численное интегрирование	130
7.1. Формула левых прямоугольников	131
7.2. Формула правых прямоугольников	131
7.3. Формула средних прямоугольников	132
7.4. Формула трапеций	132
7.5. Формула Симпсона	133
7.6. Метод Монте-Карло	133
7.7. Метод Монте-Карло для кратных интегралов	134
7.8. Задания для самостоятельной работы	134
8. Численное решение дифференциальных уравнений	138
8.1. Метод Эйлера	138
8.2. Метод Эйлера-Коши	140
8.3. Метод Рунге-Кутты	140
8.4. Решение краевой задачи для линейного дифференциального уравнения второго порядка методом конечных разностей	140
8.5. Задания для самостоятельной работы	144
9. Численное решение уравнений в частных производных	149
9.1. Общие сведения из теории уравнений в частных производных .	149
9.2. Метод сеток	150
9.3. Решение уравнений гиперболического типа	153
9.4. Решение уравнений параболического типа	155
9.5. Решение уравнений эллиптического типа	160
9.6. Задания для самостоятельной работы	163
10. Численное решение интегральных уравнений	174
10.1. Численное решение интегральных уравнений Фредгольма	174
10.2. Численное решение интегральных уравнений Вольтерра	176
10.3. Численное решение уравнений Вольтерра-Фредгольма с част- ными интегралами	178
10.4. Задания для самостоятельной работы	182
Литература	188

Предисловие

Пособие предназначено для изучения одного из наиболее популярных языков программирования - **Python**. **Python** обладает простым и удобным синтаксисом, поддерживает различные методологии разработки программного обеспечения. **Python** распространяется бесплатно и может быть установлен в операционных системах **Windows**, **GNU/Linux**, **Mac OS X**. Эти особенности языка позволяют легко использовать его при изучении дисциплин математического и компьютерного профилей в вузах и школах. Пособие содержит большое количество примеров программного кода и заданий для самостоятельной работы.

Издание ориентировано на работу с дистрибутивом **Ubuntu**, но легко может быть использовано с другими операционными системами. При подготовке пособия использована литература, список которой приведен на страницах 188-192.

1. Введение в программирование на Python

Python – высокоуровневый язык программирования общего назначения с динамической типизацией и автоматическим управлением памятью.

Официальный сайт: <https://www.python.org/>.

Автор – сотрудник голландского института CWI Гвидо ван Россум.

Год создания – 1991.

Python – очень удобный и гибкий язык программирования, который может быть использован для различных задач: системного администрирования, разработки веб-приложений, анализа данных, вычислительной математики, визуализации данных. В последнее время является лидером в области Data Science.

Особенности языка:

- Язык высокого уровня
- Объектно-ориентированный
- Интерпретируемый
- Язык со строгой динамической типизацией
- Универсальный
- Свободно-распространяемый

Основные ветки:

- Python 2.x
- Python 3.x

Парадигмы программирования:

- Императивное (структурный, процедурный, модульный подходы)
- Объектно-ориентированное
- Функциональное

Для улучшения читаемости программного кода принято использовать стандарт PEP8 (<https://www.python.org/dev/peps/pep-0008/>).

Основные правила оформления кода по стандарту PEP8:

- отступ – 4 пробела;
- длина строки < 80 символов;
- змеиный стиль именования переменных: `my_variable` ;

- константы: `CONST_NAME` .

Популярные среды разработки:

- PyCharm
- Anaconda
- Google Colab
- VS Code
- Eclipse + pydev
- Vim
- Emacs

Известные проекты на python:

- Dropbox
- Instagram
- Google (например, сервис Google Colab)
- Spotify
- Netflix
- Uber
- Pinterest
- Bitbucket

Гвидо ван Россум разработал свод правил, которым он следовал при создании языка («Дзен python»). Прочитать все эти правила можно, выполнив в python команду `import this`.

Приведем некоторые из них:

Красивое лучше, чем уродливое.

Явное лучше, чем неявное.

Простое лучше, чем сложное.

Сложное лучше, чем запутанное.

Плоское лучше, чем вложенное.

Читаемость имеет значение.

Особые случаи не настолько особые, чтобы нарушать правила.

Встретив двусмысленность, отбрось искушение угадать.

Сейчас лучше, чем никогда.

Хотя никогда зачастую лучше, чем прямо сейчас.

Если реализацию сложно объяснить, то это плохая идея.

Если реализацию легко объяснить, то идея, возможно хороша.

1.1. Простейшая программа на python

Простейшая программа на python

```
1 print("Hello, World!")
```

Листинг 1: Простейшая программа - 1

Результат:

Hello, World!

Здесь используется функция `print`, предназначенная для вывода данных на экран. В данном примере на экран выводится строка `"Hello, World!"`.

Профессиональный стиль оформления предыдущей программы имеет вид:

```
1 def main():
2     print("Hello, World!")
3
4
5 if __name__ == 'main':
6     main()
```

Листинг 2: Простейшая программа - 2

Такая структура кода позволяет указать, какой код будет выполняться при запуске программы. В данном примере это функция `main`. Подробнее функции обсуждаются дальше. Кроме этого, такой подход позволяет обеспечить корректное включение данного программного кода в качестве элемента другой программы.

Схема выполнения программы python

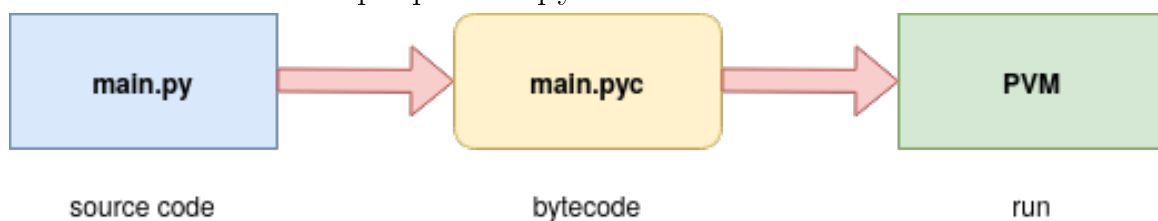


Рисунок 1.1. Этапы создания выполняемой программы

1.2. Типы данных в python

Типы данных:

- None
- Логический (bool)
- Числа (int, float, complex)
- Последовательности (str, list, tuple)
- Множества (set, frozenset)
- Словари (dict)
- Файлы (file)
- Исключения (exceptions)
- Другие (byte, ...)

1.3. Операторы

Арифметические операторы

В python представлены следующие арифметические операторы

+	сложение (конкатенация)
-	вычитание
*	умножение (повторение)
/	деление
//	деление с усечением дробной части
**	возведение в степень
%	остаток от деления

Таблица 1: Операторы присваивания

Операторы сравнения

Оп.	Описание	Пример	Описание
>	больше	<code>a > b</code>	возвращает true, если <code>a</code> больше <code>b</code>
<	меньше	<code>a < b</code>	возвращает true, если <code>a</code> меньше <code>b</code>
>=	больше или равно	<code>a >= b</code>	возвращает true, если <code>a</code> $\geq$ <code>b</code>
<=	меньше или равно	<code>a <= b</code>	возвращает true, если <code>a</code> $\leq$ <code>b</code>
==	равно	<code>a == b</code>	возвращает true, если <code>a</code> равно <code>b</code>
!=	не равно	<code>a != b</code>	возвращает true, если <code>a</code> не равно <code>b</code>

Таблица 2: Операторы сравнения

Логические операторы

Оператор	Описание	Пример	Описание
and	логическое И	a and b	true, если a=true и b=true
or	логическое ИЛИ	a or b	true, если a=true или b=true
not	логическое НЕ	not a	true, если a=false

Таблица 3: Логические операторы

Операторы проверки вхождения в последовательность

Операторы проверки вхождения в последовательность: in, not in.

Операторы присваивания

+=	a += b	a = a + b
-=	a -= b	a = a - b
*=	a *= b	a = a * b
/=	a /= b	a = a / b
**=	a **= b	a = a ** b
%=	a %= b	a = a % b

Таблица 4: Операторы присваивания

Группировка в выражениях осуществляется с помощью круглых скобок.

1.4. Переменные

В python все является объектом (в том числе и переменные).

Переменные можно представить как ссылки на объекты, содержащие значения.

В python не требуется объявлять тип переменных.

Тип данных определяется автоматически по типу выражения, стоящего после знака =. При этом можно изменять значение переменной одного типа на значение другого типа.

Тип переменной можно узнать с помощью команды type.

У каждого объекта есть идентификатор. Узнать идентификатор можно с помощью метода id().

```
1 a = 1 # int
2 print(type(a))
3 print(id(a))
```

Листинг 3: Целочисленная переменная

Результат:

```
<class 'int'>  
140020604797168
```

```
1  b = 1.0  # float  
2  print(type(b))  
3  print(id(a))
```

Листинг 4: *Вещественная переменная*

Результат:

```
<class 'float'>  
140020604797168
```

```
1  c = 1 + 1j  # complex  
2  print(type(c))  
3  print(id(c))
```

Листинг 5: *Комплексное число*

Результат:

```
<class 'complex'>  
140020303776976
```

Изменяемые и неизменяемые типы

К неизменяемым типам относятся int, float, str, tuple. К изменяемым типам относятся list, dict.

```
1  x = 1  
2  print(id(x))  
3  x = 2  
4  print(id(x))
```

Листинг 6: *Изменяемый тип*

Результат:

```
140020604797168
140020604797200
```

```
1  x = [1, 2, 3]
2  print(x)
3  print(id(x))
4  x[0] = 5
5  print(x)
6  print(id(x))
```

Листинг 7: *Неизменяемый тип*

Результат:

```
[1, 2, 3]
140020303950464
[5, 2, 3]
140020303950464
```

Имена переменных

Имя переменной может содержать только буквенно-цифровые символы и подчеркивание (Az, 0-9 и `_`).

Имя переменной должно начинаться с буквы или символа подчеркивания `_`.

Имя переменной не может начинаться с цифры.

Имена переменных чувствительны к регистру (abc, Abc и ABC – три разные переменные).

Имя переменной не должно совпадать с зарезервированным ключевым словом.

Список ключевых слов

```
1  import keyword
2
3  print(keyword.kwlist)
```

Листинг 8: *Ключевые слова*

Результат:

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await',  
'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except',  
'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is',  
'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return',  
'try', 'while', 'with', 'yield']
```

Комментарии

Однострочные комментарии начинаются с символа `#`.

```
1 print(1) # comment
```

Листинг 9: Однострочный комментарий

Многострочных комментариев нет, но можно заключать текст комментария в тройные кавычки.

```
1 """  
2 comment  
3 comment  
4 comment  
5 """
```

Листинг 10: Многострочный комментарий

Обычно такой вид комментариев используется для документирования программного кода, например создания описания функций.

1.5. Ввод и вывод данных

Для организации вывода данных используется функция `print`. По умолчанию вывод осуществляется в стандартный поток (на экран), хотя можно организовать вывод в текстовый файл или на печать.

Общий синтаксис функции `print`:

```
print(*objects, sep=' ', end='\n', file=None, flush=False)
```

Здесь `*objects` — один или несколько объектов, которые будут отправлены в поток вывода.

Пример.

```
1 print('Text1') # Вывод на экран одной строки
2 print('Text1', 'Text2') # Вывод на экран двух строк
```

Листинг 11: Использование функции `print()`

Результат:

```
Text1
Text1 Text2
```

Необязательный параметр `sep` позволяет указать разделитель между объектами. Разделитель должен быть строкового типа. По умолчанию используется пробел.

Пример.

```
1 print('Text1', 'Text2', 'Text3') # Разделитель - пробел
2 print('Text1', 'Text2', 'Text3', sep='.') # Разделитель - точка
3 print('Text1', 'Text2', 'Text3', sep=';') # Разделитель - точка
4                                           # с запятой
5 print('12', '40', '57', sep=':') # Разделитель - двоеточие
```

Листинг 12: Использование функции `print()`

Результат:

```
Text1 Text2 Text3
Text1.Text2.Text3
Text1;Text2;Text3
12:40:57
```

Необязательный параметр `end` позволяет указать символ, который будет выведен после всех объектов. По умолчанию используется символ перевода на новую строку (`'\n'`).

Пример.

```
1 print('Text1', end='.') # В конце вывода - точка.
2 print('Text2', end='.') # В конце вывода - точка.
3 print('Text1', 'Text3', sep='.', end='.') # В конце вывода -
4                                           # точка.
5 print('Text4') # В конце вывода - перевод на новую строку.
6 print('Text5') # В конце вывода - перевод на новую строку.
7 print('Text7') # В конце вывода - перевод на новую строку.
```

Листинг 13: Использование функции `print()`

Результат:

```
Text1.Text2.Text1.Text3.Text4
Text5
Text7
```

Необязательный параметр `file` позволяет указать поток вывода. По умолчанию или в случае, когда указано значение `None`, используется `sys.stdout` (консоль). С помощью этого параметра можно перенаправить вывод в текстовый файл.

Необязательный параметр `flush` позволяет отключить буферизированный вывод. По умолчанию используется значение `False`, то есть буферизация включена.

В функции `print` в качестве объектов можно использовать переменные. В этом случае в поток вывода отправляются значения переменных.

Пример.

```
1  a = 1
2  b = 5
3  c = 7
4  print('a=', a, 'b=', b, 'c=', c)
```

Листинг 14: Использование функции `print()`

Результат:

```
a= 1 b= 5 c= 7
```

1.6. Форматированный вывод

В предыдущем примере результат получен в виде

```
a= 1 b= 5 c= 7
```

Что делать, если мы хотим представить его следующим образом

```
a=1, b=5, c=7.
```

Эту задачу можно решить разными способами.

Первый вариант - использовать параметры `sep` и `end` в функции `print`.

Пример.

```
1 a = 1
2 b = 5
3 c = 7
4 print('a=', a, ', ', 'b=', b, ', ', 'c=', c, '. ', sep='')
```

Листинг 15: *Использование функции print()*

Результат:

a=1, b=5, c=7.

Пример.

```
1 a = 1
2 b = 5
3 c = 7
4 print('a=', a, ', ', 'b=', b, ', ', 'c=', c, sep='', end='.')
```

Листинг 16: *Использование функции print()*

Результат:

a=1, b=5, c=7.

Однако, назвать такие варианты реализации хорошо читаемыми едва ли возможно.

Второй способ - использование оператора % в функции `print`.

Пример.

```
1 a = 1
2 b = 5
3 c = 7
4 print('a=%1d, b=%1d, c=%1d.' % (a, b, c))
```

Листинг 17: *Использование функции print()*

Результат:

a=1, b=5, c=7.

Как видно из примера, сначала внутри строки мы записываем шаблоны для вывода значения переменной (у нас все три переменные целочисленные, состоят из одной цифры и для их вывода мы указываем шаблон `%1d`). Здесь параметр 1 означает, что при выводе выделяется место для размещения одной

цифры, параметр `d` говорит о том, что при выводе будет использовано целое число. После строки записывается оператор `%` и затем указывается кортеж, содержащий значения переменных, которые будут подставлены в соответствующие места строки.

Например, если для вывода нужно использовать три позиции, то запишем

Пример.

```
1 a = 1
2 b = 5
3 c = 7
4 print('a=%3d, b=%3d, c=%3d.' % (a, b, c))
```

Листинг 18: *Использование функции print()*

Результат:

```
a=  1, b=  5, c=  7.
```

Наиболее удобным способом форматирования вывода является применение `f` - строк («formatted string»). Этот способ стал доступен в версии python 3.6. `F`-строки позволяют использовать переменные внутри строки (при этом переменные заключаются в фигурные скобки).

Пример.

```
1 a = 1
2 b = 5
3 c = 7
4 print(f'a={a}, b={b}, c={c}.')
```

Листинг 19: *Использование f-строк*

Результат:

```
a=1, b=5, c=7.
```

`F` - строки позволяют гибко настраивать вывод чисел. Для этого после имени переменной в фигурных скобках ставится двоеточие и указывается шаблон форматирования.

Пример.

```
1 a = 1.1224354
2 b = 5.43543654
3 c = 7.464564564
4 print(f'a={a:.3f}, b={b:.4f}, c={c:.5f}.')
```

Листинг 20: Использование f-строк

Результат:

a=1.122, b=5.4354, c=7.46456.

В приведенном примере переменная `a` выводится с тремя знаками после запятой, переменная `b` – с четырьмя, переменная `c` – с пятью. Подробнее про шаблоны форматирования можно прочитать в документации python.

1.7. Математические функции (math)

Для работы с математическими функциями необходимо в начало программы добавить строку `import math`

<code>pow(x,n)</code>	x^n
<code>sqrt(x)</code>	$\sqrt{x}$
<code>cbrt(x)</code>	$\sqrt[3]{x}$
<code>fabs(x)</code>	$ x $
<code>exp(x)</code>	e^x
<code>log(x)</code>	$\ln x$
<code>log10(x)</code>	$\log_{10} x$
<code>log2(x)</code>	$\log_2 x$
<code>sin(x)</code>	$\sin x$ (x в радианах)
<code>cos(x)</code>	$\cos x$ (x в радианах)
<code>tan(x)</code>	$\tan x$ (x в радианах)
<code>asin(x)</code>	$\arcsin x$ (в радианах)
<code>acos(x)</code>	$\arccos x$ (в радианах)
<code>atan(x)</code>	$\arctan x$ (в радианах)
<code>ceil(x)</code>	Округление x сверху. Возвращает наименьшее целое значение, которое не меньше x
<code>floor(x)</code>	Округление x снизу. Возвращает наибольшее целое значение, которое не больше x
<code>round(x)</code>	Округление x до ближайшего целого (метод симметричного округления)
<code>trunc(x)</code>	Округление x до ближайшего целого (метод отбрасывания дробной части)
<code>fmod(a,b)</code>	Остаток от деления a на b

Таблица 5: Математические функции

`math.pi` — число π

`math.e` — число e

1.8. Ветвление в программе

Условный оператор `if`

Сокращенный вариант:

```
1  r = a if x else b
```

Листинг 21: Условный оператор *if* (сокращенная запись)

Если истинно условие `x` то выполняется `a`, иначе `b`.

Развернутый вариант:

```
1  if expression1:
2      code11
3      code12
4      ...
5  elif expression2:
6      code21
7      code22
8      ...
9  else:
10     code31
11     code32
12     ...
13
```

Листинг 22: Условный оператор *if*

Блоки `elif` и `else` являются необязательными.

Внутри каждого блока `if`, `elif` или `else` могут быть размещены вложенные конструкции оператора `if`.

Уровень вложенности определяется количеством пробелов.

```

1  if expression1:
2      if expression2:
3          code2
4          ...
5      code1
6      ...
7  elif expression3:
8      code3
9      ...
10 else:
11     code4
12     ...
13

```

Листинг 23: Вложенный *if*

Пример. Написать программу для нахождения наибольшего из двух целых чисел. Числа вводятся с клавиатуры.

```

1  a = int(input())
2  b = int(input())
3  if a > b:
4      print(a)
5  else:
6      print(b)
7

```

Листинг 24: Пример использования оператора *if*

1.9. Циклы

Циклы предназначены для многократного выполнения некоторого блока команд

Цикл *for*

```

1  for element in iter_object:
2      code
3      ...
4

```

Листинг 25: Цикл *for*

Цикл *for* позволяет пройти по всем элементам итерируемого объекта *iter_object*, используя переменную *element*.

Пример. Вывести на экран числа от 1 до 10.

```
1  for i in range(1,11):
2      print(i)
3  else:
4      code2
5      ...
```

Листинг 26: Пример использования цикла *for*

Результат:

```
1
2
3
4
5
6
7
8
9
10
```

Здесь функция `range(1,11)` генерирует целые числа от 1 до 10.

Цикл `while`

```
1  while condition:
2      code1
3      ...
4  else:
5      code2
6      ...
```

Листинг 27: Цикл *while*

В блоке `condition` записывается условие выполнения цикла (блок `code` будет повторяться, пока значение выражения, записанного в блоке `condition`, равно `true`).

Пример. Вывести на экран числа от 1 до 10.

```
1  i = 1
2  while i <= 10:
3      print(i)
4      i+=1
```

Листинг 28: Пример использования цикла *while*

Результат:

```
1
2
3
4
5
6
7
8
9
10
```

В циклах можно использовать инструкции `break` и `continue`. Инструкция `break` прерывает работу цикла. Инструкция `continue` осуществляет переход к следующему шагу цикла.

Инструкция `else` выполняется, если цикл завершился без использования инструкции `break`.

Кроме этого, можно использовать инструкцию `pass`, которая ничего не делает.

1.10. Последовательности

Последовательности в Python напоминают массивы в других языках программирования. Последовательности могут содержать разнородные объекты, изменять размер и осуществлять итерации по элементам.

К последовательностям относятся списки, строки и кортежи. Доступ к элементам последовательности можно осуществлять с помощью индексов (допускаются отрицательные). Используя индексы, можно получить доступ сразу к нескольким элементам последовательности (извлечение среза). Для этого используются три индекса (например, `i`, `j` и `h`), разделенные двоеточием. Индекс `h` указывает шаг, с которым осуществляется доступ к индексам последовательности. Элемент с индексом `j` не включается в срез. Если отсутствует индекс `i`, то извлекается срез от начала до элемента с индексом `j`. Если отсутствует индекс `j`, то извлекается срез от элемента с индексом `i` до конца. Индексация элементов списка начинается с нуля. С помощью знака `+` можно осуществить конкатенацию последовательностей, знака `*` – дублирование. Операторы `in` и `not in` позволяют осуществить проверку на вхождение. С помощью встроенной функции `len()` можно узнать длину последовательности. В Python можно использовать инструкцию присваивания последовательностей.

Списки

Списки могут содержать объекты любых типов, изменять содержимое и размер.

```
1  # Создать список L
2  L = [3, 1, 5, 'a', 'b', 'cd', 2.5, 5.5, 7.5]
3  print(L)
```

Листинг 29: Пример работы со списками

Результат:

```
[3, 1, 5, 'a', 'b', 'cd', 2.5, 5.5, 7.5]
```

```
1  # Срез, содержащий элементы с индексами 1-4
2  print(L[1:5])
```

Листинг 30: Пример работы со списками

Результат:

```
[1, 5, 'a', 'b']
```

```
1  # Срез, содержащий элементы от начала списка
2  # до элемента с индексом 6
3  print(L[:7])
```

Листинг 31: Пример работы со списками

Результат:

```
[3, 1, 5, 'a', 'b', 'cd', 2.5]
```

```
1  # Срез, содержащий элементы от элемента
2  # с индексом 3 до конца списка
3  print(L[3:])
```

Листинг 32: Пример работы со списками

Результат:

```
['a', 'b', 'cd', 2.5, 5.5, 7.5]
```

```
1  # Срез, содержащий каждый третий элемент списка
2  print(L[::3])
```

Листинг 33: Пример работы со списками

Результат:

[3, 'a', 2.5]

```
1 # Добавить элемент со значением 7 в конец списка
2 L.append(7)
3 print(L)
```

Листинг 34: Пример работы со списками

Результат:

[3, 1, 5, 'a', 'b', 'cd', 2.5, 5.5, 7.5, 7]

```
1 # Удалить элемент со значением 7
2 L.remove(7)
3 print(L)
```

Листинг 35: Пример работы со списками

Результат:

[3, 1, 5, 'a', 'b', 'cd', 2.5, 5.5, 7.5]

```
1 # Удалить из списка элемент с индексом 5
2 L.pop(5)
3 'cd'
4 print(L)
```

Листинг 36: Пример работы со списками

Результат:

[3, 1, 5, 'a', 'b', 2.5, 5.5, 7.5]

```
1 # Вставить в позицию 3 списка элемент со значением 'c'
2 L.insert(3, 'c')
3 print(L)
```

Листинг 37: Пример работы со списками

Результат:

```
[3, 1, 5, 'c', 'a', 'b', 2.5, 5.5, 7.5]
```

```
1 # Извлечь из списка последний элемент
2 print(L[-1])
```

Листинг 38: Пример работы со списками

Результат:

```
7.5
```

```
1
2 # Объединить списки list и [4, 'f']
3 L.extend([4, 'f'])
4 print(L)
```

Листинг 39: Пример работы со списками

Результат:

```
[3, 1, 5, 'c', 'a', 'b', 2.5, 5.5, 7.5, 4, 'f']
```

```
1 # Отсортировать список по возрастанию
2 L = [3, 1, 5, 2.5, 5.5, 7.5]
3 L.sort()
4 print(L)
```

Листинг 40: Пример работы со списками

Результат:

```
[1, 2.5, 3, 5, 5.5, 7.5]
```

```
1 # Отсортировать список по убыванию
2 L.reverse()
3 print(L)
```

Листинг 41: Пример работы со списками

Результат:

```
[7.5, 5.5, 5, 3, 2.5, 1]
```

Очень легко создавать вложенные списки

```
1 L1 = [['a', 'b', 'c'], [1, 2, 3], [1.5, 2.5, 3.5]]
2 print(L1)
3 print(L1[0])
4 print(L1[1][1])
```

Листинг 42: Пример работы со списками

Результат:

```
 [['a', 'b', 'c'], [1, 2, 3], [1.5, 2.5, 3.5]]
 ['a', 'b', 'c']
 2
```

Со списками удобно работать, используя выражения генераторов списков

```
1 L = [i for i in range(10)]
2 print(L)
3
4 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
5
6 [i+1 for i in L]
7
8 [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
9
10 L1 = [['a', 'b', 'c'], [1, 2, 3], [1.5, 2.5, 3.5]]
11 L2 = [row[1] for row in L1]
12 print(L2)
13
14 ['b', 2, 2.5]
15
```

Листинг 43: Пример работы со списками

Строки

Строки, как и списки, являются последовательностями. В отличие от списков, строки не могут изменять содержимое и размер. При попытке изменении строки в действительности создается модифицированная новая строка. Строка объявляется с помощью одинарных или двойных кавычек. Кроме этого, можно применять тройные кавычки, которые позволяют использовать в строках специальные символы без использования дополнительных обозначений. Это особенно удобно при отображении блоков текста. Для строк суще-

ствует большое количество методов, описание которых можно найти в документации.

```
1 S = 'numerical methods'; print(S)
2 'numerical methods'
```

Листинг 44: Пример работы со строками

```
1 S = 'numerical methods'
2 print(S)
3 'numerical methods'
4 print(len(S))
5 17
6 print(S.title())
7 'Numerical Methods'
8 print(S.capitalize())
9 'Numerical methods'
10 print(S.upper())
11 'NUMERICAL METHODS'
12 print(S.lower())
13 'numerical methods'
14 print(S.find('methods'))
15 10
16 print(S.count('m'))
17 2
18 S = S+'!'
19 print(S)
20 'numerical methods!'
21 print(S.replace('!', '.'))
22 'numerical methods.'
23 print(S[:9])
24 'numerical'
25 print(S[10:])
26 'methods!'
```

Листинг 45: Пример работы со строками

Кортежи

Кортежи являются последовательностями. При создании кортежи заключаются в круглые скобки. Кортежи не позволяют изменять свои значения и не имеют методов.

Присваивание последовательностей

В Python одной последовательности можно присвоить другую последовательность значений. Присваивание происходит в соответствии с позициями элементов в последовательностях.

Присваивание кортежей.

```
1 >>> a, b = 'c', 'd'
2 >>> a, b
3 ('c', 'd')
```

Листинг 46: Присваивание кортежей

Это равносильно командам `a = 'c'; b = 'd'`.

Присваивание строки кортежу

```
1 >>> a, b, c = 'ijk'
2 >>> a, b, c
3 ('i', 'j', 'k')
```

Листинг 47: Присваивание строки кортежу

Присваивание кортежа списку

```
1 >>> [A, B] = (1, 2)
2 >>> A, B
3 (1, 2)
```

Листинг 48: Присваивание кортежа списку

Присваивание последовательности целых чисел переменным a,b,c

```
1 >>> a, b, c = range(3)
2 >>> a, b, c
3 (0, 1, 2)
```

Листинг 49: Присваивание последовательности целых чисел переменным a,b,c

Обмен значений переменных с помощью кортежей

```
1     >>> a, b = 1, 2
2     >>> a, b
3     (1, 2)
4     >>> a, b = b, a
5     >>> a, b
6     (2, 1)
```

Листинг 50: *Обмен значений переменных с помощью кортежей*

Можно осуществлять присваивание одного объекта нескольким переменным

```
1     >>> a = b = c = 1
2     >>> a
3     1
4     >>> b
5     1
6     >>> c
7     1
8     >>>
```

Листинг 51: *присваивание одного объекта нескольким переменным*

Это равносильно

```
1     >>> c = 1
2     >>> b = c
3     >>> a = b
4     >>>
```

Листинг 52: *присваивание одного объекта нескольким переменным*

Такое присваивание удобно выполнять для неизменяемых объектов. Следует учитывать, что если присваивается изменяемый объект, то при его изменении с помощью одной переменной изменяются и другие переменные.

```

1     >>> a = b = c = []
2     >>> a.append(1)
3     >>> a
4     [1]
5     >>> b
6     [1]
7     >>> c
8     [1]

```

Листинг 53: *присваивание одного объекта нескольким переменным*

1.11. Функции

Функция – это фрагмент кода, который может быть вызван из другой части программы.

Для того чтобы использовать функцию, необходимо ее объявить или определить до первого вызова.

Можно создавать два типа функций:

1. Функции, не возвращающие значения.
2. Функции, возвращающие значение.

```

1     def <name>(<arguments>):
2         <code>
3         ...

```

Листинг 54: *Функции*

Очень часто тело функции содержит необязательную инструкцию `return`. Эта инструкция завершает работу функции и возвращает результат.

```

1     def summa(a, b, c):
2         return a + b + c
3
4
5     print(summa(1, 2, 4))

```

Листинг 55: *Функции*

Результат:

7

Тип значения, которое возвращает функция, определяется типами ее аргументов

```

1  def summa(a, b, c):
2      return a + b + c
3
4
5  print(summa('a', 'b', 'c'))

```

Листинг 56: *Функции*

Результат:

abc

Аргументы могут передаваться функции с использованием позиции и с использованием ключей. Во втором случае при вызове функции используется форма <имя функции>(<имя аргумента>=<значение аргумента>).

```

1  def summa(a, b, c):
2      return a + b + c
3
4
5  print(summa(c=4, b=2, a=1))

```

Листинг 57: *Функции*

Результат:

7

При объявлении функции можно использовать значения по умолчанию. Те аргументы, для которых заданы значения по умолчанию, можно не указывать при вызове функции.

```

1  def summa(a, b=2, c=4):
2      return a + b + c
3
4
5  print(summa(1))
6  print(summa(1, 2))
7  print(summa(1, 2, 4))

```

Листинг 58: *Функции*

Результат:

7 7 7

При объявлении и вызове функции перед именем аргумента можно использовать символы * и **. Использование символов * и ** при вызове функции позволяет передавать аргументы в виде кортежа или словаря. Если сим-

волы * и ** используются при объявлении функции, то они позволяют функции принимать произвольное количество аргументов в виде кортежа или словаря.

Пример передачи аргументов функции в виде кортежа

```
1 def summa(a, b, c):  
2     return a + b + c  
3  
4  
5 args=(1, 2, 4)  
6 print(summa(args))
```

Листинг 59: *Функции*

Результат:

7

Пример передачи аргументов функции в виде словаря

```
1 def summa(a, b, c):  
2     return a + b + c  
3  
4  
5 kwargs={'a': 1, 'b': 2, 'c': 4}  
6 print(summa(**kwargs))
```

Листинг 60: *Функции*

Результат:

7

Пример передачи функции произвольного числа аргументов в виде кортежа

```
1 def func1(*args):  
2     print(args)  
3  
4  
5 func1()  
6 func1(1)  
7 func1(1, 2)  
8 func1(1, 2, 'c', 2.5)
```

Листинг 61: *Функции*

Результат:

```
()  
(1,)   
(1, 2)   
(1, 2, 'c', 2.5)
```

В качестве примера объявим функцию `summa`, вычисляющую сумму произвольного числа аргументов

```
1  def summa(*args):  
2      s = 0.0  
3      for i in args:  
4          s = s + float(i)  
5      return s  
6  
7  
8  print(summa())  
9  print(summa(1))  
10 print(summa(1, 2, 3.5, '5'))
```

Листинг 62: *Функции*

Результат:

```
0.0  
1.0  
11.5
```

Пример передачи функции произвольного числа аргументов в виде словаря

```
1  def func2(**kwargs):  
2      print(kwargs)  
3  
4  
5  func2()  
6  func2(a=1)  
7  func2(a=1, b='text', c=1.5)
```

Листинг 63: *Функции*

Результат:

```
{}  
{'a': 1}  
{'a': 1, 'b': 'text', 'c': 1.5}
```

```
1  def summa(**kwargs):  
2      s = 0.0  
3      for i in kwargs.values():  
4          s = s + float(i)  
5      return s  
6  
7  
8  print(summa())  
9  print(summa(a=1))  
10 print(summa(a=1, b=2, c=4))
```

Листинг 64: *Функции*

Результат:

```
0.0  
1.0  
7.0
```

При объявлении функций можно комбинировать обычные аргументы и аргументы со знаками `*` и `**`. Например, функция `func(a,*args)` требует наличия одного обязательного аргумента `a` и произвольного количества необязательных аргументов в виде кортежа `args`. Функция `func(*args,**kwargs)` позволяет принимать произвольное количество аргументов в виде кортежа и (или) словаря.

Анонимные функции

Анонимные функции не имеют имени и состоят из единственного выражения, значение которого является возвращаемым значением функции. Анонимные функции создаются с помощью ключевого слова `lambda` и используются в виде: `lambda <аргументы>: <выражение>`. Их удобно применять для создания небольших функций. Анонимные функции являются выражением, в отличие от инструкции `def`. Вследствие этой особенности `lambda`-выражения можно применять в тех участках кода, где нельзя использовать `def`. Анонимные функции можно присваивать переменным. В начале записывается

заголовок функции. Здесь `type` - это тип данных, `function_name` - имя функции. После имени функции в круглых скобках через запятую указываются передаваемые аргументы. Перед именем каждого аргумента указывается его тип данных. Переменные, которые используются внутри функции для хранения значений аргументов, называются параметрами. Когда вызов функции произведен, параметры автоматически инициализируются значениями, переданными в функцию.

```
1 a = [lambda x: x, lambda x: 2 * x]
2 for i in a:
3     print(i(2))
```

Листинг 65: *Функции*

Результат:

2
4

1.12. Множества

Множества позволяют создавать последовательность из неповторяющихся и неупорядоченных элементов. Индексация элементов отсутствует. Создать множество можно, перечислив его элементы в фигурных скобках через запятую или с помощью ключевого слова `set`.

```
1 a = {1, 2, 3, 3} # Создание множества
2 print(a)
```

Листинг 66

Результат:

{1, 2, 3}

```
1 b = set([1, 1, 2, 3, 4, 5]) # Создание множества из списка
2 print(b)
```

Листинг 67

Результат:

{1, 2, 3, 4, 5}

```

1  c = a.intersection(b)  # Пересечение множеств a и b
2  print(c)
3  {1, 2, 3}
4  d = a.union(b)  # Объединение множеств a и b
5  print(d)
6  {1, 2, 3, 4, 5}

```

Листинг 68

1.13. Словари

Словари являются отображениями и состоят из множества пар <ключ>:<значения>. Они напоминают списки, но в отличие от них, доступ к элементам осуществляется по ключам. Для создания словаря используются фигурные скобки { } .

```

1  D = {'a': 'A', 'b': 1, 'c': 'C'}  # Создание словаря
2  print(D)
3  {'a': 'A', 'b': 1, 'c': 'C'}
4  print(D.get('a'))  # Возвратить значение для ключа 'a'
5  'A'
6  print(D.get('b'))  # Возвратить значение для ключа 'b'
7  1
8  # При отсутствии пары с ключом 'e' дополнить словарь
9  print(D.setdefault('e',2))
10 2
11 print(D)
12 {'a': 'A', 'c': 'C', 'b': 1, 'e': 2}
13 print(D.keys())  # Возвратить ключи словаря
14 dict_keys(['a', 'b', 'e', 'c'])
15 print(D.values())  # Возвратить значения словаря
16 dict_values(['A', 1, 2, 'C'])
17 print(D.items())  # Возвратить кортежи пар ключ-значение
18 dict_items([('a', 'A'), ('b', 1), ('e', 2), ('c', 'C')])

```

Листинг 69

```

1 print(D)
2 {'a': 'A', 'c': 'C', 'b': 1, 'e': 2}
3
4 print(D.keys()) # Возвратить ключи словаря
5 dict_keys(['a', 'b', 'e', 'c'])
6
7 print(D.values()) # Возвратить значения словаря
8 dict_values(['A', 1, 2, 'C'])
9
10 print(D.items()) # Возвратить кортежи пар ключ-значение
11 dict_items([('a', 'A'), ('b', 1), ('e', 2), ('c', 'C')])
12
13 print(D.pop('e')) # Удалить из словаря пару с ключом 'e'
14 2
15
16 print(D)
17 {'a': 'A', 'b': 1, 'c': 'C'}

```

Листинг 70

1.14. Модули и пакеты

Модули

Модули позволяют структурировать программы, упрощать повторное использование кода, избегать конфликтов имен. Python содержит огромную коллекцию модулей (стандартная библиотека). Модули можно объединять в пакеты. Для получения доступа к атрибутам модуля его необходимо импортировать. Импортирование осуществляется с помощью оператора `import`. Можно использовать две формы этого оператора – `import` и `from-import`. В первом случае в глобальное пространство имен импортируется имя модуля, поэтому при доступе к атрибуту необходимо записывать имя модуля и через точку имя атрибута. Второй вариант импортирует в глобальное пространство имен имя атрибута, что позволяет обращаться к нему напрямую. Оба варианта загружают весь модуль целиком. При импорте можно связать атрибут с другим именем. Для получения прямого доступа ко всем атрибутам модуля можно использовать знак `*`.

```
1 >>> import math
2 >>> print math.sin(0)
3 0.0
4 >>> print math.cos(0)
5 1.0
6 >>>
```

Листинг 71

```
1 >>> from math import sin
2 >>> sin(0)
3 0.0
```

Листинг 72

```
1 >>> from math import sin as mysin
2 >>> mysin(0)
3 0.0
4 >>>
```

Листинг 73

```
1 >>> from math import *
2 >>> sin(0), cos(0)
3 (0.0, 1.0)
4 >>>
```

Листинг 74

Для избежания конфликтов имен желательно как можно меньше использовать инструкцию `from`. Для обновления модуля можно использовать функцию `reload`.

У каждого модуля существует встроенный атрибут `__name__`, который позволяет использовать файл в качестве модуля и в качестве исполняемого файла. Если файл выполняется, то атрибуту `__name__` присваивается значение `__main__`. Если файл используется как модуль, то атрибуту `__name__` присваивается имя модуля. Например, создадим файл `test.py`, содержащий

```

1 def hello():
2     print 'Hello, World.'
3
4 if __name__ == '__main__':
5     hello()

```

Листинг 75

```

1 python
2 >>> import test
3 >>> test.hello()
4 Hello, World.
5 >>>

```

Листинг 76

Этот файл можно использовать как модуль и как отдельную программу

```

1 python test.py
2
3 Hello, World.

```

Листинг 77

Пакеты

Пакеты позволяют организовать структурирование модулей. Фактически пакет — это каталог, содержащий подкаталоги (подпакеты), внутри которых находятся модули. Доступ к модулям осуществляется с использованием точки. Например, пакет `package` имеет следующую структуру:

```

package/
    __init__.py
    module1.py
    module2.py
    module3.py

    package1/
        __init__.py
        module11.py
        module12.py
        module13.py

    package2/
        __init__.py

```

```
module21.py
module22.py
module23.py

package3/
  __init__.py
  module31.py
  module32.py
  module33.py
```

Если необходимо получить доступ к функции `myfunc()`, находящейся в файле `module33.py`, то для импорта можно использовать команду

```
1  import package.package3.module33.myfunc()
```

Листинг 78

Каждый каталог, являющийся пакетом, должен содержать файл `__init__.py`. Программный код этого файла автоматически запускается при импорте пакета в первый раз.

1.15. Файлы

Базовый тип файл предоставляет возможность работы с внешними файлами на компьютере. Для создания объекта файл используется встроенная функция `open(<имя файла>, <режим доступа к файлу>)`. По умолчанию используется режим доступа `'r'` – чтение.

```
1  >>> # Запись данных в файл
2  >>> f = open('myfile.txt', 'w')
3  >>> f.write('string1\n')
4  >>> f.write('string2')
5  >>> f.close()
6  >>> # Чтение данных из файла
7  >>> f = open('myfile.txt')
8  >>> s = f.read() # Содержимое файла записывается в строку s
9  >>> print(s)
10 string1
11 string2
12 >>> f.close()
```

Листинг 79

1.16. Документирование программного кода

Самый простой способ документирования программного кода — использование комментариев. Для этого применяется знак `#`. Весь текст до конца строки, следующий за этим знаком, игнорируется.

Кроме этого, существуют другие способы документирования программного кода. В коде программы на Python можно использовать строки, которые автоматически помещаются в атрибут `__doc__` объекта и становятся доступны сразу после импортирования модуля. Например, файл `example.py` содержит код

```
1  """
2  Description of the module
3  """
4
5  def func1():
6      """
7      Decription of the function func1()
8      """
9      pass
10
11 def hello():
12     "Function hello print string 'Hello, World!'"
13     print 'Hello, World'
```

Листинг 80

Импортируем этот модуль

```
1  python
2  >>> import example
3  >>> print example.__doc__
4  Description of the module
5
6  >>> print example.func1.__doc__
7  Decription of the function func1()
8
9  >>> print example.hello.__doc__
10 Function hello print string 'Hello, World!'
11 >>>
```

Листинг 81

Для упрощения отображения документации удобно использовать встроенную функцию `help`.

```
1 python
2 >>> import example
3 >>> help(example)
4 Help on module example:
5 NAME
6     example - Description of the module
7 FILE
8     .../example.py
9 FUNCTIONS
10     func1()
11         Description of the function func1()
12     hello()
13         Function hello print string 'Hello, World!'
14 (END)
```

Листинг 82

Для прекращения работы встроенной функции `help` необходимо нажать клавишу `q`. Для получения списка атрибутов объекта можно использовать функцию `dir()`.

```
1 >>> dir(math)
2 ['__doc__', '__file__', '__name__', 'acos', 'asin', 'atan', 'atan2',
3  'ceil', 'cos', 'cosh', 'degrees', 'e', 'exp', 'fabs', 'floor',
4  'fmod', 'frexp', 'hypot', 'ldexp', 'log', 'log10', 'modf', 'pi',
5  'pow', 'radians', 'sin', 'sinh', 'sqrt', 'tan', 'tanh']
6 >>> dir([])
7 ['__add__', '__class__', '__contains__', '__delattr__', '__delitem__',
8  '__delslice__', '__doc__', '__eq__', '__ge__', '__getattr__',
9  '__getitem__', '__getslice__', '__gt__', '__hash__', '__iadd__',
10  '__imul__', '__init__', '__iter__', '__le__', '__len__', '__lt__',
11  '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__',
12  '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__',
13  '__setslice__', '__str__', 'append', 'count', 'extend', 'index',
14  'insert', 'pop', 'remove', 'reverse', 'sort']
```

Листинг 83

Список атрибутов модуля хранится в виде словаря. Для получения этого списка можно использовать атрибут `__dict__`.

```

1 >>> math.__dict__.keys()
2 ['pow', 'cosh', 'ldexp', 'hypot', 'tan', 'asin', 'log', 'fabs', 'floor',
3 'sqrt', 'frexp', 'degrees', 'pi', 'log10', '__doc__', 'fmod', 'atan',
4 '__file__', 'ceil', 'sinh', '__name__', 'cos', 'e', 'tanh', 'radians',
5 'sin', 'atan2', 'modf', 'exp', 'acos']
6 >>>

```

Листинг 84

1.17. Математические пакеты numpy и scipy

Пакеты NumPy (<http://numpy.scipy.org>) и SciPy (<http://www.scipy.org>) являются двумя наиболее известными расширениями языка Python для проведения научных расчетов.

Пакет NumPy включает модули для эффективной работы с многомерными массивами, матрицами, функции для генерации псевдослучайных чисел, функции, реализующие алгоритмы линейной алгебры, преобразование Фурье, возможности интеграции программ на языках С и Фортран.

Пакет SciPy базируется на пакете NumPy и расширяет его возможности. Например, включает модули для численного интегрирования, статистического анализа, решения обыкновенных дифференциальных уравнений, обработки сигналов и изображений, оптимизационные алгоритмы,...

Далее рассмотрим подробнее некоторые возможности пакета NumPy.

NumPy добавляет в Python поддержку большого количества новых числовых типов: bool (True или False); int (целые числа, int32 или int64); int8 (целые числа от -128 до 127); int16 (целые числа -32768 до 32767); int32 (целые числа от -2147483648 до 2147483647); int64 (целые числа от 9223372036854775808 до 9223372036854775807); uint8 (целые числа без знака от 0 до 255); uint16 (целые числа без знака 0 до 65535); uint32 (целые числа без знака 0 до 4294967295); uint64 (целые числа без знака 0 до 18446744073709551615); float (краткая форма для float64); float32 (вещественные числа одинарной точности); float64 (вещественные числа двойной точности); complex (краткая форма для complex128); complex64 (комплексные числа, действительная и мнимая части представлены числами типа float32); complex128 (комплексные числа, действительная и мнимая части представлены числами типа float64).

Одним из основных объектов NumPy является ndarray. Он позволяет со-

здавать многомерные массивы данных одного типа и выполнять операции над ними с большой скоростью. В отличие от последовательностей в Python, массивы в NumPy имеют фиксированный размер, элементы массива должны быть одного типа. К массивам можно применять различные математические операции, которые выполняются более эффективно, чем для последовательностей Python.

```
1  >>> import numpy
2  >>> a = numpy.array([1, 2, 3]) #массив целых чисел
3  >>> print a
4  [1 2 3]
5
6  >>> b = numpy.array([1, 2, 3.0]) #массив действительных чисел
7  >>> print b
8  [ 1.  2.  3.]
9
10 >>> c = numpy.array([1, 2, 3.0+2j]) #массив комплексных чисел
11 >>> print c
12 [ 1.+0.j  2.+0.j  3.+2.j]
13 >>>
```

Листинг 85

Размерность массива можно определить с помощью атрибута `shape`, тип элементов — с помощью `dtype`.

```
1  >>> import numpy
2  >>> d = numpy.array([[1, 2, 3], [3, 4, 5]])
3  >>> print d
4  [[1 2 3]
5   [3 4 5]]
6
7
8  >>> d.shape
9  (2, 3)
10 >>> d.dtype
11 dtype('int32')
12 >>> type(d)
13 <type 'numpy.ndarray'>
14 >>>
```

Листинг 86

При создании массива можно явно указать тип его элементов

```

1  >>> import numpy
2  >>> e = numpy.array([[1, 2, 3], [3, 4, 5]], dtype=float)
3  >>> e
4  array([[ 1.,  2.,  3.],
5         [ 3.,  4.,  5.]])
6
7  >>>

```

Листинг 87

В NumPy присутствуют встроенные функции для создания массивов специального вида. Функция `arange()` возвращает массив чисел с увеличивающимися значениями.

```

1  >>> import numpy
2  >>> numpy.arange(10)
3  array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
4
5  >>> numpy.arange(1, 10, 2)
6  array([1, 3, 5, 7, 9])
7
8  >>> numpy.arange(1, 10, 2, dtype=float)
9  array([ 1.,  3.,  5.,  7.,  9.])
10
11 >>>

```

Листинг 88

Функция `linspace()` возвращает массив из n элементов на заданном отрезке, с одинаковым расстоянием между элементами. По умолчанию $n=50$.

```

1  >>> import numpy
2  >>> numpy.linspace(0, 5, 3)
3  array([ 0. ,  2.5,  5. ])
4
5  >>> numpy.linspace(0, 5, 6)
6  array([ 0.,  1.,  2.,  3.,  4.,  5.])
7
8  >>>

```

Листинг 89

Функция `zeros()` возвращает массив, состоящий из нулей.

```

1 >>> import numpy
2 >>> numpy.linspace(0, 5, 3)
3 array([ 0. ,  2.5,  5. ])
4 >>> numpy.linspace(0, 5, 6)
5 array([ 0.,  1.,  2.,  3.,  4.,  5.])
6 >>>

```

Листинг 90

Функция `ones()` возвращает массив, состоящий из единиц.

```

1 >>> import numpy
2 >>> numpy.ones((3, 3))
3 array([[ 1.,  1.,  1.],
4         [ 1.,  1.,  1.],
5         [ 1.,  1.,  1.]])

```

Листинг 91

Функция `identity()` возвращает массив, в котором $a_{ij} = 1$.

```

1 >>> import numpy
2 >>> numpy.identity(3)
3 array([[ 1.,  0.,  0.],
4         [ 0.,  1.,  0.],
5         [ 0.,  0.,  1.]])
6 >>>

```

Листинг 92

Функция `dot(a,b)` возвращает произведение массивов, `linalg.det(a)` — вычисляет определитель массива `a`, `linalg.inv(a)` — возвращает обратную матрицу к матрице `a`, `linalg.solve(a,b)` — возвращает решение системы линейных алгебраических уравнений. Для создания матриц в NumPy существует специальный класс `matrix`, унаследованный от `ndarray`. Объекты `matrix` всегда являются двумерными. Для создания объектов `matrix` можно использовать Matlab-подобный синтаксис (столбцы разделяются пробелами, строки — `;`). Вместо `matrix` можно использовать сокращение `mat`. В отличие от `ndarray`, произведение объектов `matrix` является обычным произведением матриц. Атрибут `.T` позволяют найти транспонированную матрицу, `.I` — обратную матрицу.

```

1  >>> import numpy
2  >>> a = numpy.matrix('1 0;0 1')
3  >>> b = numpy.matrix('1 2;3 4')
4  >>> a
5  matrix([[1, 0],
6           [0, 1]])
7  >>> b
8  matrix([[1, 2],
9           [3, 4]])
10 >>> a*b
11 matrix([[1, 2],
12          [3, 4]])
13 >>> b.T
14 matrix([[1, 3],
15          [2, 4]])
16 >>> b.I
17 matrix([[-2. ,  1. ],
18          [ 1.5, -0.5]])
19 >>>

```

Листинг 93

1.18. Визуализация данных с использованием matplotlib

Matplotlib — это библиотека Python для построения качественных двумерных графиков.

Рассмотрим некоторые примеры использования matplotlib.

Пример. Построить график функции $y = x^2$ на отрезке $[-10, 10]$ с шагом изменения аргумента 0.01 (рис. 2).

```

1  # -*- coding: utf8 -*-
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  x = np.arange(-10, 10.01, 0.01) # Массив значений аргумента
6  plt.plot(x, x**2) # Построение графика
7  plt.xlabel(r'$x$') # Метка по оси x в формате TeX
8  plt.ylabel(r'$f(x)$') # Метка по оси y в формате TeX
9  plt.title(r'$y=x^2$') # Заголовок в формате TeX
10 plt.grid(True) # Сетка
11 plt.show() # Показать график

```

Листинг 94

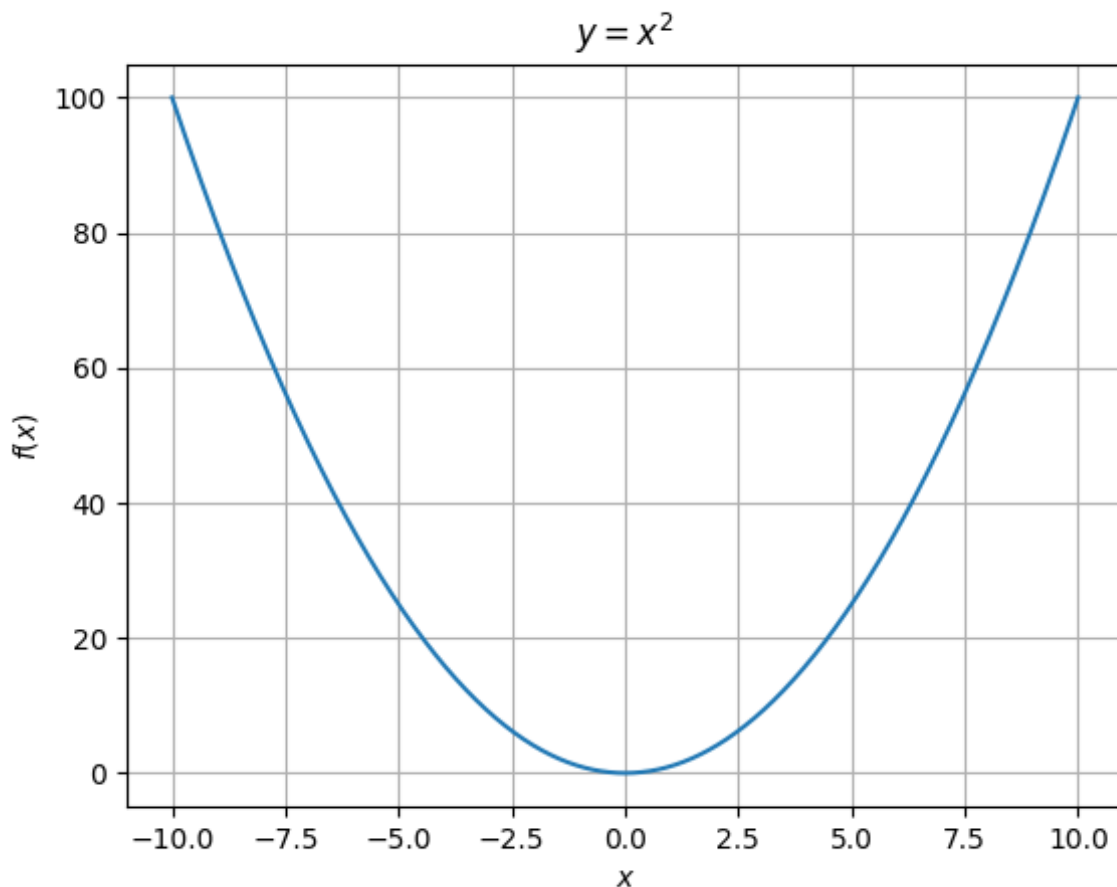


Рисунок 1.2. График функции $y = x^2$.

При построении графика можно использовать третий параметр, который позволяет изменять цвет и тип графика. Значение этого параметра по умолчанию равно 'b-', что означает сплошная синяя линия. Например, на рисунке 3 точки изображаются в виде кружков зеленого цвета.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.arange(-10, 11, 1)
5 plt.plot(x, x**2, 'go')
6 plt.show()
```

Листинг 95

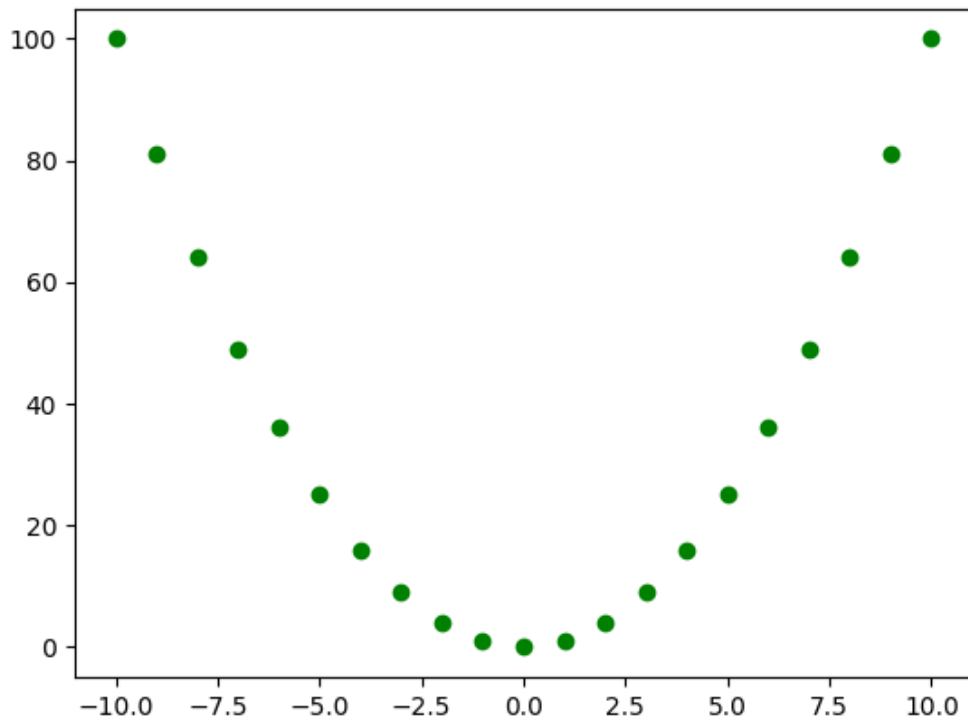


Рисунок 1.3. График функции $y = x^2$ с параметром 'go'.

Подробное описание различных стилей отображения графиков можно найти в документации.

Очень легко можно построить графики нескольких функций на одном рисунке (рис. 1.4).

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 x = np.arange(-10, 10.01, 0.01)
4 plt.plot(x, np.sin(x), x, np.cos(x), x, -x)
5 plt.xlabel(r'$x$')
6 plt.ylabel(r'$f(x)$')
7 plt.title(r'$f_1(x)=\sin(x), \ f_2(x)=\cos(x), \ f_3(x)=-x$')
8 plt.grid(True)
9 plt.show()
```

Листинг 96

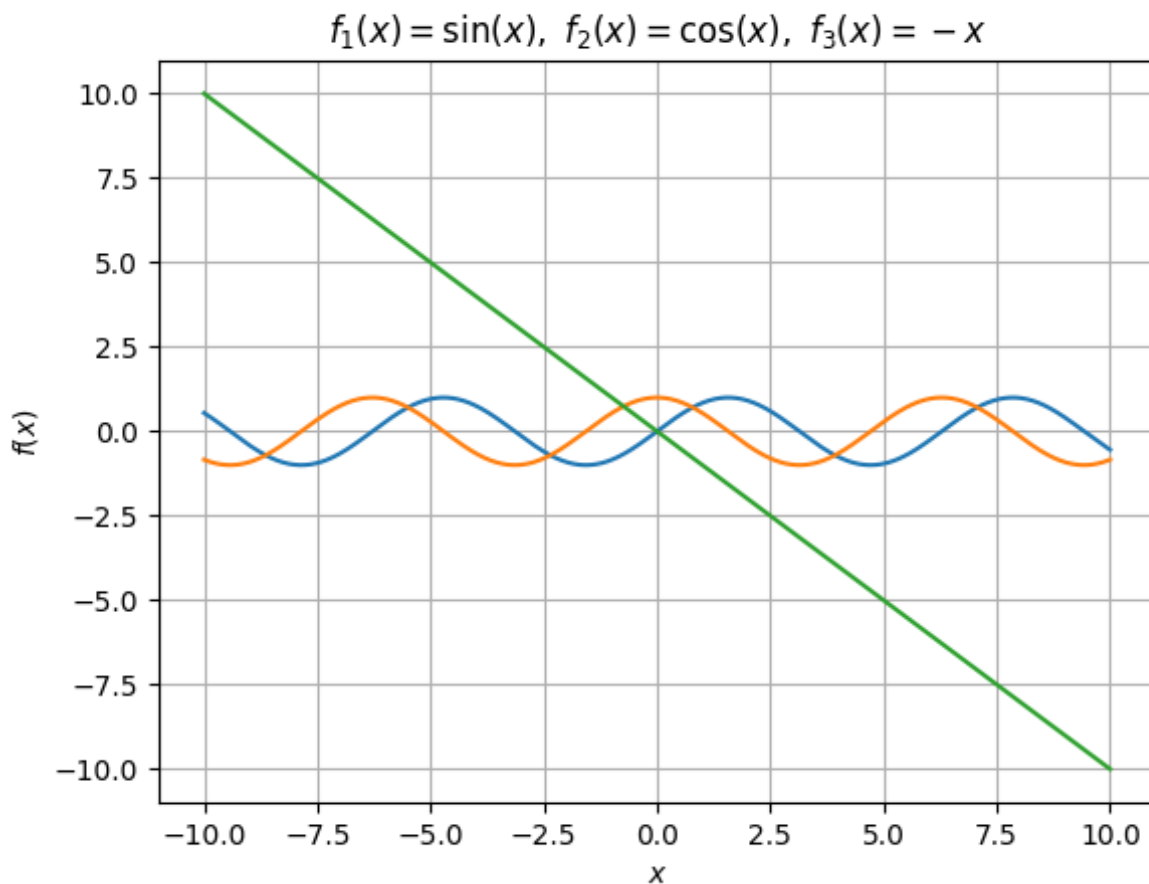


Рисунок 1.4. Графики нескольких функций.

Работа с `matplotlib` основана на использовании графических окон и осей (оси позволяют задать некоторую графическую область). Все построения применяются к текущим осям. Это позволяет изображать несколько графиков в одном графическом окне. По умолчанию создается одно графическое

окно `figure(1)` и одна графическая область `subplot(111)` в этом окне. Команда `subplot` позволяет разбить графическое окно на несколько областей. Она имеет три параметра: `nr`; `nc`; `nr`. Параметры `nr` и `nc` определяют количество строк и столбцов, на которые разбивается графическая область, параметр `nr` определяет номер текущей области (`nr` принимает значения от 1 до `nr*nc`). Если `nr*nc < 10`, то передавать параметры `nr, nc, nr` можно без использования запятой. Например, допустимы формы `subplot(2,2,1)` и `subplot(221)`.

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  x = np.arange(-10, 10.01, 0.01); t = np.arange(-10, 11, 1)
5
6  #subplot 1
7  plt.subplot(221)
8  plt.plot(x, np.sin(x))
9  plt.title(r'$\sin(x)$')
10 plt.grid(True)
11
12 #subplot 2
13 plt.subplot(222)
14 plt.plot(x, np.cos(x), 'g')
15 plt.axis('equal')
16 plt.grid(True)
17 plt.title(r'$\cos(x)$')
18
19 #subplot 3
20 plt.subplot(223)
21 plt.plot(x, x**2, t, t**2, 'ro')
22 plt.title(r'$x^2$')
23
24 #subplot 4
25 plt.subplot(224)
26 plt.plot(x, x)
27 plt.subplot(224).spines['left'].set_position('center')
28 plt.subplot(224).spines['bottom'].set_position('center')
29 plt.title(r'$x$')
30 plt.show()
```

Листинг 97

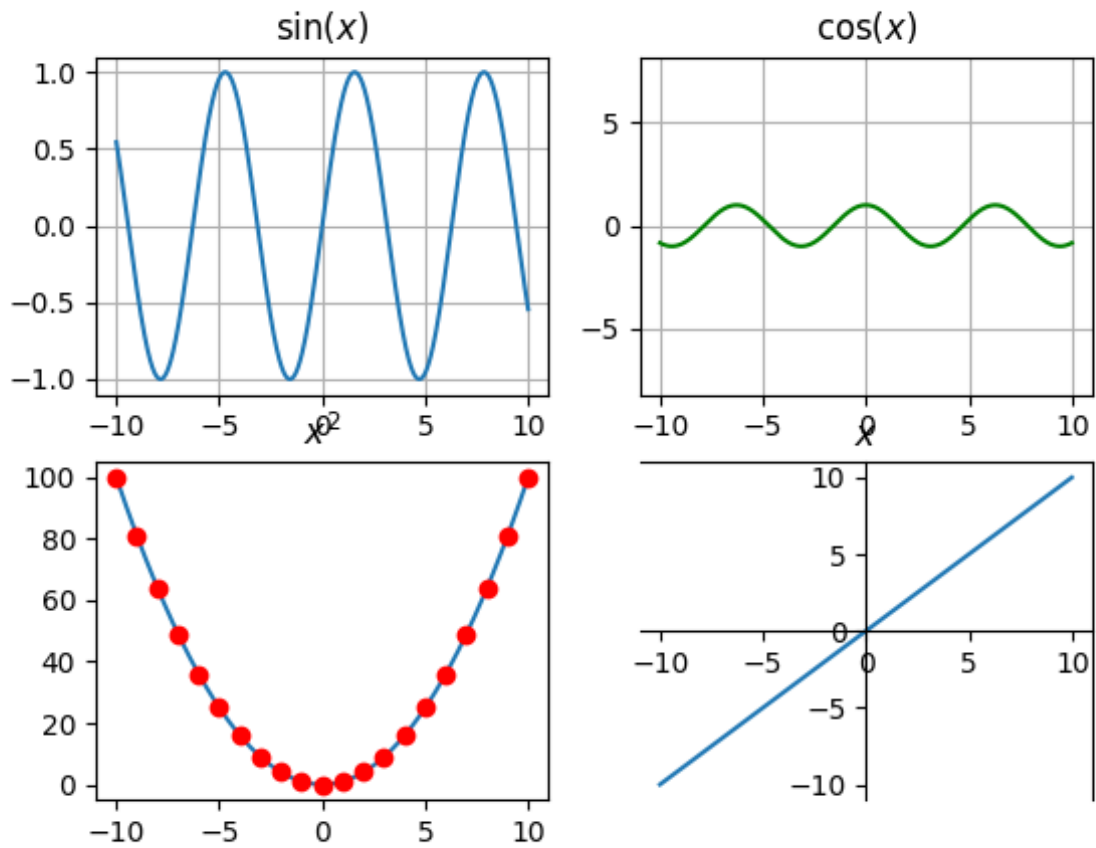


Рисунок 1.5. Пример использования нескольких графических областей.

Построение графика функции, заданной параметрически

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  t = np.arange(0, 2 * np.pi, 0.01); r = 4
4  plt.plot(r * np.sin(t), r * np.cos(t), lw=3)
5  plt.axis('equal'); plt.show()

```

Листинг 98

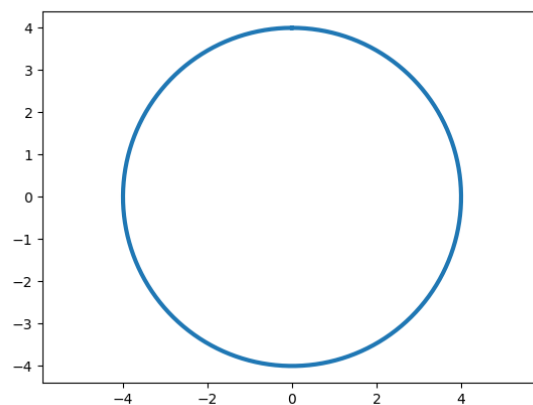


Рисунок 1.6. График функции, заданной параметрически.

Построение графика функции в полярной системе координат

```
1 import numpy as np
2
3 import matplotlib.pyplot as plt
4
5 plt.subplot(111, polar=True)
6
7 phi = np.arange(0, 2 * np.pi, 0.01)
8
9 rho = 2 * phi
10
11 plt.plot(phi, rho, lw=2)
12
13 plt.show()
```

Листинг 99

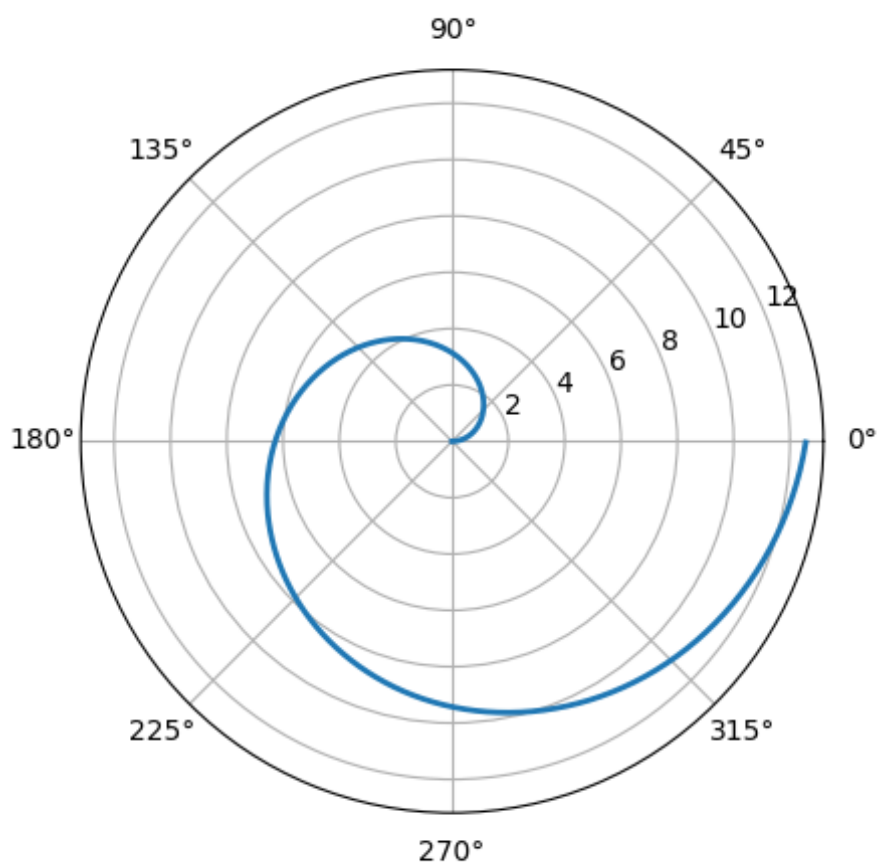


Рисунок 1.7. График в полярной системе координат.

Построение графика функции двух переменных

```
1 import matplotlib.pyplot as plt
2 from mpl_toolkits.mplot3d import axes3d
3 import numpy as np
4
5 fig = plt.figure()
6 ax = fig.add_subplot(projection='3d')
7 i = np.arange(-1, 1, 0.01)
8 X, Y = np.meshgrid(i, i)
9 Z = X**2 - Y**2
10 ax.plot_wireframe(X, Y, Z, rstride=10, cstride=10)
11 plt.show()
```

Листинг 100

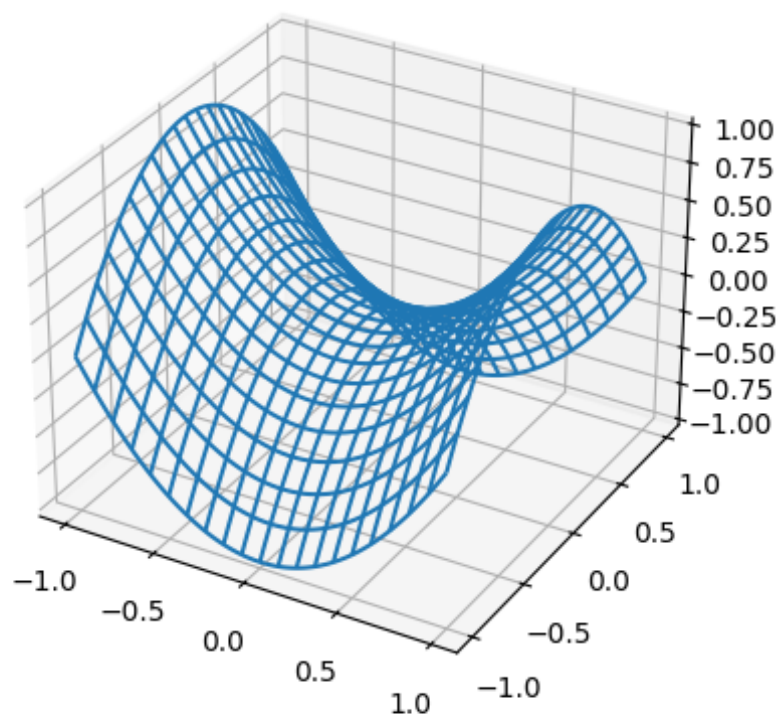


Рисунок 1.8. График функции двух переменных - 1.

```

1  import matplotlib.pyplot as plt
2  from mpl_toolkits.mplot3d import axes3d
3  import numpy as np
4
5  fig = plt.figure()
6  ax = fig.add_subplot(projection='3d')
7
8  i = np.arange(-1, 1, 0.01)
9
10 X, Y = np.meshgrid(i, i)
11 Z = X**2-Y**2
12
13 ax.plot_surface(X, Y, Z, rstride=10, cstride=10)
14
15 plt.show()

```

Листинг 101

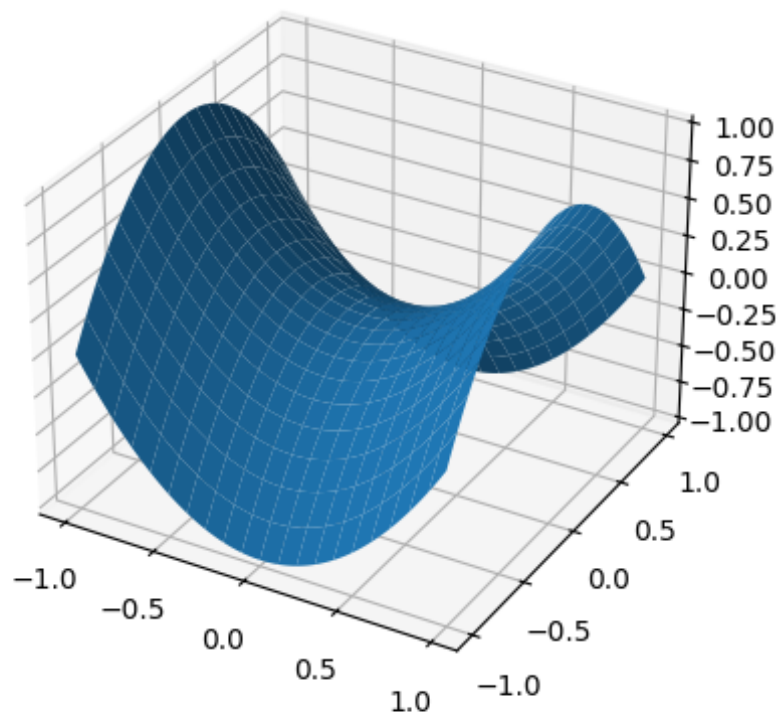


Рисунок 1.9. График функции двух переменных - 2.

Построение графика функции двух переменных, заданной параметрически

```

1  import matplotlib.pyplot as plt
2  from mpl_toolkits.mplot3d import axes3d
3  import numpy as np
4
5  fig = plt.figure()
6  ax = fig.add_subplot(projection='3d')
7
8  u = np.linspace(0, 2*np.pi, 100)
9  v = np.linspace(0, np.pi, 100)
10 r = 5
11 x = r * np.outer(np.cos(u), np.sin(v))
12 y = r * np.outer(np.sin(u), np.sin(v))
13 z = r * np.outer(np.ones(np.size(u)), np.cos(v))
14
15 ax.plot_surface(x, y, z, rstride=4, cstride=4)
16 plt.show()

```

Листинг 102

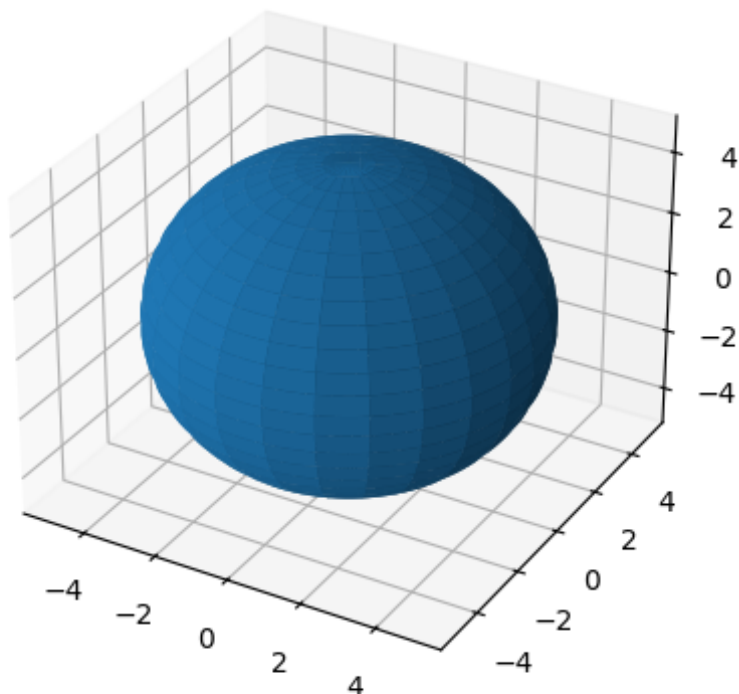


Рисунок 1.10. График функции двух переменных, заданной параметрически.

1.19. Объектно-ориентированное программирование

Основные черты ООП

- Абстракция
- Инкапсуляция
- Полиморфизм
- Наследование

Абстракция позволяет выделять общие черты (атрибуты и методы) некоторого множества объектов.

Например, при работе с множеством объектов автомобиля можно выделить атрибуты: модель автомобиля, год выпуска, серийный номер. На основе данных об автомобиле можно разработать метод для оценки его стоимости.

Инкапсуляция позволяет в рамках одной конструкции языка создать описание данных и методов обработки этих данных (обычно такой конструкцией является класс). Еще одно предназначение инкапсуляции - ограничение прямого доступа к данным. Доступ к атрибутам должен быть реализован с применением методов, что позволяет обеспечить защиту данных и надежность работы программы.

Полиморфизм дает возможность реализовать разное поведение программы в зависимости от различных входных данных. Например, если на вход программы передаются целые числа, выполняется один алгоритм их обработки, если вещественные числа - второй, комплексные числа - третий, строки - четвертый. При этом имя вызываемого метода одно и то же.

Наследование предназначено для создания иерархий классов. Например, при проектировании системы классов легковой автомобиль, грузовой автомобиль, спортивный автомобиль, можно выделить общие атрибуты для всех классов и поместить их в родительский класс автомобиль, а остальные классы сделать его наследниками. При этом все атрибуты и методы родительского класса автоматически становятся доступными для классов-наследников.

Классы

Класс - это тип данных, определяемый пользователем.

Класс описывает структуру объектов (экземпляров класса).

Класс объединяет данные и функции, которые управляют этими данными.

Данные называются *атрибутами* класса, а функции - *методами*.

Атрибуты и методы называются членами класса.

```
1  class Name([Superclass]):
2      """ Class description """
3
4      var1 = value1
5      var2 = value2
6      ...
7
8      def method1(self):
9          code
10         return value1
11
12     def method2(self):
13         code
14         return value2
15
16     def method3(self):
17         code
18         return value3
19     ...
```

Листинг 103: Общая форма объявления класса

Пример.

Опишем класс точек на плоскости.

```
1  class Point:
2      x = 0
3      y = 0
```

Листинг 104: Пример класса

Создание экземпляра класса.

```

1
2  # Объявление класса
3  class Point:
4      x = 0
5      y = 0
6
7
8  point1 = Point()
9  print(point1.x, point1.y)

```

Листинг 105: *Создание экземпляра класса*

```

1      0 0

```

Листинг 106

Для изменения координат точки (перемещения точки на плоскости) создадим функцию `move(a,b)`. Если функция размещена внутри класса, то ее принято называть методом.

```

1
2  # Объявление класса
3  class Point:
4      x = 0
5      y = 0
6
7      # Создание метода
8      def move(self, a, b):
9          self.x = self.x + a
10         self.y = self.y + b
11
12 point1 = Point()
13 print(point1.x, point1.y)
14 point1.move(5, 5)
15 print(point1.x, point1.y)

```

Листинг 107: *Создание метода*

```

1      0 0
2      5 5

```

Листинг 108

Пример. Класс, описывающий множество окружностей.

```

1
2  # Объявление класса
3  class Circle:
4      x = 0
5      y = 0
6      r = 0
7
8  circle1 = Circle()
9  print(circle1.x, circle1.y, circle1.r)

```

Листинг 109: Создание экземпляра класса

```

1      0 0 0

```

Листинг 110

Пример. Добавим метод для вычисления длины окружности ($C = 2\pi r$).

```

1  import math
2
3  # Объявление класса
4  class Circle:
5      x = 0
6      y = 0
7      r = 0
8
9      def circumference(self):
10         return 2 * math.pi * self.r
11
12
13  circle1 = Circle()
14  circle1.r = 5
15  print(circle1.x, circle1.y, circle1.r)
16  print(circle1.circumference())

```

Листинг 111: Создание экземпляра класса

```

1      0 0 5
2      31.41592653589793

```

Листинг 112

Наследование.

С помощью наследования можно строить иерархии классов.

Наследование позволяет избежать дублирования кода и улучшить структуру программы.

Например, создадим класс `Point`, описывающий множество точек на плоскости, и класс `Circle`, который наследуется от класса `Point` (координаты центра окружности) и позволяет указать радиус.

```
1  import math
2
3
4  class Point:
5      x = 0
6      y = 0
7
8      def move(self, a, b):
9          self.x = self.x + a
10         self.y = self.y + b
11
12
```

Листинг 113: *Наследование*

```
1  # Класс Circle наследуется от класса Point
2  class Circle(Point):
3
4      r = 0
5
6      def circumference(self):
7          return 2 * math.pi * self.r
8
9
10 circle1 = Circle()
11 circle1.r = 5
12 circle1.move(3, 7)
13 print(circle1.x, circle1.y, circle1.r)
14 print(circle1.circumference())
15
16 3 7 5
17 31.41592653589793
```

Листинг 114: *Наследование*

Класс `Point` называется родительским классом, а класс `Circle` — классом-потомком. Как видно из примера, класс `Circle` наследует все поля и методы

родительского класса.

Конструкторы

Конструктор - это метод, который вызывается автоматически при создании объекта. Имя конструктора совпадает с именем класса.

Например, при описании класса точек на плоскости `Point` требуется, чтобы при создании объекта этого класса значения координат точки были равны (0;0).

```
1  class Point:
2      x = 0
3      y = 0
4
5      def __init__(self):
6          self.x = 5
7          self.y = 5
8
9
10 point1 = Point()
11 print(point1.x, point1.y)
```

Листинг 115: *Конструктор*

Результат: 5 5

Деструкторы

Деструктор - это метод, который вызывается автоматически в момент уничтожения объекта.

Для создания деструктора используется метод `__del__(self)`.

Деструкторы выполняются в порядке, обратном порядку создания объектов.

```
1
2  __del__(self):
3      code
4      ...
5
```

Листинг 116: *Деструктор*

Пример:

```
1
2 class Point:
3     x = 0
4     y = 0
5
6     def __init__(self):
7         self.x = 5
8         self.y = 5
9
10    def __del__(self):
11        print("Object deleted")
12
13
14 point1 = Point()
15 print(point1.x, point1.y)
16
```

Листинг 117: Деструктор

Результат:

0 0

Object deleted

Переопределение методов

Переопределение методов позволяет реализовать различное поведение программы в зависимости от ситуации (полиморфизм).

Например, создадим класс Point, описывающий множество точек на плоскости, и классы Circle и Square, которые наследуются от класса Point.

```
1 import math
2
3
4 class Point:
5     x = 0
6     y = 0
7
8     def move(self, a, b):
9         self.x = self.x + a
10        self.y = self.y + b
11
```

Листинг 118: Наследование

```

1  import math
2
3
4  class Point:
5      x = 0
6      y = 0
7
8      def move(self, a, b):
9          self.x = self.x + a
10         self.y = self.y + b
11
12
13     # Класс Circle наследуется от класса Point
14     # Окружность определяется координатами центра и радиусом
15
16     class Circle(Point):
17         r = 0
18
19         def circumference(self):
20             return 2 * math.pi * self.r
21
22         def area(self):
23             return math.pi * self.r**2
24
25

```

Листинг 119: *Наследование*

```

1
2     # Класс Square наследуется от класса Point
3     # Квадрат определяется координатами центра и длиной стороны
4
5     class Square(Point):
6         a = 0
7
8         def perimetr(self):
9             return 4 * self.a
10
11         def area(self):
12             return self.a**2
13

```

Листинг 120: *Наследование*

```

1
2 circle1 = Circle()
3 circle1.r = 5
4 circle1.move(3, 7)
5 print(circle1.x, circle1.y, circle1.r)
6 print(circle1.area())
7
8 square1 = Square()
9 square1.a = 4
10 print(square1.x, square1.y, square1.a)
11 print(square1.area())
12
13
14 3 7 5
15 78.53981633974483
16 0 0 4
17 16

```

Листинг 121: *Наследование*

Как видно из примера, и в классе `Circle`, и в классе `Square` есть метод для вычисления площади фигуры – `area`. При этом работает данный метод по-разному, в зависимости от класса. Если мы создаем объект класса `Circle`, то `area` возвращает значение площади, вычисленное по формуле πr^2 , а если объект класса `Square` – то значение площади, вычисленное по формуле r^2 .

Статические методы

Статические методы могут быть вызваны без создания объекта. Для создания статического метода используется декоратор `@staticmethod`

```

1 class MyMath:
2
3     @staticmethod
4     def summa(x, y):
5         return x + y
6
7
8 print(MyMath.summa(4, 5))

```

Листинг 122: *Наследование*

Результат: 9

1.20. Задания для самостоятельной работы

1. Вычислить значение выражения

$$\frac{23.5 + \frac{75}{2 + \frac{1}{5}}}{2 - \frac{4}{7}}.$$

2. Вычислить значение функции $f(x) = 3 \cdot \frac{\sin(x)+1}{5}$ при $x = \pi$ и $x = \frac{\pi}{2}$.

3. Построить таблицу значений функции $f(x) = 2 \cdot \cos(x)$ на отрезке $[0, \pi]$ с шагом $h = 0.01$.

4. Объявить функцию

$$f(x) = \begin{cases} x, & x < -2, \\ 10, & -2 \leq x \leq 3, \\ x^2, & x > 3. \end{cases}$$

Вычислить значение этой функции при $x = -5, x = 0, x = 5$.

5. Объявить функцию $sum(n)$ для вычисления суммы последовательности целых чисел от 1 до n с помощью цикла *for*.

6. Объявить функцию $prod(n)$ для вычисления произведения последовательности целых чисел от 1 до n с помощью цикла *while*.

7. Объявить функцию $\sin(x, \epsilon)$ для вычисления значения выражения $\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots + \frac{(-1)^n x^{2n+1}}{(2n+1)!} + \dots$ с заданной точностью ϵ . Вычислить с точностью 10^{-4} выражения $\sin(0), \sin(\frac{\pi}{6}), \sin(\frac{\pi}{2})$.

8. Объявить функцию $scalarprod(x, y)$ для вычисления скалярного произведения векторов x и y . Найти скалярное произведение векторов $a = (1, 5, 8)$ и $b = (4, 5, 1)$.

9. Объявить функцию $change(A, i, j)$, которая в матрице A меняет местами строки i и j .

10. Объявить функцию $fib(n)$ для нахождения значения n -го числа Фибоначчи.

11. Объявить функцию, позволяющую численно проверить первый замечательный предел $\lim_{x \rightarrow 0} \frac{\sin x}{x} = 1$. Проверку можно реализовать, задавая x значения $1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \dots$ до тех пор, пока левая часть не будет отличаться от правой менее чем на заданную погрешность ϵ .

12. Объявить функцию, которая определяет, является ли заданная целочисленная матрица латинским квадратом. Латинским квадратом порядка n называется квадратная таблица размером $n \times n$, каждая строка и каждый

столбец которой содержат все числа от 1 до n .

13. Объявить функцию, которая определяет, является ли заданная целочисленная матрица магическим квадратом. Магическим квадратом порядка n называется квадратная таблица $n \times n$, состоящая из чисел $1, 2, \dots, n^2$ так, что суммы по каждому столбцу, каждой строке и каждой из двух диагоналей равны между собой.

14. Объявить функцию для построения магического квадрата заданного размера.

15. Объявить функцию, находящую слова-палиндромы в списке, состоящем из русских слов. Палиндромами называются слова, одинаково читающиеся как слева направо, так и справа налево.

16. Объявить функцию, находящую анаграммы в списке, состоящем из русских слов. Анаграммами называются пары слов, при прочтении каждого из которых в обратном порядке образуется другое слово пары.

17. Найти сумму, разность, произведение двух матриц

$$A = \begin{pmatrix} 1 & -2 & 5 \\ -1 & 10 & 7 \\ 3 & 4 & -5 \end{pmatrix}, \quad B = \begin{pmatrix} -6 & 3 & -2 \\ 1 & 5 & 4 \\ 2 & -3 & 8 \end{pmatrix}.$$

18. Построить график функции $f(x) = \sin(2 \cdot x)$ на отрезке $[-2\pi, 2\pi]$ с шагом $h = 0.01$.

19. Построить на плоскости кривую, заданную параметрически

$$x(t) = \frac{3}{4} \cdot R \cdot \cos\left(\frac{t}{4}\right) + \frac{1}{4} \cdot R \cdot \cos\left(\frac{3 \cdot t}{4}\right),$$

$$y(t) = \frac{3}{4} \cdot R \cdot \sin\left(\frac{t}{4}\right) - \frac{1}{4} \cdot R \cdot \sin\left(\frac{3 \cdot t}{4}\right),$$

где $R = 4$, а t изменяется на отрезке $[0, 8\pi]$ с шагом $h = 0.01$.

20. Построить на плоскости кривую в полярной системе координат $\rho(\varphi) = 10 \cdot \sin(3 \cdot \varphi)$, где φ изменяется на отрезке $[-\pi, \pi]$ с шагом $h = 0.01$.

21. Построить график функции $f(x, y) = x^2 + y^2$ в области $D = [-1, 1] \times [-1, 1]$ с шагом изменения t и s , равным 0.1.

Вопросы к защите лабораторной работы

1. Базовые типы данных в Python.
2. Переменные. Операторы выражений.
3. Последовательности. Присваивание последовательностей.
4. Словари. Файлы.
5. Условная инструкция. Циклы.
6. Функции.
6. Модули и пакеты.
7. Документация
8. Пакет NumPy.
9. Пакет matplotlib.

2. Теория погрешностей и машинная арифметика

В компьютерах чаще всего используется способ представления чисел в форме с плавающей запятой. Представление чисел в этой форме имеет вид $a = M \cdot 10^p$.

Число M называется мантиссой, p – показателем степени. Для избежания неоднозначности представления чисел обычно требуют, чтобы первая цифра мантиссы не равнялась нулю, т.е. используют нормализованные числа.

Множество чисел с плавающей запятой конечно, т.е. справедливо неравенство

$$0 < X_0 \leq |x| < X_\infty,$$

где x — множество представимых на компьютере чисел.

Если результат операции превышает X_∞ , то происходит переполнение, и большинство компьютеров выдает сообщение об ошибке. В этом случае говорят о получении машинной бесконечности. Все числа по модулю, большие X_∞ , рассматриваются как машинная бесконечность.

Если результатом операции является число, меньшее X_0 , то компьютеры заменяют результат нулем. В этом случае говорят о получении машинного нуля. Все числа по модулю, меньшие X_0 , рассматриваются как машинный нуль.

При сложении различных машинных чисел результат может оказаться равен одному из слагаемых. Наименьшее число с плавающей запятой, которое при сложении с числом 1 дает результат, больший чем 1, называется машинный эпсилон.

2.1. Основные понятия теории приближенных вычислений

Пусть X – точное значение некоторой величины, а x – ее приближенное значение. Абсолютной погрешностью x называется число

$$e_x = |X - x|.$$

Верхняя граница Δx абсолютной погрешности удовлетворяет неравенству

$$|X - x| \leq \Delta x.$$

Относительной погрешностью x называется число

$$r_x = \frac{|X - x|}{|x|}.$$

Верхняя граница относительной погрешности x определяется равенством

$$\delta_x = \frac{\Delta x}{|x|}.$$

Цифра числа называется верной (в широком смысле), если абсолютная погрешность числа не превосходит единицы разряда, в котором стоит эта цифра.

Цифра числа называется верной в строгом смысле, если абсолютная погрешность этого числа не превосходит половины единицы разряда, в котором стоит эта цифра.

Значащими цифрами числа, записанного в виде десятичной дроби, называют все его верные цифры, начиная с первой слева, отличной от нуля. Правильной записью приближенного числа называется запись этого числа только верными знаками.

Для исключения неверных цифр используют округление. При этом возникает погрешность округления. Обычно используют округление методом отбрасывания и симметричного округления.

При использовании округления методом отбрасывания используют понятие цифры, верной в широком смысле, при использовании округления методом симметричного округления — понятие цифры, верной в строгом смысле.

Пусть x — приближенное значение числа X , x_1 — округленное значение x . Тогда

$$\Delta x_1 = |X - x_1| \leq |X - x| + |x - x_1| \leq \Delta x + \Delta_{\text{окр}}.$$

2.2. Определение количества верных цифр по относительной погрешности приближенного числа

Для того чтобы по заданной величине δx найти количество верных значащих цифр в числе x , достаточно подобрать наибольшее натуральное число n так, чтобы выполнялось неравенство $\delta x \leq 10^{-n}$, и уменьшить n на единицу. В некоторых случаях в числе x могут оказаться верными n цифр.

Пример. Пусть $x = 15,327$, $\delta x = 0,0007$. Так как $0,0007 \leq 10^{-3}$, то x имеет не менее двух верных в строгом смысле цифр.

Если первая значащая цифра в значении δx меньше 5, то можно использовать условие

$$\delta x \leq \frac{1}{2} \cdot 10^{-n}.$$

Тогда число x имеет не менее n верных в строгом смысле цифр.

Пример. Пусть $x = 127,453$, $\delta x = 0,004$. Так как $0,004 < 0,005 = \frac{1}{2} \cdot 10^{-2}$, то в x являются верными в строгом смысле не менее двух цифр.

2.3. Формулы для вычисления погрешностей арифметических действий

Пусть $S = X + Y$ — сумма точных чисел, а $s = x + y$ — сумма их приближенных значений. Тогда

$$\Delta s = \Delta x + \Delta y.$$

$$\delta(x + y) = \frac{|x|}{|x + y|} \cdot \delta x + \frac{|y|}{|x + y|} \cdot \delta y,$$

$$\delta(x - y) = \frac{|x|}{|x - y|} \cdot \delta x + \frac{|y|}{|x - y|} \cdot \delta y.$$

Пусть x и y — приближенные значения некоторых чисел ($x, y > 0$). Тогда

$$\Delta(xy) = x \cdot \Delta y + y \cdot \Delta x,$$

$$\Delta\left(\frac{x}{y}\right) = \frac{x \cdot \Delta y + y \cdot \Delta x}{y^2},$$

$$\delta(x \cdot y) = \delta\left(\frac{x}{y}\right) = \delta x + \delta y.$$

2.4. Вычисление погрешностей значений элементарных функций

Пусть функция $f(x)$ дифференцируема в некоторой окрестности x . Тогда

$$\Delta f = |f'(x)| \cdot \Delta x.$$

Пример. Определить количество верных в строгом смысле цифр в значении функции $f(x) = \sin^2 x$ при $x = 2,321$. В числе x все цифры верны в строгом смысле.

Решение.

$$f'(x) = \sin 2x, \quad \Delta x = 0,0005.$$

$$\Delta f(x) = |\sin(2 \cdot 2, 321)| \cdot 0,0005 \approx 0.0004987618591 < 0.0005.$$

Таким образом, в значении $\sin^2(2, 321) \approx 0.535165434904$ верны в строгом смысле 4 цифры.

2.5. Задания для самостоятельной работы

1. Найти значения машинного нуля, машинной бесконечности, машинного эпсилон.
2. У приближенных чисел a и b все цифры верны в строгом смысле. Указать границы абсолютных и относительных погрешностей этих чисел.
3. По заданным значениям приближенных чисел и их относительных погрешностей установить количество цифр, верных в строгом смысле.
4. Произвести указанные действия и определить абсолютные и относительные погрешности результатов (исходные числа заданы верными в строгом смысле цифрами).
5. Исходные числовые значения аргумента заданы цифрами, верными в строгом смысле. Определить количество верных в строгом смысле цифр в следующих значениях элементарных функций.

Варианты
Задание №2

1. $a = 25,4532, b = 1,02$.
2. $a = 23,23, b = 0,0005$.
3. $a = 0,00451, b = 2,324$.
4. $a = 1,2013, b = 0,35$.
5. $a = 2,056, b = 1,004$.
6. $a = 41,56, b = 2,034$.
7. $a = 2,54, b = 0,345$.
8. $a = 3,45, b = 1,23$.
9. $a = 6,3456, b = 32,65$.
10. $a = 1,35, b = 2,34$.
11. $a = 4,5678, b = 3,2$.
12. $a = 3,047, b = 0,403$.
13. $a = 1,04, b = 3,05$.
14. $a = 7,65, b = 1,034$.
15. $a = 3,04, b = 0,32$.

Задание №3

1. $a = 12,2503, \delta x = 0,0006$.
2. $a = 2,234, \delta x = 0,0003$.
3. $a = 4,0451, \delta x = 0,0001$.
4. $a = 1,201, \delta x = 0,003$.
5. $a = 2,0396, \delta x = 0,001$.
6. $a = 1,542, \delta x = 0,00007$.
7. $a = 4,22, \delta x = 0,001$.
8. $a = 5,435, \delta x = 0,0002$.
9. $a = 7,323, \delta x = 0,006$.
10. $a = 2,234, \delta x = 0,05$.
11. $a = 2,0643, \delta x = 0,00004$.
12. $a = 1,037, \delta x = 0,0003$.
13. $a = 5,025, \delta x = 0,006$.
14. $a = 7,6503, \delta x = 0,0002$.
15. $a = 3,0452, \delta x = 0,00008$.

Задание №4

1. а) $2, 23 - 9, 17$, б) $3, 32 + 8, 3$, в) $334, 5 \cdot 123, 5$, г) $\frac{27,3}{0,34}$.
2. а) $2, 323 - 0, 132$, б) $1, 345 + 1, 233$, в) $331, 5 \cdot 111, 5$, г) $\frac{22,1}{0,345}$.
3. а) $1, 233 - 7, 2$, б) $3, 3 + 3, 1$, в) $3, 34 \cdot 12, 14$, г) $\frac{2,32}{0,67}$.
4. а) $4, 332 - 5, 172$, б) $3, 32 + 8, 3$, в) $334, 5 \cdot 123, 5$, г) $\frac{27,3}{0,34}$.
5. а) $1, 213 - 1, 123$, б) $1, 234 + 2, 4$, в) $33, 51 \cdot 1, 121$, г) $\frac{12,13}{0,34}$.
6. а) $1, 234 - 1, 37$, б) $6, 32 + 7, 12$, в) $23, 12 \cdot 11, 35$, г) $\frac{26,12}{1,234}$.
7. а) $4, 231 - 1, 132$, б) $1, 22 + 4, 7$, в) $34, 5 \cdot 124, 5$, г) $\frac{23,31}{0,34}$.
8. а) $1, 23 - 12, 13$, б) $4, 56 + 7, 16$, в) $12, 6 \cdot 12, 35$, г) $\frac{134,31}{0,1234}$.
9. а) $1, 32 - 7, 1$, б) $12, 3 + 3, 7$, в) $8, 1 \cdot 1, 25$, г) $\frac{7,32}{0,328}$.
10. а) $2, 123 - 3, 324$, б) $3, 22 + 1, 31$, в) $324, 5 \cdot 121, 3$, г) $\frac{12,32}{1,15}$.
11. а) $1, 23 - 6, 11$, б) $3, 343 + 9, 12$, в) $4, 5 \cdot 1, 22$, г) $\frac{7,233}{1,354}$.
12. а) $2, 123 - 1, 156$, б) $1, 78 + 1, 293$, в) $123, 56 \cdot 1, 67$, г) $\frac{1,23}{6,212}$.
13. а) $45, 23 - 12, 17$, б) $0, 326 + 2, 12$, в) $34, 51 \cdot 133, 54$, г) $\frac{21,22}{0,334}$.
14. а) $123, 223 - 4, 16$, б) $1, 322 + 5, 123$, в) $2, 5 \cdot 123, 4$, г) $\frac{24,6}{0,36}$.
15. а) $1, 234 - 65, 1$, б) $1, 22 + 1, 35$, в) $34, 54 \cdot 11, 8$, г) $\frac{7,13}{1,87}$.

Задание №5

1. $f(x) = \sin x$, $x = 0, 356$.
2. $f(x) = \cos x$, $x = 0, 452$.
3. $f(x) = \arcsin x$, $x = 0, 573$.
4. $f(x) = \sqrt{x}$, $x = 0, 453$.
5. $f(x) = \frac{1}{x}$, $x = 0, 342$.
6. $f(x) = \operatorname{tg} x$, $x = 0, 234$.
7. $f(x) = \ln x$, $x = 0, 325$.
8. $f(x) = \lg x$, $x = 0, 456$.
9. $f(x) = e^x$, $x = 0, 34$.
10. $f(x) = \operatorname{arctg} x$, $x = 0, 23$.
11. $f(x) = \sin^2 x$, $x = 0, 237$.
12. $f(x) = \operatorname{ctg} x$, $x = 0, 345$.
13. $f(x) = \operatorname{arcctg} x$, $x = 0, 343$.
14. $f(x) = \operatorname{tg}^2 x$, $x = 0, 435$.
15. $f(x) = \arcsin(x)$, $x = 0, 34$.

Вопросы к защите лабораторной работы

1. Представление чисел в ЭВМ. Машинный нуль, машинная бесконечность, машинный эпсилон.
2. Источники и классификация погрешностей.
3. Приближенные числа. Абсолютная и относительная погрешности. Верные и значащие цифры. Способы округления.
4. Погрешности арифметических операций над приближенными числами.
5. Погрешность вычисления функций одной переменной.
6. Числа a, b, c заданы приближенно:
 $a = 254.671, \Delta a = 3.12,$
 $b = 0.9375, \delta b = 0.03,$
 $c = -15.146, \Delta c = 0.07.$
Записать эти числа со всеми верными знаками.
7. Числа a, b заданы приближенно: $a = 10.17, b = 1.057, \Delta a = \Delta b = 0.011.$
Оценить погрешности:
а) суммы и разности a и b ; б) произведения и частного a и b .
Записать ответ с учетом верных цифр.
8. Указать правила оценки абсолютных и относительных погрешностей функций:
а) $f(x) = 2x^{3a}$; б) $f(x) = 5a^{2x}, a > 0$; в) $f(x) = e^{73x}.$

3. Решение нелинейных уравнений

Решение нелинейного уравнения обычно состоит из двух этапов: локализации (отделения) корней, т.е. нахождения достаточно малых отрезков, на которых находится только по одному корню, и уточнения корней, т.е. нахождения на каждом из отрезков значения корня с заданной точностью.

Нелинейные уравнения делятся на алгебраические (т.е. представимые в виде полинома n -й степени) и трансцендентные.

3.1. Локализация корней

Будем считать, что функция $F(x)$ непрерывна. Во многих случаях локализацию корней уравнения $F(x) = 0$ удобно проводить с помощью графика.

В некоторых случаях для локализации корней удобнее использовать следующий алгоритм: Отрезок $[a, b]$, на котором находятся все искомые корни уравнения $F(x) = 0$, разбивается на равные части с некоторым достаточно малым шагом h . Для каждого из полученных отрезков проверяется, меняет ли функция $F(x)$ знак. Если меняет, то на этом отрезке находится по крайней мере один корень уравнения $F(x) = 0$. При использовании данного алгоритма следует учитывать, что выполнение данного условия не позволяет утверждать, что на отрезке находится только один корень и что на отрезке нет корней, если условие не выполняется. Чтобы гарантировать, что на отрезке находится только один корень, достаточно, чтобы функция $F(x)$ на этом отрезке была непрерывна, строго монотонна и меняла знак.

3.2. Решение нелинейных уравнений с помощью библиотечной функции fsolve

Формат функции fsolve (сокращенный):

`fsolve(F, x0)`,

где x_0 — начальное значение, F — имя функции ($F(x) = 0$).

В качестве x_0 можно использовать список.

Пример. Найти корни уравнения $x^2 - 25 = 0$.

```

1  from scipy.optimize import fsolve
2
3  def f(x):
4      return pow(x, 2) - 25
5
6
7  roots = fsolve(f, [-10, 10])
8  print(roots)

```

Листинг 123: *Решение нелинейного уравнения с помощью библиотечной функции fsolve*

Результат:

[-5. 5.]

3.3. Метод деления отрезка пополам

Рассмотрим уравнение

$$F(x) = 0.$$

Будем считать, что на отрезке $[a, b]$ существует единственный корень этого уравнения и функция $F(x)$ непрерывна. Уточнение корня методом деления отрезка пополам заключается в том, что отрезок $[a, b]$ делится точкой $c = \frac{a+b}{2}$ на две равные части. Если $F(c) = 0$, то c — корень данного уравнения. Если $F(c) \neq 0$, то выбирается та часть, на которой выполняется одно из неравенств $F(a) \cdot F(c) < 0, F(b) \cdot F(c) < 0$. После этого для выбранного отрезка процедура деления повторяется. Прекратив процесс деления на некотором n -м шаге, получим отрезок $[a_n, b_n]$. Следовательно, в качестве корня можно принять значение

$$x = \frac{a_n + b_n}{2}.$$

При этом погрешность

$$\Delta x \leq \frac{b_n - a_n}{2}.$$

Следовательно, если требуется найти корень с точностью ϵ , то нужно продолжать деление до тех пор, пока $b_n - a_n > 2\epsilon$.

3.4. Теорема Банаха

Множество X называется *метрическим пространством*, если каждой паре его элементов x, y поставлено в соответствие неотрицательное действительное число $\rho(x, y)$, называемое расстоянием между элементами x и y и

удовлетворяющее следующим условиям (аксиомам метрики):

1. $\rho(x, y) = 0 \iff x = y$;
2. $\rho(x, y) = \rho(y, x)$;
3. $\rho(x, y) \leq \rho(x, z) + \rho(z, y)$.

Элементы метрического пространства называются точками.

Последовательность $\{x_n\}$ элементов метрического пространства X называется *фундаментальной*, если для любого $\varepsilon > 0$ найдется номер N такой, что $\rho(x_n, x_m) < \varepsilon$ при $n, m > N$.

Точка $x_0 \in X$ называется *пределом последовательности* $\{x_n\}$, если $\rho(x_n, x_0) \rightarrow 0$ при $n \rightarrow \infty$. Последовательность, имеющая предел, называется *сходящейся*.

Запись $\lim_{n \rightarrow \infty} x_n = x_0$ означает, что x_0 является пределом последовательности (x_n) , или, что последовательность (x_n) сходится к x_0 .

Если в метрическом пространстве X каждая фундаментальная последовательность сходится, то *пространство X называется полным*.

Множество E называется линейным (векторным) пространством над полем R , если для любых $x, y \in E$ определена сумма $x + y \in E$ и для любых $x \in E, \alpha \in R$ определено произведение $\alpha x \in E$, причем выполнены следующие условия: $\forall x, y, z \in E, \forall \alpha, \beta \in R$

- 1) $x + y = y + x$; 2) $x + (y + z) = (x + y) + z$;
- 3) $\alpha(x + y) = \alpha x + \alpha y$; 4) $(\alpha + \beta)x = \alpha x + \beta x$;
- 5) $\alpha(\beta x) = (\alpha\beta)x$; 6) $1 \cdot x = x$;

7) существует элемент $\theta \in E$ такой, что $x + \theta = x$. θ называется нулем векторного пространства.

Линейное пространство E называется нормированным пространством, если каждому элементу $x \in E$ поставлено в соответствие действительное число $\|x\|$, называемое нормой этого элемента и удовлетворяющее условиям: $\forall x, y \in E, \forall \alpha \in R$:

- 1) $\|x\| \geq 0$;
- 2) $\|x\| = 0 \iff x = \theta$;
- 3) $\|\alpha x\| = |\alpha| \|x\|$;
- 4) $\|x + y\| \leq \|x\| + \|y\|$.

Нормированное пространство является метрическим пространством с расстоянием $\rho(x, y) = \|x - y\|$.

Линейное нормированное пространство, полное в метрике, порожденной нормой, называется *банаховым пространством*.

Примером банахового пространства является пространство $C(\Omega)$ непрерывных на отрезке или на прямоугольнике Ω функций $x(\omega)$ с нормой $\|x\| = \max_{\omega \in \Omega} |x(\omega)|$. Используемые в дальнейшем пространства также банаховы.

Отображение A , переводящее элементы метрического пространства X в элементы этого же пространства, называется непрерывным в точке x_0 , если для любой последовательности точек $x_n \in X$ ($n = 1, 2, \dots$), сходящейся к точке x_0 , последовательность (Ax_n) сходится к Ax_0 , т.е.

$$\lim_{n \rightarrow \infty} Ax_n = Ax_0.$$

Пусть X — метрическое пространство, x и y — элементы этого пространства и A — отображение, переводящее элементы пространства X снова в элементы этого пространства, т.е. $A : X \rightarrow X$.

Точка $x \in X$ называется *неподвижной точкой отображения A* , заданного в метрическом пространстве, если выполняется условие $Ax = x$.

Отображение A называется *сжимающим*, если существует такое число $0 \leq \alpha < 1$, что для любых элементов x, y выполняется условие

$$\rho(Ax, Ay) \leq \alpha \rho(x, y).$$

Теорема Банаха. *Сжимающее отображение полного метрического пространства в себя имеет единственную неподвижную точку, т.е. уравнение $x = Ax$ имеет единственное решение и оно может быть получено методом итераций: $x_{n+1} = Ax_n$ ($n = 0, 1, 2, \dots$) при любом начальном приближении x_0 .*

Доказательство. Пусть x_0 — произвольная точка полного метрического пространства. Определим последовательность

$$x_n = Ax_{n-1} \quad (n = 1, 2, \dots).$$

Покажем, что она фундаментальная. Пусть для определенности $m \geq n$. Тогда

$$\begin{aligned} \rho(x_n, x_m) &= \rho(A^n x_0, A^m x_0) \leq \alpha^n \rho(x_0, x_{m-n}) \leq \\ &\leq \alpha^n (\rho(x_0, x_1) + \rho(x_1, x_2) + \dots + \rho(x_{m-n-1}, x_{m-n})) \leq \\ &\leq \alpha^n \rho(x_0, x_1) (1 + \alpha + \dots + \alpha^{m-n-1}) = \alpha^n \rho(x_0, x_1) \frac{1 - \alpha^{m-n}}{1 - \alpha} \leq \end{aligned}$$

$$\leq \alpha^n \rho(x_0, x_1) \frac{1}{1 - \alpha}.$$

Так как $\lim_{n \rightarrow \infty} \alpha^n \rho(x_0, x_1) \frac{1}{1 - \alpha} = 0$, то (x_n) — фундаментальная последовательность и, следовательно, в силу полноты пространства имеет предел. Обозначим его через x . Проверим, что x — неподвижная точка. Так как сжимающее отображение непрерывно, то

$$Ax = A \lim_{n \rightarrow \infty} x_n = \lim_{n \rightarrow \infty} Ax_n = \lim_{n \rightarrow \infty} x_{n+1} = x.$$

Докажем, что неподвижная точка является единственной. Пусть

$$Ax = x, Ay = y.$$

Так как A — сжимающее отображение, то

$$0 \leq \rho(x, y) = \rho(Ax, Ay) \leq \alpha \rho(x, y).$$

Так как $0 \leq \alpha < 1$, то отсюда следует неравенство

$$0 \leq (1 - \alpha) \rho(x, y) \leq 0.$$

Тогда $\rho(x, y) = 0$, что означает $x = y$. □

Теорема Банаха не только доказывает существование и единственность решения уравнения $x = Ax$, но и указывает способ приближенного нахождения этого решения.

Действительно, пусть x_* — точное решение уравнения. Переходя в неравенстве

$$\rho(x_n, x_m) \leq \frac{\alpha^n}{1 - \alpha} \rho(x_0, x_1)$$

к пределу при $m \rightarrow \infty$, получим

$$\rho(x_n, x_*) \leq \frac{\alpha^n}{1 - \alpha} \rho(x_0, x_1). \quad (1)$$

Из последнего неравенства видно, что если x_n считать приближенным решением уравнения $x = Ax$, то получаемая погрешность не превосходит числа

$$\frac{\alpha^n}{1 - \alpha} \rho(x_0, x_1).$$

Поэтому решение x_* может быть приближено с любой наперед заданной точностью элементом x_n , который может быть найден из рекуррентных соотношений

$$x_1 = f(x_0), x_2 = f(x_1), \dots, x_n = f(x_{n-1}), \dots \quad (2)$$

при произвольно выбранном x_0 . Из (1) видно, что требуемая точность дости-

гаются тем быстрее, чем ближе x_0 к x_* .

Отметим, что элементы последовательности (2) называют последовательными приближениями к элементу x_* , а сам метод построения их — методом последовательных приближений.

3.5. Метод простых итераций

Рассмотрим уравнение

$$F(x) = 0. \quad (2)$$

Преобразуем это уравнение к равносильному

$$x = \varphi(x). \quad (3)$$

Будем считать, что на отрезке $[a, b]$ существует единственный корень уравнения (2) и функция φ отображает отрезок $[a, b]$ в себя.

Выберем начальное приближение корня $x_0 \in [a, b]$. Для вычисления следующих приближений будем использовать итерационную формулу

$$x_{n+1} = \varphi(x_n), \quad n = 0, 1, 2, \dots \quad (4)$$

В силу теоремы Банаха о сжимающем отображении эта последовательность будет сходиться к корню уравнения (2), если $y = \varphi(x)$ — сжимающее отображение, т.е.

$$\rho(\varphi(x_1), \varphi(x_2)) \leq \alpha \cdot \rho(x_1, x_2), \quad 0 \leq \alpha < 1. \quad (5)$$

Выясним условия, при которых будет выполняться неравенство (5).

Пусть функция $\varphi(x)$ определена и дифференцируема на отрезке $[a, b]$.

В качестве метрики $\rho(x_1, x_2)$ можно выбрать функцию $|x_1 - x_2|$. Тогда неравенство (5) запишется в виде

$$|\varphi(x_1) - \varphi(x_2)| \leq \alpha \cdot |x_1 - x_2|, \quad 0 \leq \alpha < 1. \quad (6)$$

По теореме Лагранжа

$$\varphi(x_1) - \varphi(x_2) = \varphi'(c)(x_1 - x_2), \quad c \in (x_1, x_2). \quad (7)$$

Переходя в последнем равенстве к модулям, получим

$$|\varphi(x_1) - \varphi(x_2)| = |\varphi'(c)(x_1 - x_2)| = |\varphi'(c)| \cdot |x_1 - x_2|. \quad (8)$$

Следовательно, отображение $y = \varphi(x)$ будет сжимающим, если $\alpha = \sup_{[a, b]} |\varphi'(x)| < 1$.

В силу теоремы Банаха погрешность n -го приближения Δx_n допускает

оценку

$$\Delta x_n \leq \alpha^n |x_0 - x_1| \frac{1}{1 - \alpha}.$$

Так как $0 \leq \alpha < 1$, то можно принять в качестве оценки погрешности величину

$$\tilde{\Delta} x_n = \frac{\alpha}{1 - \alpha} |x_0 - x_1|.$$

Приняв $x_0 = x_{k-1}$, получим

$$\tilde{\Delta} x_n = \frac{\alpha}{1 - \alpha} |x_{k-1} - x_k|.$$

Отсюда следует, что для получения результата с точностью ϵ нужно продолжать итерации до тех пор, пока не начнет выполняться неравенство

$$|x_{k-1} - x_k| < \frac{\epsilon(1 - \alpha)}{\alpha}.$$

Пример. Найти корень уравнения $\sin(x + 2) - x = 0$ на отрезке $[0, 1]$ с точностью 0,001.

Решение. Пусть $F(x) = \sin(x + 2) - x$. Преобразуем уравнение $F(x) = 0$ к виду $x = \varphi(x)$ таким образом, чтобы отображение $y = \varphi(x)$ было сжимающим.

Уравнение $F(x) = 0$ преобразуем к виду

$$x = x - m \cdot F(x),$$

где $m = \text{const} \neq 0$. В этом случае

$$\varphi(x) = x - m \cdot F(x).$$

Тогда $\varphi'(x) = 1 - m \cdot F'(x)$. Следовательно, для того чтобы было

$$|\varphi'(x)| = |1 - m \cdot F'(x)| \leq \alpha < 1,$$

достаточно подобрать m так (если это возможно), чтобы для всех x из отрезка $[0, 1]$ выполнялось неравенство

$$0 < p \leq m \cdot F'(x) < 1.$$

В силу непрерывности функции $F'(x)$ это условие будет выполняться, если для всех $x \in [0, 1]$ $m \cdot F'(x) > 0$ и $|m| = \frac{1}{\max |F'(x)|}$.

Найдем $\max |F'(x)|$ на отрезке $[0, 1]$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 x = np.arange(0, 1.01, 0.01)
4 plt.plot(x, np.cos(x + 2) - 1)
5 plt.xlabel(r'$x$')
6 plt.ylabel(r"$dF(x)$")
7 plt.title(r"$dF(x) = \cos(x + 2) - 1$")
8 plt.grid(True);
9 plt.show()

```

Листинг 124

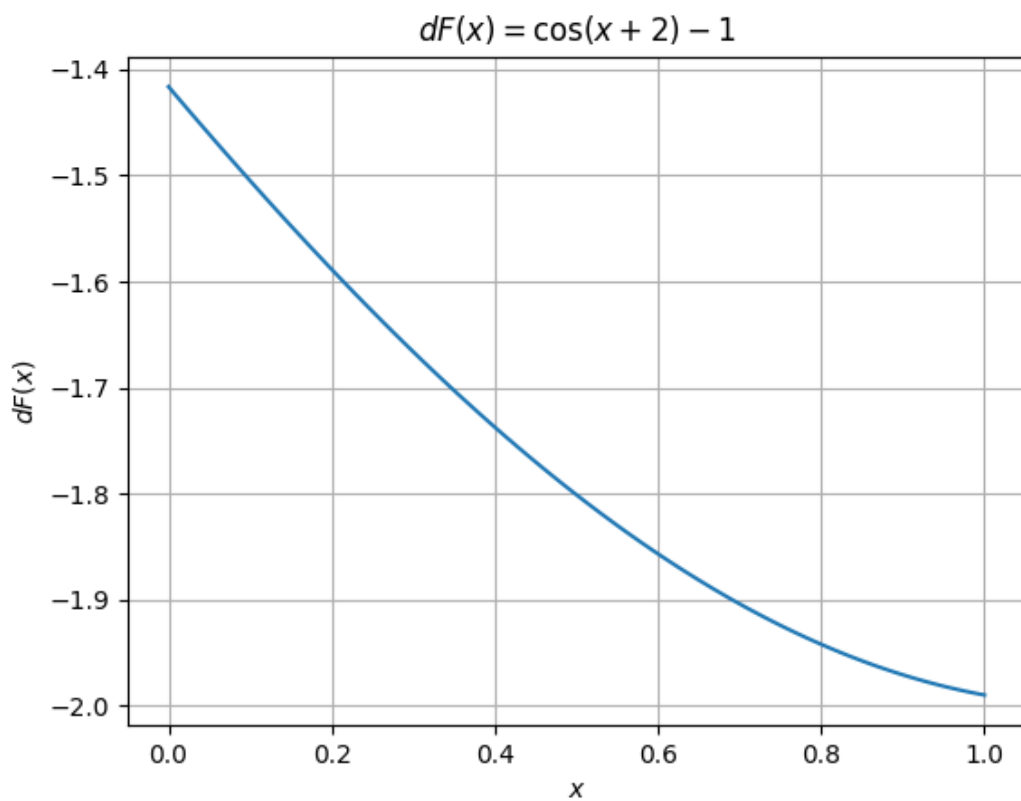


Рисунок 3.1. График производной функции $F(x)$.

Из графика видно, что на отрезке $[0, 1]$

$$\max|F'(x)| = |F'(1)| \text{ и } F'(x) < 0.$$

Таким образом, для выполнения условия

$$0 < p \leq m \cdot F'(x) < 1,$$

следовательно, и условия $|\varphi'(x)| \leq \alpha < 1$ достаточно принять

$$m = \frac{1}{F'(1)} \approx -0.503.$$

Найдем значение α .

$$\varphi'(x) = 1 + 0.503 \cdot (\cos(x + 2) - 1).$$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 x=np.arange(0, 1.01, 0.01)
4 plt.plot(x, 1 + 0.503 * (np.cos(x + 2) - 1))
5 plt.xlabel(r'$x$')
6 plt.ylabel(r"$\varphi'(x)$")
7 plt.title(r"$\varphi'(x)=1+0.503\cdot (\cos(x+2)-1)$")
8 plt.grid(True)
9 plt.show()
```

Листинг 125

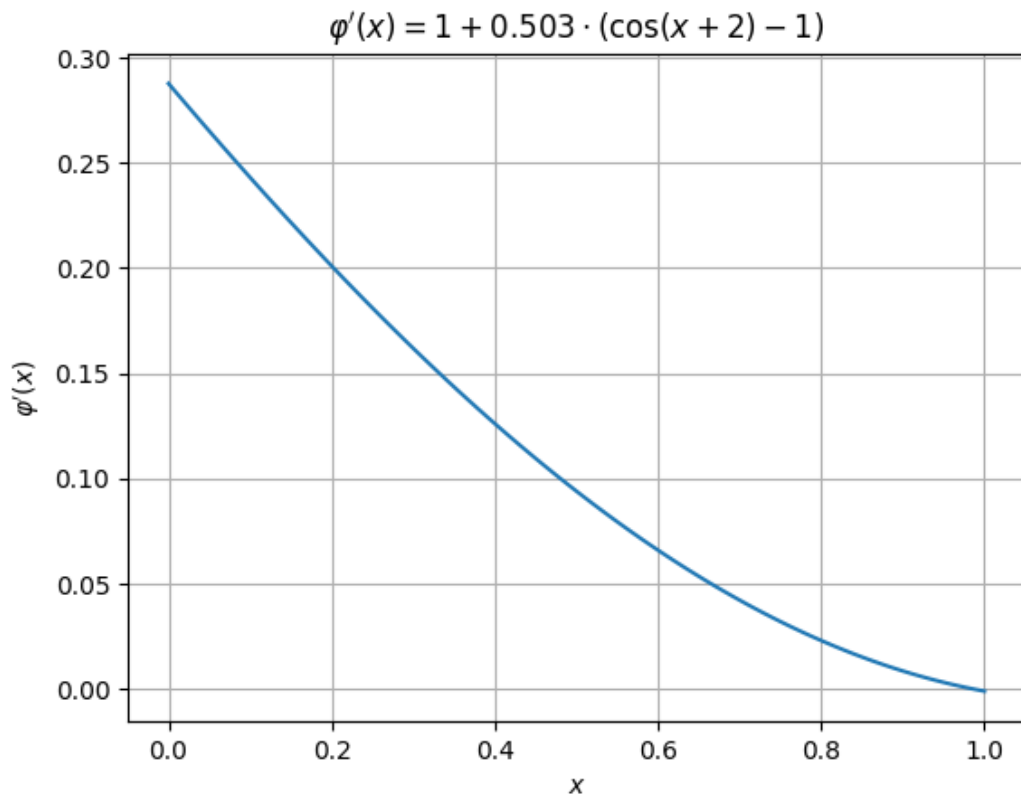


Рисунок 3.2. График производной $\varphi'(x)$.

Из графика видно, что

$$\max_{[0,1]} |\varphi'(x)| = \varphi'(0).$$

Так как $\varphi'(0) \approx 0,288$, то можно принять $\alpha = 0,288$.

Тогда уравнение приобретает вид

$$x = x + 0.503 \cdot (\sin(x + 2) - x) \equiv \varphi(x), \quad \alpha = 0,288,$$

причем $\varphi(x)$ — сжимающее отображение отрезка $[0, 1]$ в себя. По теореме Банаха, уравнение имеет единственное решение на отрезке $[0, 1]$ и его можно найти методом итераций.

Алгоритм можно представить в виде программы на языке python.

```

1  import numpy as np
2  e=0.001; q=0.288;
3  x0=0; x1=x0 + 0.503 * (np.sin(x0 + 2) - x0)
4  while abs(x1 - x0)>e * (1 - ) / q:
5      x0 = x1
6      x1 = x0 + 0.503 * (np.sin(x0 + 2) - x0)
7  print(x1)

```

Листинг 126: Решение нелинейного уравнения методом итераций

Результат:

0.55414

3.6. Метод Ньютона

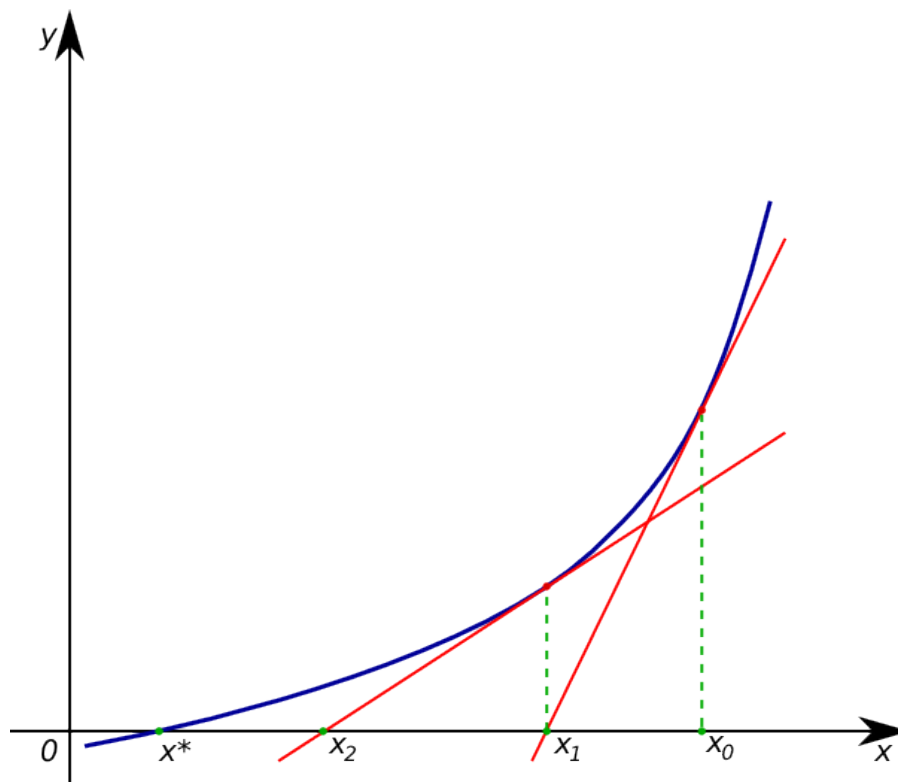


Рисунок 3.3. Метод Ньютона.

Пусть уравнение $F(x) = 0$ имеет единственный корень на отрезке $[a, b]$. Выберем начальное значение $x_0 \in [a, b]$. Элементы итерационной последова-

тельности находятся по формуле

$$x_{i+1} = x_i - \frac{F(x_i)}{F'(x_i)}, \quad (i = 0, 1, 2, \dots).$$

3.7. Модифицированный метод Ньютона

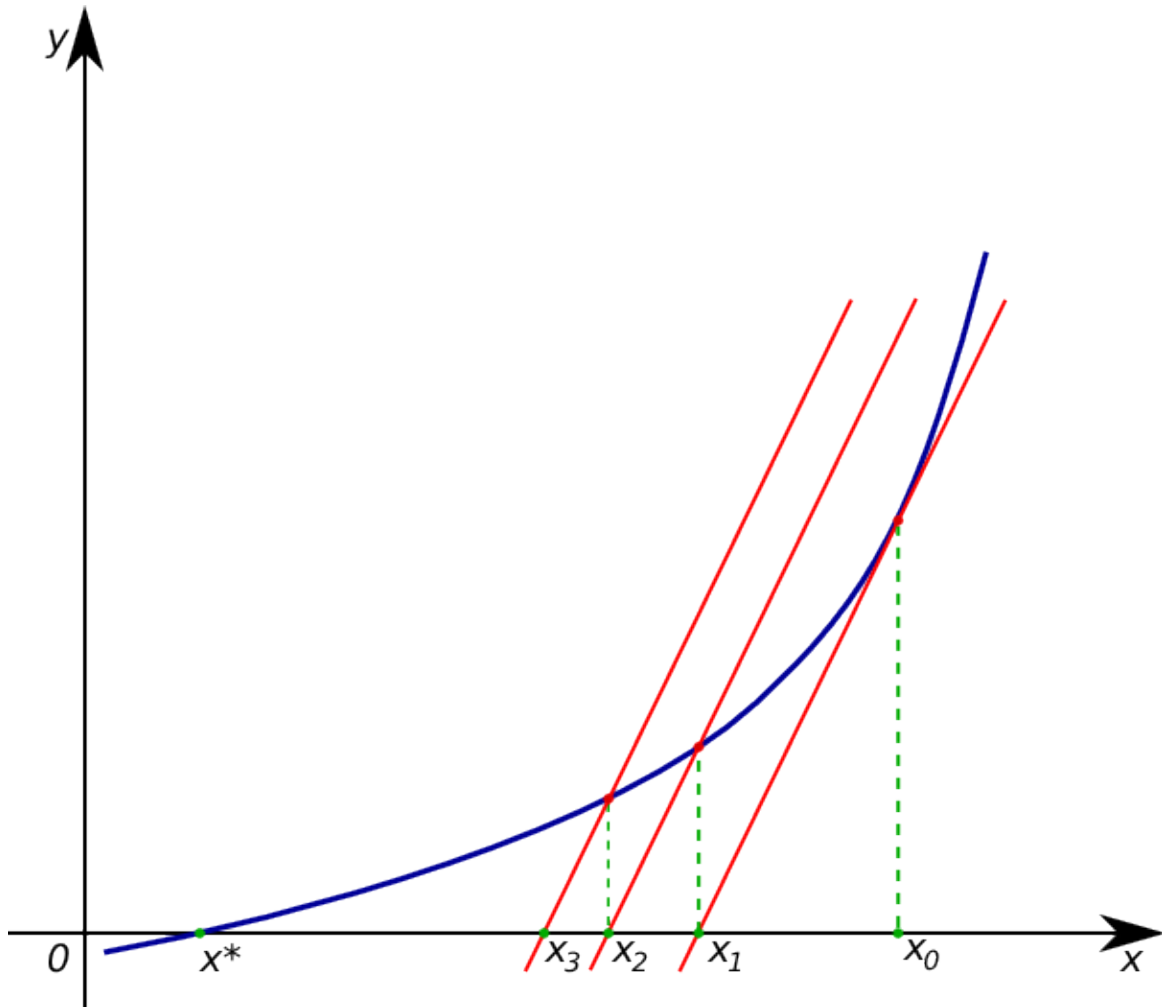


Рисунок 3.4. Модифицированный метод Ньютона.

Пусть уравнение $F(x) = 0$ имеет единственный корень на отрезке $[a, b]$. Выберем начальное значение $x_0 \in [a, b]$. Элементы итерационной последовательности находятся по формуле

$$x_{i+1} = x_i - \frac{F(x_i)}{F'(x_0)}, \quad (i = 0, 1, 2, \dots).$$

3.8. Метод хорд

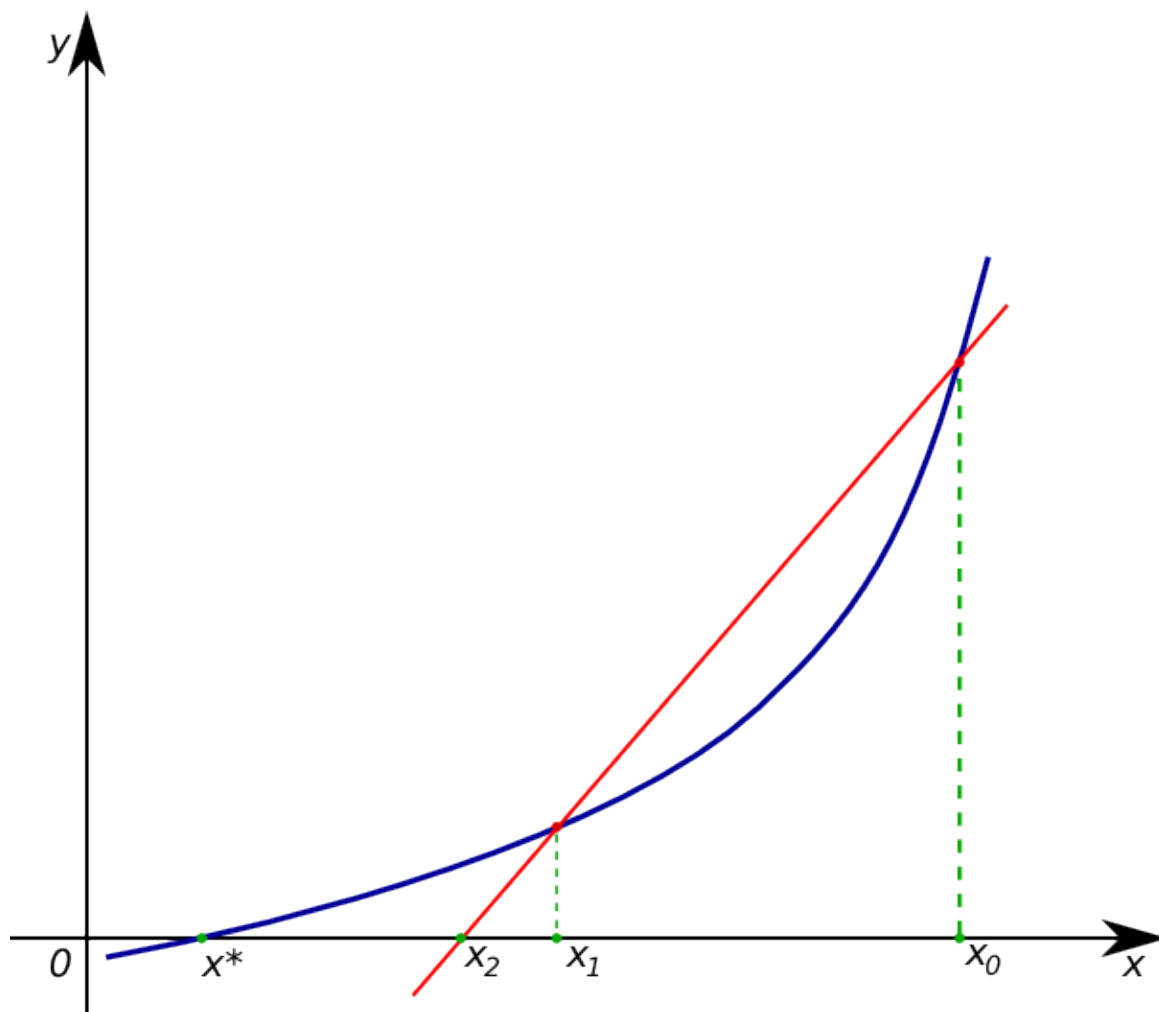


Рисунок 3.5. Метод хорд.

Пусть уравнение $F(x) = 0$ имеет единственный корень на отрезке $[a, b]$. Выберем начальные значения $x_0, x_1 \in [a, b]$. Элементы итерационной последовательности находятся по формуле

$$x_{i+1} = x_i - \frac{x_{i-1} - x_i}{F(x_{i-1}) - F(x_i)} F(x_i), \quad (i = 1, 2, \dots).$$

3.9. Задания для самостоятельной работы

1. Локализовать корни уравнения $F(x) = 0$ графически.
2. Объявить функцию для локализации корней уравнения $F(x) = 0$ на отрезке $[a, b]$ с шагом h .
3. Объявить функцию для нахождения корня уравнения $F(x) = 0$ на отрезке $[a, b]$ с заданной точностью ϵ методом деления отрезка пополам. Локализовать корни уравнения $F(x) = 0$ и уточнить их с помощью метода деления отрезка пополам с точностью $\epsilon = 10^{-5}$.

4. Локализовать корни и решить уравнение $F(x) = 0$ методом простых итераций. Оценить погрешность.

5. Локализовать корни и решить уравнение $F(x) = 0$ модифицированным методом Ньютона.

6. Локализовать корни и решить уравнение $F(x) = 0$ методом Ньютона.

7. Локализовать корни и решить уравнение $F(x) = 0$ методом хорд.

Варианты

1. $(0.2x)^3 - \cos x = 0$.

2. $x - 10 \sin x = 0$ при $-10 \leq x \leq -2$.

3. $2^{-x} - \sin x = 0$ при $x < 8$.

4. $2^x - 2 \cos x = 0$ при $x > -6$.

5. $\lg(x + 5) = \cos x$.

6. $\sqrt{4x + 7} = 3 \cos x$ при $x > 0$.

7. $x \sin x - 1 = 0$ при $-7.5 \leq x \leq 0$.

8. $8 \cos x - x = 6$ при $x > -6$.

9. $\sin x - 0.2x = 0$.

10. $10 \cos x - 0.1x^2 = 0$ при $-8 \leq x \leq 0$.

11. $2\lg(x + 7) - 5\sin x = 0$ при $-8 \leq x \leq 5$.

12. $4 \cos x + 0.3x = 0$ при $-3 \leq x \leq 5$.

13. $4 \sin x = \sqrt{1 - x}$ при $x > -8$.

14. $2x^2 - 5 = 2^x$.

15. $2^{-x} = \cos\left(\frac{x}{1.5} + 5\right)$ при $x < 10$.

Вопросы к защите лабораторной работы

1. Постановка задачи решения нелинейных уравнений. Основные этапы решения задачи.
2. Локализация (отделение) корней уравнения.
3. Метод деления отрезка пополам.
4. Метод простых итераций. Достаточные условия сходимости. Оценка погрешности.
5. Графическая иллюстрация сходящегося и расходящегося итерационного процесса.
6. Преобразование уравнения к итерационному виду.
7. Метод Ньютона. Связь между методом Ньютона и методом итераций. Достаточные условия сходимости. Графическая иллюстрация.
8. Модифицированный метод Ньютона. Достаточные условия сходимости. Графическая иллюстрация.
9. Метод хорд. Графическая иллюстрация.

4. Прямые методы решения систем линейных алгебраических уравнений

4.1. Метод обратной матрицы

Решение системы линейных алгебраических уравнений вида

$$A \cdot X = B$$

методом обратной матрицы находится по формуле

$$X = A^{-1} \cdot B.$$

4.2. Метод Гаусса

Рассмотрим систему линейных алгебраических уравнений

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = a_{1n+1}, \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = a_{2n+1}, \\ \dots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = a_{nn+1}. \end{cases} \quad (9)$$

Метод Гаусса состоит из двух этапов. Первый этап заключается в преобразовании системы (9) к треугольному виду (прямой ход метода Гаусса)

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = a_{1n+1}, \\ \quad a_{22}^{(1)}x_2 + \dots + a_{2n}^{(1)}x_n = a_{2n+1}^{(1)}, \\ \quad \quad \dots \\ \quad \quad \quad a_{nn}^{(n-1)}x_n = a_{nn+1}^{(n-1)}. \end{cases} \quad (10)$$

Получим расчетные формулы для выполнения прямого хода метода Гаусса. Разделим первую строку на элемент a_{11} ($a_{11} \neq 0$). Элемент a_{11} называют ведущим или главным элементом.

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = a_{1n+1}, : a_{11} \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = a_{2n+1}, \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n = a_{nn+1}. \end{cases}$$

$$\begin{cases} x_1 + \frac{a_{12}}{a_{11}}x_2 + \dots \frac{a_{1n}}{a_{11}}x_n &= \frac{a_{1n+1}}{a_{11}}, \\ a_{21}x_1 + a_{22}x_2 + \dots a_{2n}x_n &= a_{2n+1}, \\ \vdots & \\ a_{n1}x_1 + a_{n2}x_2 + \dots a_{nn}x_n &= a_{nn+1}, \end{cases}$$

Система после первого шага прямого хода метода Гаусса

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots a_{1n}x_n &= a_{1n+1}, \\ a_{22}^{(1)}x_2 + \dots a_{2n}^{(1)}x_n &= a_{2n+1}^{(1)}, \\ \vdots & \\ a_{nn}^{(1)}x_n &= a_{nn+1}^{(1)}, \end{cases}$$

$$a_{ij}^{(1)} = a_{ij} - a_{i1} \cdot \frac{a_{1j}}{a_{11}}, (i = 2, \dots, n, j = n + 1, \dots, 1)$$

После успешного проведения всех $(n-1)$ шагов прямого хода метода Гаусса получим систему вида

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \dots a_{1n}x_n &= a_{1n+1}, \\ a_{22}^{(1)}x_2 + a_{23}^{(1)}x_3 + \dots a_{2n}^{(1)}x_n &= a_{2n+1}^{(1)}, \\ \vdots & \\ a_{kk}^{(k-1)}x_k + \dots a_{kn}^{(k-1)}x_n &= a_{kn+1}^{(k-1)}, \\ \vdots & \\ a_{nn}^{(n-1)}x_n &= a_{nn+1}^{(n-1)}, \end{cases} \quad (11)$$

Расчетные формулы для преобразования системы (9) к виду (11) имеют вид

Для каждого $k = 1, 2, \dots, n-1$, выполнить преобразования

$$a_{ij}^{(k)} = a_{ij}^{(k-1)} - a_{ik}^{(k-1)} \cdot \frac{a_{kj}^{(k-1)}}{a_{kk}^{(k-1)}},$$

$$(i = k + 1, k + 2, \dots, n, j = n + 1, n, \dots, k)$$

Второй этап состоит в последовательном исключении неизвестных $x_n, x_{n-1}, \dots, x_1$ из системы треугольного вида (обратный ход метода Гаусса).

Расчетные формулы для нахождения x_i имеют вид

При $i = n$

$$x_n = \frac{a_{nn+1}^{(n-1)}}{a_{nn}^{(n-1)}}$$

При $i = n-1, n-2, \dots, 1$

$$x_i = \frac{a_{in+1}^{(i-1)} - \sum_{j=i+1}^n a_{ij}^{(i-1)} x_j}{a_{ii}^{i-1}} \quad (12)$$

4.3. Метод прогонки решения систем с трехдиагональными матрицами коэффициентов

Рассмотрим систему линейных алгебраических уравнений вида:

$$b_i x_{i-1} + c_i x_i + d_i x_{i+1} = r_i \quad (i = 1, 2, \dots, n, \quad b_1 = 0, \quad d_n = 0). \quad (13)$$

Метод прогонки заключается в нахождении коэффициентов δ_i, λ_i по формулам

$$\begin{aligned} \delta_1 &= -\frac{d_1}{c_1}, \delta_i = -\frac{d_i}{c_i + b_i \delta_{i-1}} \quad (i = 2, \dots, n-1), \delta_n = 0, \\ \lambda_1 &= -\frac{r_1}{c_1}, \lambda_i = \frac{r_i - b_i \lambda_{i-1}}{c_i + b_i \delta_{i-1}} \quad (i = 2, \dots, n) \end{aligned}$$

и получении x_i по формуле

$$x_n = \lambda_n = \frac{r_n - b_n \lambda_{n-1}}{c_n + b_n \delta_{n-1}}, x_i = \delta_i x_{i+1} + \lambda_i \quad (i = n-1, n-2, \dots, 1).$$

4.4. Задания для самостоятельной работы

1. Объявить функцию для решения системы линейных алгебраических уравнений методом Гаусса.
2. Объявить функцию для решения системы линейных алгебраических уравнений методом Гаусса с выбором главного элемента.
3. Объявить функцию для решения системы линейных алгебраических уравнений методом Жордана-Гаусса.
4. Объявить функцию для решения системы линейных алгебраических уравнений с трехдиагональными матрицами коэффициентов методом прогонки.
5. Объявить функцию для вычисления определителя матрицы, используя метод Гаусса.
6. Объявить функцию для нахождения обратной матрицы.

Варианты

$$1. \begin{cases} x_1 + x_2 + 2x_3 + 3x_4 = 1, \\ 3x_1 - x_2 - x_3 - 2x_4 = -4, \\ 2x_1 + 3x_2 - x_3 - x_4 = -6, \\ x_1 + 2x_2 + 3x_3 - x_4 = -4. \end{cases}$$

$$2. \begin{cases} x_1 + 2x_2 + 3x_3 - 2x_4 = 6, \\ x_1 - x_2 - 2x_3 - 3x_4 = -2, \\ 3x_1 + 2x_2 - x_3 + 2x_4 = 4, \\ 2x_1 - 3x_2 + 2x_3 + x_4 = 1. \end{cases}$$

$$3. \begin{cases} x_1 + 2x_2 + 3x_3 + 4x_4 = 5, \\ 2x_1 + x_2 + 2x_3 + 3x_4 = 1, \\ 3x_1 + 2x_2 + x_3 + 2x_4 = 1, \\ 4x_1 + 3x_2 + 2x_3 + x_4 = -5. \end{cases}$$

$$4. \begin{cases} x_1 + 3x_2 + 5x_3 + 7x_4 = -8, \\ 3x_1 + 5x_2 + 7x_3 + x_4 = 0, \\ 5x_1 + 7x_2 + x_3 + 3x_4 = 8, \\ 7x_1 + x_2 + 3x_3 + 5x_4 = 0. \end{cases}$$

$$5. \begin{cases} x_1 + 5x_2 + 3x_3 - 4x_4 = 20, \\ 3x_1 + x_2 - 2x_3 = 9, \\ 5x_1 - 7x_2 + 10x_4 = -9, \\ 3x_2 - 5x_3 = 1. \end{cases}$$

$$6. \begin{cases} 2x_1 + x_2 - 5x_3 + x_4 = 8, \\ x_1 - 3x_2 - 6x_4 = 9, \\ 2x_2 - x_3 + 2x_4 = -5, \\ x_1 + 4x_2 - 7x_3 + 6x_4 = 0. \end{cases}$$

$$7. \begin{cases} 2x_1 - x_2 + 3x_3 + 2x_4 = 4, \\ 3x_1 + 3x_2 + 3x_3 + 2x_4 = 6, \\ 3x_1 - x_2 - x_3 + 2x_4 = 6, \\ 3x_1 - x_2 + 3x_3 - x_4 = 6. \end{cases}$$

$$8. \begin{cases} 4x_1 + x_2 - x_4 = -9, \\ x_1 - 3x_2 + 4x_3 = -7, \\ 3x_2 - 2x_3 + 4x_4 = 12, \\ x_1 + 2x_2 - x_3 - 3x_4 = 0. \end{cases}$$

$$9. \begin{cases} x_1 + x_2 - x_3 - x_4 = 0, \\ x_2 + 2x_3 - x_4 = 2, \\ x_1 - x_2 - x_4 = -1, \\ -x_1 + 3x_2 - 2x_3 = 0. \end{cases}$$

$$10. \begin{cases} 5x_1 + x_2 - x_4 = -9, \\ 3x_1 - 3x_2 + x_3 + 4x_4 = -7, \\ 3x_1 - 2x_3 + x_4 = -16, \\ x_1 - 4x_2 + x_4 = 0. \end{cases}$$

$$11. \begin{cases} 2x_1 + x_3 + 4x_4 = 9, \\ x_1 + 2x_2 - x_3 + x_4 = 8, \\ 2x_1 + x_2 + x_3 + x_4 = 5, \\ x_1 - x_2 + 2x_3 + x_4 = -1. \end{cases}$$

$$12. \begin{cases} 2x_1 - 6x_2 + 2x_3 + 2x_4 = 12, \\ x_1 + 3x_2 + 5x_3 + 7x_4 = 12, \\ 3x_1 + 5x_2 + 7x_3 + x_4 = 0, \\ 5x_1 + 7x_2 + x_3 + 3x_4 = 4. \end{cases}$$

$$13. \begin{cases} x_1 + 5x_2 = 2, \\ 2x_1 - x_2 + 3x_3 + 2x_4 = 4, \\ 3x_1 - x_2 - x_3 + 2x_4 = 6, \\ 3x_1 - x_2 + 3x_3 - x_4 = 6. \end{cases}$$

$$14. \begin{cases} x_1 - 4x_2 - x_4 = 2, \\ x_1 + x_2 + 2x_3 + 3x_4 = 1, \\ 2x_1 + 3x_2 - x_3 - x_4 = -6, \\ x_1 + 2x_2 + 3x_3 - x_4 = -4. \end{cases}$$

$$15. \begin{cases} 5x_1 - x_2 + x_3 + 3x_4 = -4, \\ x_1 + 2x_2 + 3x_3 - 2x_4 = 6, \\ 2x_1 - x_2 - 2x_3 - 3x_4 = 8, \\ 3x_1 + 2x_2 - x_3 + 2x_4 = 4. \end{cases}$$

Вопросы к защите лабораторной работы

1. Метод обратной матрицы.
2. Метод Крамера.
3. Метод Гаусса. Метод Гаусса с выбором главного элемента. Метод Жордана-Гаусса.
4. Вычисление определителей с помощью метода Гаусса.
5. Нахождение обратных матриц.
6. Метод прогонки решения систем с трехдиагональными матрицами коэффициентов.

5. Итерационные методы решения систем уравнений

5.1. Итерационные методы решения систем линейных алгебраических уравнений

Метод простых итераций

Рассмотрим систему линейных алгебраических уравнений

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1m}x_m = b_1, \\ \dots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mm}x_m = b_m. \end{cases} \quad (14)$$

Систему (14) запишем в матричном виде

$$A \cdot X = B.$$

Будем считать, что система (14) имеет единственное решение.

Преобразуем эту систему к равносильной

$$\begin{cases} x_1 = \alpha_{11}x_1 + \alpha_{12}x_2 + \cdots + \alpha_{1m}x_m + \beta_1, \\ \dots \\ x_m = \alpha_{m1}x_1 + \alpha_{m2}x_2 + \cdots + \alpha_{mm}x_m + \beta_m. \end{cases}$$

Последнюю систему запишем в матричном виде

$$X = C \cdot X + D.$$

Выберем начальное приближение решения

$$X_0 = \begin{pmatrix} x_1^0 \\ \vdots \\ x_m^0 \end{pmatrix}.$$

Для вычисления следующих приближений будем использовать итерационную формулу

$$\begin{cases} x_1^{n+1} = \alpha_{11}x_1^n + \alpha_{12}x_2^n + \cdots + \alpha_{1m}x_m^n + \beta_1, \\ \dots \\ x_m^{n+1} = \alpha_{m1}x_1^n + \alpha_{m2}x_2^n + \cdots + \alpha_{mm}x_m^n + \beta_m, \end{cases} \\ n = 0, 1, 2, \dots$$

или в матричном виде

$$X_{n+1} = C \cdot X_n + D, \quad n = 0, 1, 2, \dots$$

В силу теоремы Банаха о сжимающем отображении эта последовательность будет сходиться к решению системы, если $\|C\| < 1$, а оценка погрешности n -го приближения ΔX_n может определяться равенством

$$\Delta X_n = \frac{\alpha}{1 - \alpha} \|X_{k-1} - X_k\|, \quad \alpha = \|C\|.$$

Пример. Решить систему линейных алгебраических уравнений методом простых итераций с точностью 0.0001.

$$\begin{cases} 100x_1 + 6x_2 - x_3 = 200, \\ 6x_1 + 200x_2 - 10x_3 = 600, \\ x_1 + 2x_2 - 100x_3 = 500. \end{cases}$$

Преобразуем эту систему к равносильной

$$\begin{cases} x_1 = -0.06x_2 + 0.01x_3 + 2, \\ x_2 = -0.03x_1 + 0.05x_3 + 3, \\ x_3 = 0.01x_1 + 0.02x_2 - 5. \end{cases}$$

Проверим условие сжимаемости отображения F .

$$\begin{aligned} \alpha &= \left\| \begin{pmatrix} 0 & -0.06 & 0.01 \\ -0.03 & 0 & 0.05 \\ 0.01 & 0.02 & 0 \end{pmatrix} \right\| = \\ &= \sqrt{\sum_{i=1}^3 \sum_{j=1}^3 (C_{i,j})^2} \approx 0.087 < 1. \end{aligned}$$

Следовательно, F — сжимающее отображение.

Реализовать итерационный процесс на python можно следующим образом:

```

1  from numpy import array, dot, linalg
2  i = 1
3  e = 0.0001
4  alpha = 0.087
5  X = array([[2], [3], [-5]], float)
6  C = array([[0, -0.06, 0.01], [-0.03, 0, 0.05], [0.01, 0.02, 0]],\
7           float)
8  D = array([[2], [3], [-5]], float)
9  X1 = dot(C, X) + D
10 while linalg.norm(X1 - X) > e * (1 - alpha) / alpha:
11     X = X1
12     X1 = dot(C, X) + D
13     i = i + 1
14 print("X=\n", X1)

```

Листинг 127: Решение СЛАУ методом итераций

Результат:

alpha=0.0076

x1=1.78866

x2=2.69989

x3=-4.92809

Метод Зейделя

Метод Зейделя является модификацией метода простых итераций. Отличие состоит в том, что на каждом шаге итерационного процесса сразу же учитываются вычисленные значения x_i ($i = 1, 2, \dots, m$). Итерационная формула имеет вид

$$\begin{cases} x_1^{n+1} = \alpha_{11}x_1^n + \alpha_{12}x_2^n + \dots + \alpha_{1m}x_m^n + \beta_1, \\ x_2^{n+1} = \alpha_{21}x_1^{n+1} + \alpha_{22}x_2^n + \dots + \alpha_{2m}x_m^n + \beta_2, \\ \dots \\ x_m^{n+1} = \alpha_{m1}x_1^{n+1} + \alpha_{m2}x_2^{n+1} + \dots + \alpha_{mm}x_m^n + \beta_m, \end{cases}$$

$$n = 0, 1, 2, \dots$$

Пример. Решить систему линейных алгебраических уравнений методом Зейделя с точностью 10^{-4} .

$$\begin{cases} x_1 + x_2 + x_3 = 3, \\ 2x_1 + x_2 + x_3 = 4, \\ x_1 + 3x_2 + x_3 = 5. \end{cases}$$

5.2. Решение систем нелинейных уравнений

Использование библиотечной функции fsolve

Использование библиотечной функции fsolve

Функцию fsolve можно использовать как для решения нелинейных уравнений, так и для решения систем нелинейных уравнений.

Пример. Решить систему уравнений

$$\begin{cases} 3\cos(x_1) + 3x_0 = 4.5, \\ 4x_1 - 2\sin(x_0 - 0.5) = 2, \end{cases}$$

если $x_0, x_1 \in [0, \frac{\pi}{3}]$.

```
1 from numpy import sin, cos, zeros
2 from scipy.optimize import fsolve
3 def F(X):
4     F = zeros(len(X), float)
5     F[0] = 3 * cos(X[1]) + 3 * X[0] - 4.5
6     F[1] = 4 * X[1] - 2 * sin(X[0] - 0.5) - 2
7     return F
8 X = fsolve(F, [0, 0])
9 print("X=\n", X.reshape(2, 1))
```

Листинг 128: Решение систем нелинейных уравнений с помощью встроенной функции fsolve

Результат:

X=

```
[[0.66459448]
 [0.58192615]]
```

Метод простых итераций

Рассмотрим систему нелинейных уравнений

$$\begin{cases} F_1(x_1, x_2, \dots, x_m) = 0, \\ \dots \\ F_m(x_1, x_2, \dots, x_m) = 0. \end{cases} \Leftrightarrow F(X) = 0. \quad (15)$$

Преобразуем эту систему к равносильной

$$\begin{cases} x_1 = \varphi_1(x_1, x_2, \dots, x_m), \\ \dots \\ x_m = \varphi_m(x_1, x_2, \dots, x_m). \end{cases} \Leftrightarrow \varphi(X) = 0. \quad (16)$$

Выберем начальное приближение решения

$$X_0 = \begin{pmatrix} x_1^0 \\ \vdots \\ x_m^0 \end{pmatrix}.$$

Для вычисления следующих приближений будем использовать итерационную формулу

$$X_{n+1} = \varphi(X_n), \quad n = 0, 1, 2, \dots$$

Эта последовательность будет сходиться к решению системы, если отображение $F = \varphi(X)$ будет сжимающим, например, если

$$\sup_{X \in L} \|\varphi'(X)\| < 1,$$

где L - отрезок, соединяющий точки X_1 и X_2 , функция $\varphi(X)$ дифференцируема в некоторой области $G \supset L$ и $\varphi'(X)$ - матрица Якоби системы (16).

$$\varphi'(X) = \begin{pmatrix} \frac{\partial \varphi_1(X)}{\partial x_1} & \frac{\partial \varphi_1(X)}{\partial x_2} & \dots & \frac{\partial \varphi_1(X)}{\partial x_m} \\ \dots & \dots & \dots & \dots \\ \frac{\partial \varphi_m(X)}{\partial x_1} & \frac{\partial \varphi_m(X)}{\partial x_2} & \dots & \frac{\partial \varphi_m(X)}{\partial x_m} \end{pmatrix}.$$

Из теоремы Банаха следует, что оценка погрешности n -го приближения ΔX_n может определяться равенством

$$\Delta X_n = \frac{\alpha}{1 - \alpha} \|X_{k-1} - X_k\|, \quad \alpha = \sup_{X \in L} \|\varphi'(X)\|.$$

Пример. Решить систему уравнений

$$\begin{cases} 3\cos(x_1) + 3x_0 = 4.5, \\ 4x_1 - 2\sin(x_0 - 0.5) = 2 \end{cases}$$

с точностью 0.0001, если $x_0, x_1 \in [0, \frac{\pi}{3}]$.

Преобразуем эту систему к равносильной

$$\begin{cases} x_0 = \frac{4.5-3\cos(x_1)}{3}, \\ x_1 = \frac{2+2\sin(x_0-0.5)}{4}. \end{cases}$$

Проверим условие сжимаемости отображения φ .

$$\varphi'(X) = \begin{pmatrix} 0 & \sin(x_1) \\ \frac{1}{2}\cos(x_0 - 0.5) & 0 \end{pmatrix},$$

$$\alpha = \sup_{X \in L} \|\varphi'(X)\| \approx 0.866 < 1.$$

Следовательно, φ — сжимающее отображение.

Итерационный процесс на Python можно записать следующим образом

```

1  from numpy import sin, cos, array, linalg
2  i = 1
3  e = 0.0001
4  alpha = 0.866
5  x0 = 0
6  x1 = 0
7  X = array([[x0], [x1]], float)
8  x0 = (4.5 - 3 * cos(x1)) / 3.0
9  x1 = (2 + 2 * sin(x0 - 0.5)) / 4.0
10 X1 = array([[x0], [x1]], float)
11 while linalg.norm(X1 - X) > e * (1 - alpha) / alpha:
12     X = X1
13     x0 = (4.5 - 3 * cos(x1)) / 3.0
14     x1 = (2 + 2 * sin(x0 - 0.5)) / 4.0
15     X1 = array([[x0], [x1]], float)
16     i = i + 1
17 print("X=\n", X1)

```

Листинг 129: Решение систем нелинейных уравнений методом итераций

Результат:

X=

```

[[0.66459002]
 [0.58192395]]

```

Метод Ньютона

Рассмотрим систему нелинейных уравнений

$$\begin{cases} F_1(x_1, x_2, \dots, x_m) = 0, \\ \dots \\ F_m(x_1, x_2, \dots, x_m) = 0. \end{cases} \Leftrightarrow F(X) = 0. \quad (17)$$

Выберем начальное приближение решения

$$X_0 = \begin{pmatrix} x_1^0 \\ \vdots \\ x_m^0 \end{pmatrix}.$$

Для вычисления следующих приближений будем использовать итерационную формулу

$$X_{n+1} = X_n - (F'(X_n))^{-1} \cdot F(X_n), \quad n = 0, 1, 2, \dots,$$

где $(F'(X_n))^{-1}$ — матрица, обратная матрице Якоби системы (17).

Условия сходимости и оценка погрешности n -го приближения такие же, как в методе простых итераций.

Пример. Решить систему уравнений

$$\begin{cases} 3\cos(x_1) + 3x_0 = 4.5, \\ 4x_1 - 2\sin(x_0 - 0.5) = 2, \end{cases}$$

если $x_0, x_1 \in [0, \frac{\pi}{3}]$.

```
1 from numpy import sin, cos, array, dot, linalg
2 X = array([[0], [0]], float)
3 for i in range(100):
4     dF = array([[3, -3 * sin(X[1])], \
5                 [-2 * cos(X[0] - 0.5), 4]], float)
6     F = array([3 * X[0] + 3 * cos(X[1]) - 4.5, \
7                4 * X[1] - 2 * sin(X[0] - 0.5) - 2], float)
8     X = X - dot(linalg.inv(dF), F)
9 print("X=\n", X)
```

Листинг 130: Решение систем нелинейных уравнений методом Ньютона

Результат:

X=

[[0.66459448]

[0.58192615]]

Модифицированный метод Ньютона

Рассмотрим систему нелинейных уравнений

$$\begin{cases} F_1(x_1, x_2, \dots, x_m) = 0, \\ \dots \\ F_m(x_1, x_2, \dots, x_m) = 0. \end{cases} \Leftrightarrow F(X) = 0. \quad (18)$$

Выберем начальное приближение решения

$$X_0 = \begin{pmatrix} x_1^0 \\ \vdots \\ x_m^0 \end{pmatrix}.$$

Для вычисления следующих приближений будем использовать итерационную формулу

$$X_{n+1} = X_n - (F'(X_n))^{-1} \cdot F(X_n), \quad n = 0, 1, 2, \dots,$$

где $(F'(X_n))^{-1}$ — матрица, обратная матрице Якоби системы (18).

Условия сходимости и оценка погрешности n -го приближения такие же, как в методе простых итераций.

5.3. Задания для самостоятельной работы

1. Решить систему линейных алгебраических уравнений методом простых итераций с точностью $\epsilon = 0,00001$.

2. Решить систему линейных алгебраических уравнений методом Зейделя с точностью $\epsilon = 0,00001$.

3. Решить систему нелинейных уравнений методом простых итераций. Оценить погрешность. Сделать проверку.

4. Решить систему нелинейных уравнений методом Ньютона. Сделать проверку.

5. Решить систему нелинейных уравнений модифицированным методом Ньютона. Сделать проверку.

Варианты

Задания 1-2

$$1. \begin{cases} 50x_1 + x_2 + 2x_3 + 3x_4 = 70, \\ 3x_1 - 100x_2 - x_3 - 2x_4 = -208, \\ 2x_1 + 3x_2 - 50x_3 - x_4 = -146, \\ x_1 + 2x_2 + 3x_3 - 40x_4 = 174. \end{cases}$$

$$2. \begin{cases} 40x_1 + 2x_2 + 3x_3 - 2x_4 = 35, \\ x_1 - 50x_2 - 2x_3 - 3x_4 = -53, \\ 3x_1 + 2x_2 - 100x_3 + 2x_4 = 109, \\ 2x_1 - 3x_2 + 2x_3 + x_4 = -1. \end{cases}$$

$$3. \begin{cases} 60x_1 + 2x_2 + 3x_3 + 4x_4 = 67, \\ 2x_1 + 20x_2 + 2x_3 + 3x_4 = 26, \\ 3x_1 + 2x_2 - 40x_3 + 2x_4 = 49, \\ 2x_1 + 3x_2 + 2x_3 + 100x_4 = 203. \end{cases}$$

$$4. \begin{cases} 100x_1 + 3x_2 + 5x_3 + 7x_4 = -75, \\ 3x_1 + 50x_2 + 7x_3 + x_4 = 106, \\ 5x_1 + 7x_2 + 20x_3 + 3x_4 = 35, \\ 7x_1 + x_2 + 3x_3 + 5x_4 = 8. \end{cases}$$

$$5. \begin{cases} 20x_1 + 5x_2 + 3x_3 - 4x_4 = -6, \\ 3x_1 + 40x_2 - 2x_3 = 115, \\ 5x_1 - 7x_2 + 100x_3 + 10x_4 = 84, \\ 3x_1 - 5x_3 + 50x_4 = 42. \end{cases}$$

$$6. \begin{cases} 10x_1 + x_2 - 5x_3 + x_4 = -4, \\ x_1 - 20x_2 - 6x_4 = -33, \\ 2x_2 - 40x_3 + 2x_4 = -118, \\ x_1 + 4x_2 - 7x_3 + 60x_4 = -72. \end{cases}$$

$$7. \begin{cases} 20x_1 - x_2 + 3x_3 + 2x_4 = 35, \\ 3x_1 - 40x_2 + 3x_3 + 2x_4 = -60, \\ 3x_1 - 2x_2 + 60x_3 + 2x_4 = 187, \\ 3x_1 - x_2 + 3x_3 + 40x_4 = 170. \end{cases}$$

$$8. \begin{cases} 40x_1 + x_2 - x_4 = 41, \\ x_1 - 20x_2 + 4x_3 = -47, \\ 3x_1 - 2x_2 + 60x_3 + 4x_4 = -117, \\ x_1 + 2x_2 - x_3 - 60x_4 = -53. \end{cases}$$

$$9. \begin{cases} 10x_1 + x_2 - x_3 - x_4 = 11, \\ x_2 + 20x_2 - x_4 = 38, \\ x_1 - x_2 + 40x_3 - x_4 = -84, \\ -x_1 + 3x_2 - 2x_3 + 80x_4 = 249. \end{cases}$$

$$10. \begin{cases} 50x_1 + x_2 - x_4 = -100, \\ 3x_1 - 20x_2 + x_3 + 4x_4 = -20, \\ 3x_1 - 40x_3 + x_4 = -85, \\ x_1 - 4x_2 + x_3 + 60x_4 = 56. \end{cases}$$

$$11. \begin{cases} -20x_1 + x_3 + 4x_4 = -14, \\ x_1 + 20x_2 - x_3 + x_4 = -20, \\ 2x_1 + x_2 + 40x_3 + x_4 = 82, \\ x_1 - x_2 + 2x_3 + 10x_4 = 16. \end{cases}$$

$$12. \begin{cases} 20x_1 - 6x_2 + 2x_3 + 2x_4 = 24, \\ x_1 + 60x_2 + 5x_3 + 7x_4 = -62, \\ 3x_1 + 5x_2 + 40x_3 + x_4 = -81, \\ 5x_1 + 7x_2 + x_3 - 10x_4 = -14. \end{cases}$$

$$13. \begin{cases} 10x_1 + 5x_2 = 20, \\ 2x_1 - 20x_2 + 3x_3 + 2x_4 = -42, \\ 3x_1 - x_2 - 40x_3 + 2x_4 = 83, \\ 3x_1 - x_2 + 3x_3 - 60x_4 = -65. \end{cases}$$

$$14. \begin{cases} 10x_1 - 4x_2 - x_4 = 1, \\ x_1 - 20x_2 + 2x_3 + 3x_4 = -34, \\ 2x_1 + 3x_2 - 40x_3 - x_4 = -33, \\ x_1 + 2x_2 + 3x_3 + 50x_4 = 58. \end{cases}$$

$$15. \begin{cases} 5x_1 - x_2 + x_3 + 3x_4 = -4, \\ x_1 + 2x_2 + 3x_3 - 2x_4 = 6, \\ 2x_1 - x_2 - 2x_3 - 3x_4 = 8, \\ 3x_1 + 2x_2 - x_3 + 2x_4 = 4. \end{cases}$$

Задание 3

$$1. \begin{cases} 50x_1 + x_2 + 2x_3 + 3x_4 = 70, \\ 3x_1 - 100x_2 - x_3 - 2x_4 = -208, \\ 2x_1 + 3x_2 - 50x_3 - x_4 = -146, \\ x_1 + 2x_2 + 3x_3 - 40x_4 = 174. \end{cases}$$

$$2. \begin{cases} 40x_1 + 2x_2 + 3x_3 - 2x_4 = 35, \\ x_1 - 50x_2 - 2x_3 - 3x_4 = -53, \\ 3x_1 + 2x_2 - 100x_3 + 2x_4 = 109, \\ 2x_1 - 3x_2 + 2x_3 + 10x_4 = 17. \end{cases}$$

$$3. \begin{cases} 60x_1 + 2x_2 + 3x_3 + 4x_4 = 67, \\ 2x_1 + 20x_2 + 2x_3 + 3x_4 = 26, \\ 3x_1 + 2x_2 - 40x_3 + 2x_4 = 49, \\ 2x_1 + 3x_2 + 2x_3 + 100x_4 = 203. \end{cases}$$

$$4. \begin{cases} 100x_1 + 3x_2 + 5x_3 + 7x_4 = -75, \\ 3x_1 + 50x_2 + 7x_3 + x_4 = 106, \\ 5x_1 + 7x_2 + 20x_3 + 3x_4 = 35, \\ 7x_1 + x_2 + 3x_3 + 5x_4 = 8. \end{cases}$$

$$5. \begin{cases} 20x_1 + 5x_2 + 3x_3 - 4x_4 = -6, \\ 3x_1 + 40x_2 - 2x_3 = 115, \\ 5x_1 - 7x_2 + 100x_3 + 10x_4 = 84, \\ 3x_1 - 5x_3 + 50x_4 = 42. \end{cases}$$

$$6. \begin{cases} 10x_1 + x_2 - 5x_3 + x_4 = -4, \\ x_1 - 20x_2 - 6x_4 = -33, \\ 2x_2 - 40x_3 + 2x_4 = -118, \\ x_1 + 4x_2 - 7x_3 + 60x_4 = -72. \end{cases}$$

$$7. \begin{cases} 20x_1 - x_2 + 3x_3 + 2x_4 = 35, \\ 3x_1 - 40x_2 + 3x_3 + 2x_4 = -60, \\ 3x_1 - 2x_2 + 60x_3 + 2x_4 = 187, \\ 3x_1 - x_2 + 3x_3 + 40x_4 = 170. \end{cases}$$

$$8. \begin{cases} 40x_1 + x_2 - x_4 = 41, \\ x_1 - 20x_2 + 4x_3 = -47, \\ 3x_1 - 2x_2 + 60x_3 + 4x_4 = -117, \\ x_1 + 2x_2 - x_3 - 60x_4 = -53. \end{cases}$$

$$9. \begin{cases} 10x_1 + x_2 - x_3 - x_4 = 11, \\ x_2 + 20x_2 - x_4 = 38, \\ x_1 - x_2 + 40x_3 - x_4 = -84, \\ -x_1 + 3x_2 - 2x_3 + 80x_4 = 249. \end{cases}$$

$$10. \begin{cases} 50x_1 + x_2 - x_4 = -100, \\ 3x_1 - 20x_2 + x_3 + 4x_4 = -20, \\ 3x_1 - 40x_3 + x_4 = -85, \\ x_1 - 4x_2 + x_3 + 60x_4 = 56. \end{cases}$$

$$11. \begin{cases} -20x_1 + x_3 + 4x_4 = -14, \\ x_1 + 20x_2 - x_3 + x_4 = -20, \\ 2x_1 + x_2 + 40x_3 + x_4 = 82, \\ x_1 - x_2 + 2x_3 + 10x_4 = 16. \end{cases}$$

$$12. \begin{cases} 20x_1 - 6x_2 + 2x_3 + 2x_4 = 24, \\ x_1 + 60x_2 + 5x_3 + 7x_4 = -62, \\ 3x_1 + 5x_2 + 40x_3 + x_4 = -81, \\ 5x_1 + 7x_2 + x_3 - 10x_4 = -14. \end{cases}$$

$$13. \begin{cases} 10x_1 + 5x_2 = 20, \\ 2x_1 - 20x_2 + 3x_3 + 2x_4 = -42, \\ 3x_1 - x_2 - 40x_3 + 2x_4 = 83, \\ 3x_1 - x_2 + 3x_3 - 60x_4 = -65. \end{cases}$$

$$14. \begin{cases} 10x_1 - 4x_2 - x_4 = 1, \\ x_1 - 20x_2 + 2x_3 + 3x_4 = -34, \\ 2x_1 + 3x_2 - 40x_3 - x_4 = -33, \\ x_1 + 2x_2 + 3x_3 + 50x_4 = 58. \end{cases}$$

$$15. \begin{cases} 5x_1 - x_2 + x_3 + 3x_4 = -4, \\ x_1 + 2x_2 + 3x_3 - 2x_4 = 6, \\ 2x_1 - x_2 - 2x_3 - 3x_4 = 8, \\ 3x_1 + 2x_2 - x_3 + 2x_4 = 4. \end{cases}$$

Задания 4-7

$$1. \begin{cases} 2y - \sin(x - 0.5) = 1, \\ \cos(y) + x = 1, 5. \end{cases}$$

$$2. \begin{cases} \sin(x) - 2y = 1, \\ \cos(y + 0, 5) - x = 2. \end{cases}$$

$$3. \begin{cases} \sin(x) - 2y = 1, \\ \sin(y - 1) + x = 1, 3. \end{cases}$$

$$4. \begin{cases} \sin(x + y) - y - 1.2 = 0, \\ 2x + \cos(y) - 2 = 0. \end{cases}$$

$$5. \begin{cases} \sin(0.5x + y) - 1.2x - 1 = 0, \\ x^2 + y^2 - 1 = 0. \end{cases}$$

$$6. \begin{cases} \cos(x - 1) + y - 0.5 = 0, \\ \sin(x) + 2y - 2 = 0. \end{cases}$$

$$7. \begin{cases} \sin(x - 7) - 2y = -0, 294, \\ x - \sin(y + 2) - 1, 6 = -2, 346. \end{cases}$$

$$8. \begin{cases} \sin(x) + 2y - 2 = 0, \\ \cos(x) + y - 1.5 = 0. \end{cases}$$

$$9. \begin{cases} \sin(x + y) - 1.3x - 1 = 0, \\ x^2 + 0.2y^2 - 1 = 0. \end{cases}$$

10.
$$\begin{cases} \cos(x) + y - 1.5 = 0, \\ 2x - \sin(y - 0.5) - 1 = 0. \end{cases}$$
11.
$$\begin{cases} \cos(x + 0.7) + y - 2 = -1,96, \\ \sin(y) - 5x + 3 = 1,021. \end{cases}$$
12.
$$\begin{cases} \sin(x) - 2y - 3 = -2,601, \\ \cos(y - 1) + x - 1 = -0,346. \end{cases}$$
13.
$$\begin{cases} \sin(x + y) - 1.2x - 0.1 = 0, \\ x^2 + y^2 - 1 = 0. \end{cases}$$
14.
$$\begin{cases} \sin(x + y) - y - 1.5 = 0, \\ x + \cos(y - 0.5) - 0.5 = 0. \end{cases}$$
15.
$$\begin{cases} \cos(x + 0.5) + y - 1 = 0, \\ \sin(y) - 2x - 2 = 0. \end{cases}$$

Вопросы к защите лабораторной работы

1. Метод простых итераций. Достаточные условия сходимости. Оценка погрешности.
2. Метод Зейделя. Достаточные условия сходимости. Оценка погрешности.
3. Решение системы нелинейных уравнений методом простых итераций. Достаточное условие сходимости.
4. Решение системы нелинейных уравнений методом Ньютона.
5. Решение системы нелинейных уравнений мод. методом Ньютона.

6. Приближение функций

6.1. Постановка задачи

Пусть функция $f(x)$ задана таблицей

x	x_0	x_1	$\dots$	x_n
$f(x)$	y_0	y_1	$\dots$	y_n

(19)

Нужно вычислить значение функции $f(x)$, причем $x \in [x_0, x_n]$ и $x \neq x_i$ ($i = 0, 1, \dots, n$).

Для этого необходимо найти аналитическое выражение приближающей функции $F(x) \approx f(x)$, задача нахождения которой называется аппроксимацией.

Задача интерполяции состоит в нахождении приближающей функции при условии $f(x_i) = F(x_i)$ ($i = 0, 1, 2, \dots, n$). При этом точки x_i называются узлами интерполяции.

Экстраполяция состоит в нахождении значений приближающей функции за пределами отрезка $[x_0, x_n]$.

6.2. Встроенные функции SciPy

Пакет `scipy.interpolate` содержит различные возможности для проведения интерполяции:

класс `interp1d`, содержащий несколько методов для проведения одномерной интерполяции;

функцию `griddata` для проведения многомерной интерполяции;

функции для проведения одномерной и двухмерной интерполяции кубическими сплайнами, предоставляющие процедурный и объектно-ориентированный интерфейсы для доступа к библиотеке для языка Fortran FITPACK;

функции для проведения интерполяции с использованием радиальных базисных функций.

Наиболее простым способом проведения интерполяции является использование класса `interp1d`.

Функция принимает входные параметры `x`, `y`, `kind`, где `x` – вектор, содержащий монотонно возрастающие вещественные значения, `y` – массив, содержащий действительные значения, `kind` – вид интерполяции. Более подробно

о классе `interp1d` можно узнать с помощью функции `help`.

Пример. С использованием библиотечной функции SciPy `interp1d` провести кусочно-линейную интерполяцию для функции, заданной таблицей

x 1 2 3 4 5 6

y 0 3 4 1 2 5

Решение.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.interpolate import interp1d
4 x = np.array([1, 2, 3, 4, 5, 6], dtype = float)
5 y = np.array([0, 3, 4, 1, 2, 5], dtype = float)
6 f = interp1d(x, y)
7 xnew = np.linspace(1, 6, 300)
8 plt.plot(x, y, 'o', xnew, f(xnew))
9 plt.grid(True)
10 plt.show()
```

Листинг 131

Результат:

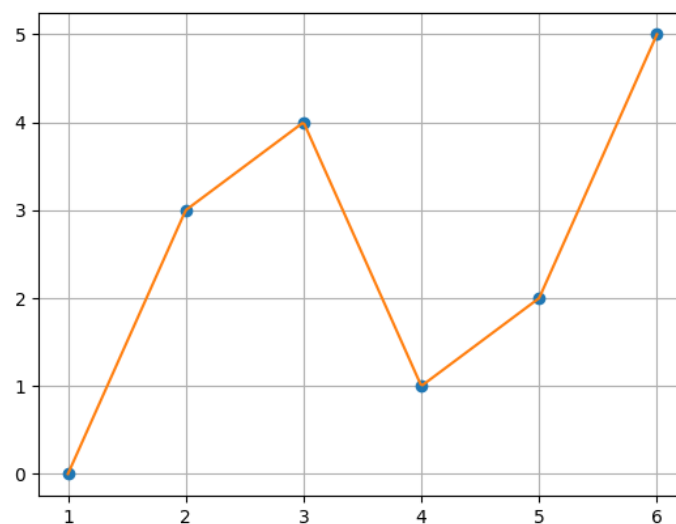


Рисунок 6.1. Линейная интерполяция с применением SciPy.

Пример. С использованием библиотечной функции SciPy `interp1d` провести интерполяцию кубическими сплайнами для функции, заданной таблицей

x 1 2 3 4 5 6

y 0 3 4 1 2 5

Решение

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.interpolate import interp1d
4  x = np.array([1, 2, 3, 4, 5, 6], dtype = float)
5  y = np.array([0, 3, 4, 1, 2, 5], dtype = float)
6  f = interp1d(x, y, kind = 'cubic')
7  xnew = np.linspace( 1, 6, 300)
8  plt.plot(x, y, 'o', xnew, f(xnew))
9  plt.grid(True)
10 plt.show()

```

Листинг 132

Результат:

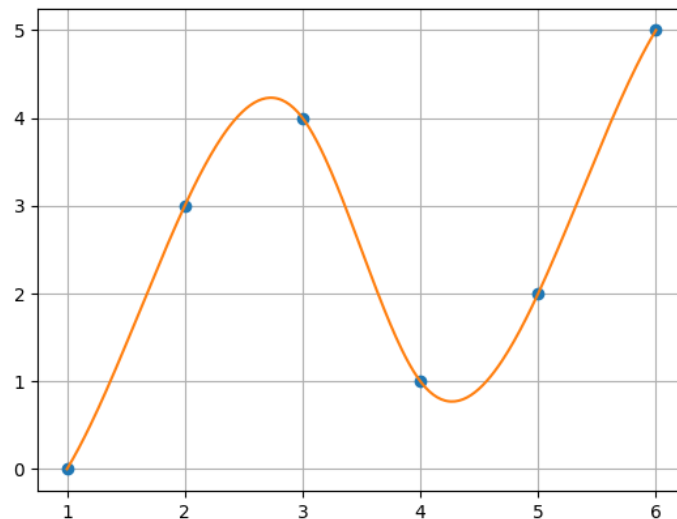


Рисунок 6.2. Интерполяция кубическими сплайнами с применением SciPy.

6.3. Интерполяционный многочлен Лагранжа

Пусть функция f задана таблицей (19). Интерполяционным многочленом Лагранжа называется многочлен

$$L_n(x) = \sum_{i=0}^n y_i \frac{(x - x_0) \dots (x - x_{i-1})(x - x_{i+1}) \dots (x - x_n)}{(x_i - x_0) \dots (x_i - x_{i-1})(x_i - x_{i+1}) \dots (x_i - x_n)}.$$

Введем обозначения

$$\prod_{n+1}(x) = (x - x_0)(x - x_1) \dots (x - x_n),$$

$$\prod_{n+1}'(x) = \sum_{i=0}^n (x - x_0) \dots (x - x_{i-1})(x - x_{i+1}) \dots (x - x_n).$$

При $x = x_i$ ($i = 0, 1, \dots, n$)

$$\prod_{n+1}'(x_i) = (x_i - x_0) \dots (x_i - x_{i-1})(x_i - x_{i+1}) \dots (x_i - x_n).$$

Тогда

$$\begin{aligned} L_n(x) &= \sum_{i=0}^n y_i \frac{\prod_{n+1}(x)}{(x - x_i) \cdot \prod_{n+1}'(x_i)} = \\ &= \prod_{n+1}(x) \sum_{i=0}^n \frac{y_i}{(x - x_i) \cdot \prod_{n+1}'(x_i)}. \end{aligned}$$

Пример. Построить график интерполяционного многочлена Лагранжа для функции, заданной таблицей

x	2	5	-6	7	4	3	8	9	1	-2
$f(x)$	-1	77	-297	249	33	9	389	573	-3	-21

Решение.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  x = np.array([2, 5, -6, 7, 4, 3, 8, 9, 1, -2], dtype = float)
4  y = np.array([-1, 77, -297, 249, 33, 9, 389, 573, -3, -21], \
5              dtype = float)
6  def lagranz(x, y, t):
7      z = 0
8      for j in range(len(y)):
9          p1 = 1; p2 = 1
10         for i in range(len(x)):
11             if i == j:
12                 p1 = p1 * 1; p2 = p2 * 1
13             else:
14                 p1 = p1 * (t - x[i])
15                 p2 = p2 * (x[j] - x[i])
16         z = z + y[j] * p1 / p2
17     return z
18  xnew = np.linspace(np.min(x), np.max(x), 100)
19  ynew = [lagranz(x,y,i) for i in xnew]
20  plt.plot(x, y, 'o', xnew, ynew)
21  plt.grid(True)
22  plt.show()

```

Результат:

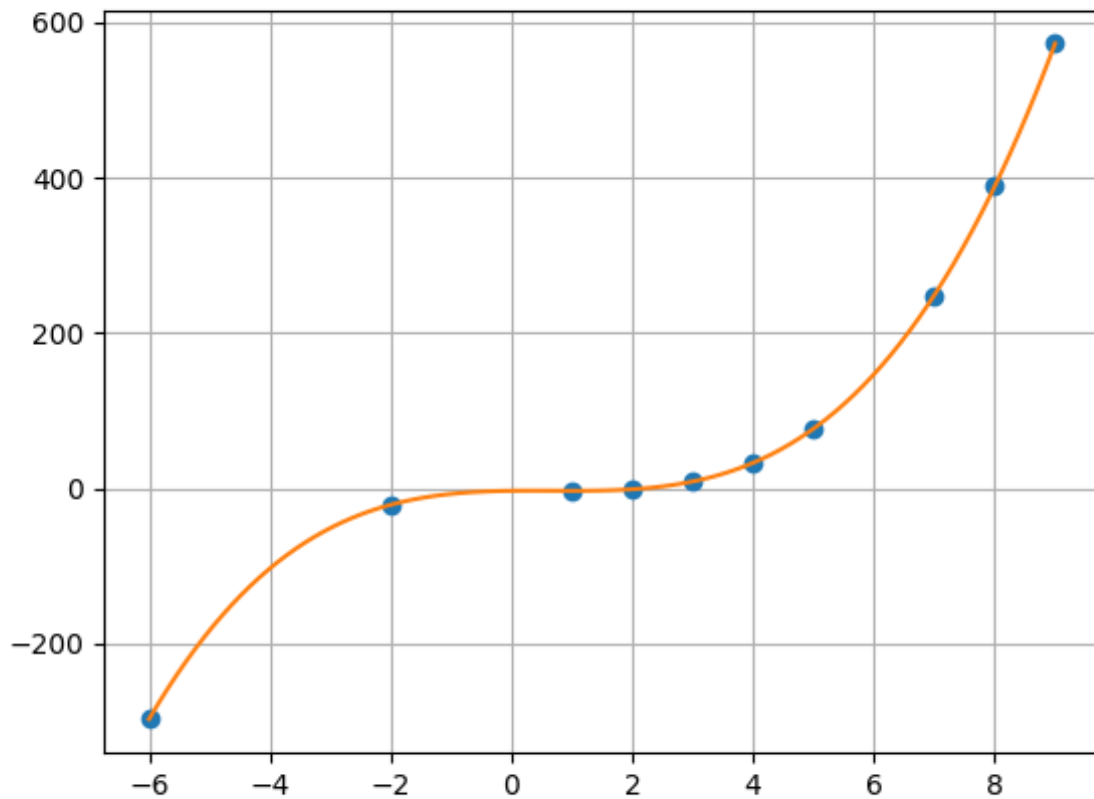


Рисунок 6.3. Интерполяционный многочлен Лагранжа.

6.4. Интерполяционные многочлены Ньютона для равноотстоящих узлов

Пусть функция задана таблицей

x	x_0	x_1	$\dots$	x_n
$f(x)$	y_0	y_1	$\dots$	y_n

где

$$h = x_{i+1} - x_i = \text{const} \ (i = 0, 1, \dots, n-1).$$

Конечными разностями первого порядка называются значения

$$\Delta y_i = y_{i+1} - y_i \ (i = 0, 1, \dots, n-1).$$

Конечными разностями второго порядка называются значения

$$\Delta^2 y_i = \Delta y_{i+1} - \Delta y_i \ (i = 0, 1, \dots, n-2).$$

Конечными разностями k -го порядка называются значения

$$\Delta^k y_i = \Delta^{k-1} y_{i+1} - \Delta^{k-1} y_i \quad (i = 0, 1, \dots, n - k).$$

Конечные разности любого порядка можно выразить через значения функции

$$\Delta^k y_i = y_{i+k} - k y_{i+k-1} + \frac{k(k-1)}{2!} y_{i+k-2} - \dots + (-1)^k y_i.$$

Первая интерполяционная формула Ньютона

Пусть функция задана таблицей с постоянным шагом. Первая интерполяционная формула Ньютона имеет вид

$$\begin{aligned} P_n(x) = & y_0 + \frac{\Delta y_0}{h} (x - x_0) + \frac{\Delta^2 y_0}{2! h^2} (x - x_0) (x - x_1) + \\ & \dots + \frac{\Delta^n y_0}{n! h^n} (x - x_0) \dots (x - x_{n-1}). \end{aligned}$$

Вводя обозначение $\frac{x-x_0}{h} = t$, получим

$$\begin{aligned} P_n(x) = P_n(x_0 + th) = & y_0 + t \Delta y_0 + \frac{t(t-1)}{2!} \Delta^2 y_0 + \\ & \dots + \frac{t(t-1) \dots (t-n+1)}{n!} \Delta^n y_0. \end{aligned}$$

Вторая интерполяционная формула Ньютона

$$\begin{aligned} P_n(x) = & P_n(x_n + th) \\ = & y_n + t \Delta y_{n-1} + \frac{t(t+1)}{2!} \Delta^2 y_{n-2} + \\ & \dots + \frac{t(t+1) \dots (t+n-1)}{n!} \Delta^n y_0. \end{aligned}$$

Пример.

Построить интерполяционный многочлен Ньютона для функции, заданной таблицей

x	1	2	3	4	5	6
$f(x)$	0	3	4	1	2	5

Решение.

```
1  import numpy as np
2
3  import matplotlib.pyplot as plt
4
5  x = np.array([1, 2, 3, 4, 5, 6], dtype = float)
6
7  y = np.array([0, 3, 4, 1, 2, 5], dtype = float)
8
9  def newton(x, y, t):
10     h = x[1] - x[0]
11     n = len(x)
12     DeltaY = np.zeros((n, n), dtype = float)
13
14     for j in range(n):
15         DeltaY[j, 0] = y[j]
16         k = 1
17         for k in range(1, n):
18             for j in range(n - k):
19                 DeltaY[j, k] = \
20                     DeltaY[j + 1, k - 1] - DeltaY[j, k - 1]
21     z = y[0]
22
23     for i in range(1, n):
24         p1 = 1
25         for k in range(i):
26             p1 = p1 * ((t - x[k]) / h)
27         z = z + \
28             (p1 / np.prod(range(1, i+1))) * DeltaY[0, i]
29
30     return z
31
32 xnew = np.linspace(np.min(x), np.max(x), 100)
33
34 ynew = [newton(x, y, i) for i in xnew]
35
36 plt.plot(x, y, 'o', xnew, ynew)
37
38 plt.grid(True)
39
40 plt.show()
```

Листинг 134: Интерполяционный многочлен Ньютона

Результат:

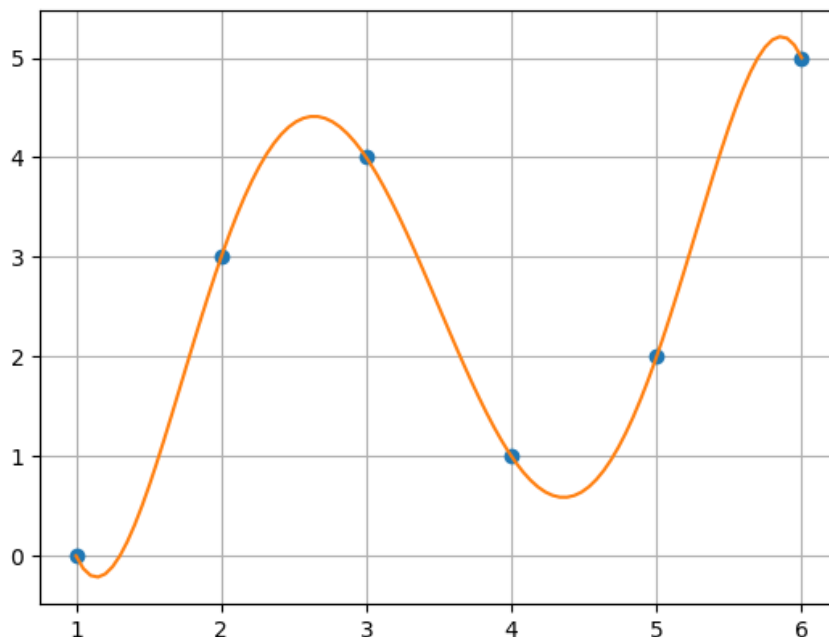


Рисунок 6.4. Интерполяционный многочлен Ньютона.

6.5. Интерполяция сплайнами

Пусть функция $f(x)$ задана таблицей

x	x_0	x_1	$\dots$	x_n
$f(x)$	y_0	y_1	$\dots$	y_n

где $h_i = x_i - x_{i-1}$ ($i = 1, 2, \dots, n$).

Сплайн — это функция, которая на каждом отрезке $[x_{i-1}, x_i]$ ($i = 1, 2, \dots, n$) является алгебраическим многочленом, а на всем заданном отрезке $[x_0, x_n]$ непрерывна вместе с несколькими производными. Максимальная степень многочленов по всем отрезкам называется степенью сплайна.

Построим сплайн третьей степени (кубический сплайн).

На каждом из отрезков $[x_{i-1}, x_i]$ построим алгебраический многочлен вида

$$S(x) = a_i + b_i(x - x_{i-1}) + c_i(x - x_{i-1})^2 + d_i(x - x_{i-1})^3, \quad (20)$$

$$(i = 1, 2, \dots, n),$$

где a_i, b_i, c_i, d_i — $4n$ неизвестных.

Требуя $S(x_i) = f(x_i)$ ($i = 0, 1, \dots, n$), $S''(x_0 + 0) = 0$, $S''(x_n - 0) = 0$ и

непрерывность $S'(x)$ и $S''(x)$, получим систему из $4n$ линейных алгебраических уравнений с $4n$ неизвестными

$$\begin{cases} y_{i-1} = a_i, & i = 1, 2, \dots, n, \\ b_i h_i + c_i h_i^2 + d_i h_i^3 = y_i - a_i, & i = 1, 2, \dots, n, \\ b_{i+1} - b_i - 2c_i h_i - 3d_i h_i^2 = 0, & i = 1, 2, \dots, n-1, \\ c_{i+1} - c_i - 3d_i h_i = 0, & i = 1, 2, \dots, n-1, \\ c_1 = 0, \\ c_n + 3d_n h_n = 0. \end{cases} \quad (21)$$

Решая эту систему, найдем a_i, b_i, c_i, d_i .

Пример. Построить сплайн третьей степени для функции, заданной таблицей

x	1	2	4	7
$f(x)$	0	3	1	5

Решение. Здесь $n = 3$, $h_1 = 1$, $h_2 = 2$, $h_3 = 3$.

Для построения сплайна третьей степени

$$S_1(x) = a_1 + b_1(x-1) + c_1(x-1)^2 + d_1(x-1)^3, \quad 1 \leq x \leq 2;$$

$$S_2(x) = a_2 + b_2(x-2) + c_2(x-2)^2 + d_2(x-2)^3, \quad 2 \leq x \leq 4;$$

$$S_3(x) = a_3 + b_3(x-4) + c_3(x-4)^2 + d_3(x-4)^3, \quad 4 \leq x \leq 7$$

необходимо найти a_i, b_i, c_i, d_i ($i = 1, 2, 3$). Для этого составим систему вида (21)

$$\begin{cases} a_1 = 0, \\ a_2 = 3, \\ a_3 = 1, \\ b_1 + c_1 + d_1 = 3 - a_1, \\ 2b_2 + 4c_2 + 8d_2 = 1 - a_2, \\ 3b_3 + 9c_3 + 27d_3 = 5 - a_3, \\ b_2 - b_1 - 2c_1 - 3d_1 = 0, \\ b_3 - b_2 - 4c_2 - 12d_2 = 0, \\ c_2 - c_1 - 3d_1 = 0, \\ c_3 - c_2 - 6d_2 = 0, \\ c_1 = 0, \\ c_3 + 9d_3 = 0. \end{cases}$$

Релизовать построение сплайна на python можно следующим образом:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.array([1, 2, 4, 7], dtype = float)
5 y = np.array([0, 3, 1, 5], dtype = float)
6 A = np.matrix('\
7 1, 1, 1, 0, 0, 0, 0, 0, 0;\
8 0, 0, 0, 2, 4, 8, 0, 0, 0;\
9 0, 0, 0, 0, 0, 0, 3, 9, 27;\
10 -1, -2, -3, 1, 0, 0, 0, 0, 0;\
11 0, 0, 0, -1, -4, -12, 1, 0, 0;\
12 0, -1, -3, 0, 1, 0, 0, 0, 0;\
13 0, 0, 0, 0, -1, -6, 0, 1, 0;\
14 0, 1, 0, 0, 0, 0, 0, 0, 0;\
15 0, 0, 0, 0, 0, 0, 0, 1, 9\
16 ')
17 B = np.matrix('3; -2; 4; 0; 0; 0; 0; 0; 0')
18 X = np.linalg.solve(A, B)
19
20 def spline1(x, y, X, t):
21     s = y[0]+X[0]*(t-x[0])+X[1]*(t-x[0])**2+X[2]*(t-x[0])**3
22     return float(s)
23
24 def spline2(x, y, X, t):
25     s = y[1]+X[3]*(t-x[1])+X[4]*(t-x[1])**2+X[5]*(t-x[1])**3
26     return float(s)
27
28 def spline3(x, y, X, t):
29     s = y[2]+X[6]*(t-x[2])+X[7]*(t-x[2])**2+X[8]*(t-x[2])**3
30     return float(s)
31
32 x1 = np.linspace(1, 2, 100)
33 x2 = np.linspace(2, 4, 100)
34 x3 = np.linspace(4, 7, 100)
35 y1 = [spline1(x, y, X, i) for i in x1]
36 y2 = [spline2(x, y, X, i) for i in x2]
37 y3 = [spline3(x, y, X, i) for i in x3]
38 plt.plot(x, y, 'o', x1, y1, 'b-', x2, y2, 'g-', x3, y3, 'r-')
39 plt.grid(True)
40 plt.show()
```

Листинг 135: Интерполяция сплайнами

Результат:

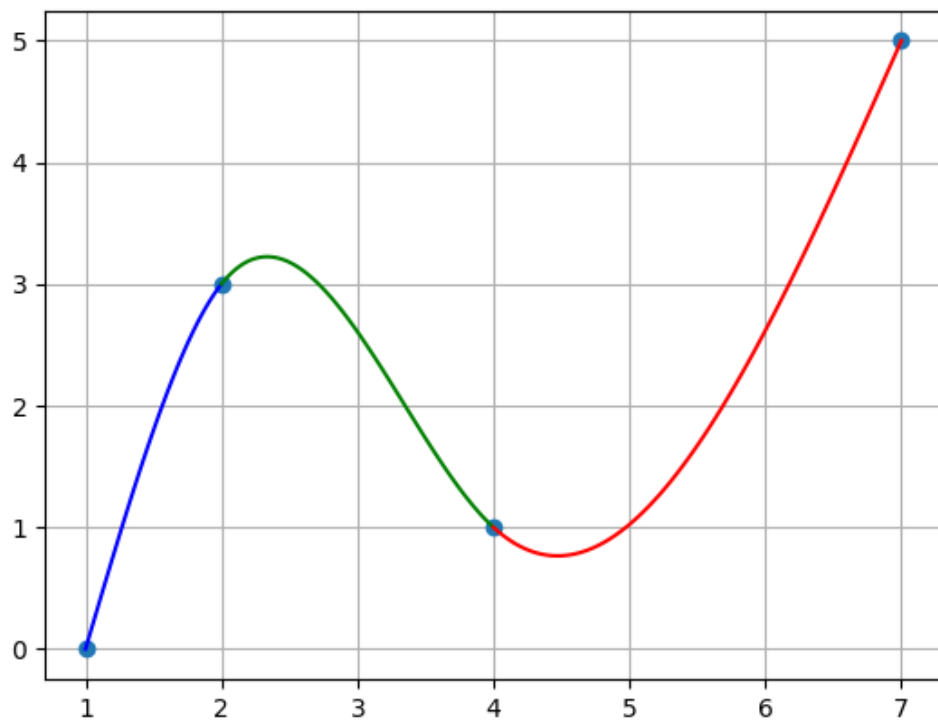


Рисунок 6.5. Интерполяция сплайнами.

6.6. Метод наименьших квадратов

Пусть функция $f(x)$ задана таблицей

x	x_0	x_1	$\dots$	x_n
$f(x)$	y_0	y_1	$\dots$	y_n

При нахождении аналитического выражения приближающей функции методом наименьших квадратов не требуется совпадения значений исходной и приближающей функций в точках x_i . Однако необходимо учитывать класс функций, к которому относится искомая функция. На практике это можно сделать, отметив на графике исходные точки и учитывая дополнительные

условия задачи. Часто используют следующие функции:

- 1). $y = ax + b$ (линейная функция);
- 2). $y = ax^2 + bx + c$ (квадратичная функция);
- 3). $y = ax^m$ (степенная функция);
- 4). $y = ae^{mx}$ (показательная функция);
- 5). $y = \frac{ax+b}{cx+d}$ (дробно-линейная функция);
- 6). $y = a \cdot \ln x + b$ (логарифмическая функция);
- 7). $y = \frac{a}{x} + b$ (гипербола).

Для нахождения параметров конкретной приближающей формулы в методе наименьших квадратов требуется, чтобы сумма квадратов отклонений значений приближающей функции от значений исходной функции была минимальной.

Будем рассматривать нахождение параметров $a_1, a_2, \dots, a_n$ приближающей функции $F(x, a_1, a_2, \dots, a_n)$.

Пусть

$$\sum_{i=0}^n (y_i - F(x_i, a_1, a_2, \dots, a_n))^2 = \Phi(a_1, a_2, \dots, a_n).$$

При применении метода наименьших квадратов необходимо найти точки минимума функции $\Phi(a_1, a_2, \dots, a_n)$, т.е. точки, в которых

$$\frac{\partial \Phi}{\partial a_1} = 0, \frac{\partial \Phi}{\partial a_2} = 0, \dots, \frac{\partial \Phi}{\partial a_n} = 0.$$

Таким образом, получим систему n линейных алгебраических уравнений с n неизвестными

$$\begin{cases} \sum_{i=0}^n (y_i - F(x_i, a_1, a_2, \dots, a_n)) \cdot \frac{\partial F(x_i, a_1, a_2, \dots, a_n)}{\partial a_1} = 0, \\ \sum_{i=0}^n (y_i - F(x_i, a_1, a_2, \dots, a_n)) \cdot \frac{\partial F(x_i, a_1, a_2, \dots, a_n)}{\partial a_2} = 0, \\ \dots \\ \sum_{i=0}^n (y_i - F(x_i, a_1, a_2, \dots, a_n)) \cdot \frac{\partial F(x_i, a_1, a_2, \dots, a_n)}{\partial a_n} = 0. \end{cases} \quad (22)$$

Решая эту систему, найдем значения параметров $a_1, a_2, \dots, a_n$.

Пример. Используя метод наименьших квадратов, найти приближающую функцию в виде линейной функции

$$F(x, a, b) = ax + b.$$

Составим систему вида (22)

$$\begin{cases} \sum_{i=0}^n (y_i - ax_i - b) \cdot x_i = 0, \\ \sum_{i=0}^n (y_i - ax_i - b) = 0. \end{cases}$$

Преобразуем эту систему к равносильной

$$\begin{cases} \left(\frac{1}{n} \sum_{i=0}^n (x_i)^2 \right) \cdot a + \left(\frac{1}{n} \sum_{i=0}^n x_i \right) \cdot b = \frac{1}{n} \sum_{i=0}^n y_i x_i, \\ \left(\frac{1}{n} \sum_{i=0}^n x_i \right) \cdot a + b = \frac{1}{n} \sum_{i=0}^n y_i, \quad i = 1, \dots, n. \end{cases}$$

Решая эту систему, найдем значения параметров a и b .

6.7. Задания для самостоятельной работы

1. Объявить функцию для вычисления значений приближающей функции с помощью интерполяционного многочлена Лагранжа. На основе заданной таблицы значений функции построить график приближающей функции и отметить на нем узловые точки.

2. Объявить функцию для вычисления значений приближающей функции с помощью интерполяционного многочлена Ньютона. На основе заданной таблицы значений функции построить график приближающей функции и отметить на нем узловые точки.

3. По заданной таблице значений функции провести интерполяцию кубическими сплайнами. Построить график и отметить на нем узловые точки.

4. По заданной таблице значений функции построить эмпирическую формулу методом наименьших квадратов. (Предварительно построить точечный график, с помощью которого выбрать наиболее предпочтительный вид приближающей функции). Построить график полученной функции.

Варианты заданий

1-4.

1.

x	-1	1	3	5
y	-3	5	2	7

2.

x	2	3	4	5
y	4	1	7	2

3.

x	0	2	4	6
y	-1	-4	2	4

4.

x	7	9	11	13
y	2	-2	3	5

5.

x	-3	-1	1	3
y	7	-1	4	6

6.

x	1	2	3	4
y	-3	-7	2	3

7.

x	-2	-1	0	1
y	4	9	1	7

8.

x	2	4	6	8
y	9	-3	6	3

9.

x	-4	-2	0	2
y	2	8	5	1

10.

x	-1	1	3	5
y	4	-7	1	8

11.

x	2	4	6	8
y	-1	-6	3	7

12.

x	-9	-7	-5	-3
y	3	-3	4	9

13.

x	0	1	2	3
y	7	-1	8	2

14.

x	-8	-6	-4	-2
y	9	-2	4	9

15.

x	-7	-5	-3	-1
y	4	-4	5	-1

5.

1.

x	5.84	3.82	6.19	9.22	7.87	6.29	4.43	8.91	5.34	2.21
y	12.56	8.49	16.53	20.99	21.54	13.5	13.79	18.96	15.58	5.35

2.

x	0.12	2.48	-1.45	2.19	1.81	-2.81	0.84	-2.08	-1.28	3.44
y	1.14	5.29	9.36	5.64	3.87	24.54	-0.99	16.12	7.68	13.61

3.

x	2.7	3.1	5.4	5.1	3.59	5.7	4.67	6.75	6.5	4.3
y	61.5	89.4	475.2	398.4	139.1	555.9	307.2	924.8	825.8	240.9

4.

x	5.7	3.1	5.4	4.3	3.59	5.3	4.1	4.75	3.8	6.5
y	0.18	0.33	0.187	0.233	0.283	0.192	0.25	0.213	0.268	0.156

5.

x	9.7	11.1	10.4	6.3	4.59	8.3	12.1	15.75	20.8	7.5
y	4.773	4.838	5.055	4.006	3.165	4.73	5.433	5.96	6.378	4.395

6.

x	2.7	-3.1	-2.4	3.3	2	3.3	-3.1	2.5	-1	2
y	-3.32	2.99	1.9	-4.	-2.37	-4.26	2.64	-3.02	0.2	-2.73

7.

x	-1.5	-2.1	2.4	0.3	-1.4	-0.5	1.1	-1.5	2.5	2.8
y	-4.49	-12.84	2.33	-2.34	-6.98	-0.32	-2.65	-8.1	6.75	10.83

8.

x	-1.5	-2.1	0.7	0.3	-0.9	-0.5	1.1	-0.5	0.6	0.9
y	2.94	1.14	16.13	6.92	1.45	2.87	29.21	1.99	11.6	18.43

9.

x	-1.5	-2.1	0.7	0.3	-0.9	2.5	1.5	-0.5	2.6	3.9
y	12.36	16.46	4.32	4.77	9.62	4.83	3.89	7.43	4.71	8.94

10.

x	-1.62	-1.91	0.81	0.12	-0.85	2.35	1.67	-0.49	2.58	3.67
y	0.143	-0.343	8.6	6.06	4.096	12.398	11.43	4.138	12.92	16.3

11.

x	1.52	1.78	0.67	0.15	0.89	2.31	1.68	0.44	2.56	3.77
y	8.15	9.56	4.31	-2.8	4.63	9.81	8.16	1.17	11.36	12.83

12.

x	-1.52	-1.78	-0.67	-0.15	0.89	-2.31	1.68	1.44	2.56	2.77
y	4.65	6.73	0.92	1.61	-0.13	14.11	-3.25	-2.26	-16.32	-20.79

13.

x	-1.49	-1.75	-0.64	-0.14	0.87	-2.3	1.67	1.43	2.55	2.76
y	0.82	0.539	0.737	1.089	1.557	0.66	2.466	2.289	3.929	4.213

14.

x	-0.8	-1.7	-0.6	-0.12	0.8	-1.2	0.9	1.3	0.5	1.7
y	6.803	17.439	6.156	2.604	0.857	11.4	1.35	1.186	1.323	3.747

15.

x	-0.8	-1.5	-0.6	-0.12	0.8	-1.2	0.9	1.3	0.5	1.7
y	-3.48	-19.23	0.23	2.29	2.13	-9.3	2.53	2.34	2.68	7.66

Вопросы к защите лабораторной работы

1. Интерполяционный многочлен Лагранжа.
2. Интерполяционные формулы Ньютона.
3. Приближение кубическими сплайнами.
4. Метод наименьших квадратов.

7. Численное интегрирование

Самым распространенным способом численного интегрирования является замена интеграла конечной суммой

$$\int_a^b f(x)dx = \sum_{i=0}^n A_i \cdot f(x_i) + R(f).$$

Формулы такого типа называются квадратурными,

$$x_i = a + i \cdot h \quad (h = \frac{b-a}{n}, i = 0, 1, \dots, n)$$

узлами квадратурной формулы, A_i — коэффициентами квадратурной формулы, $R(f)$ — погрешностью квадратурной формулы.

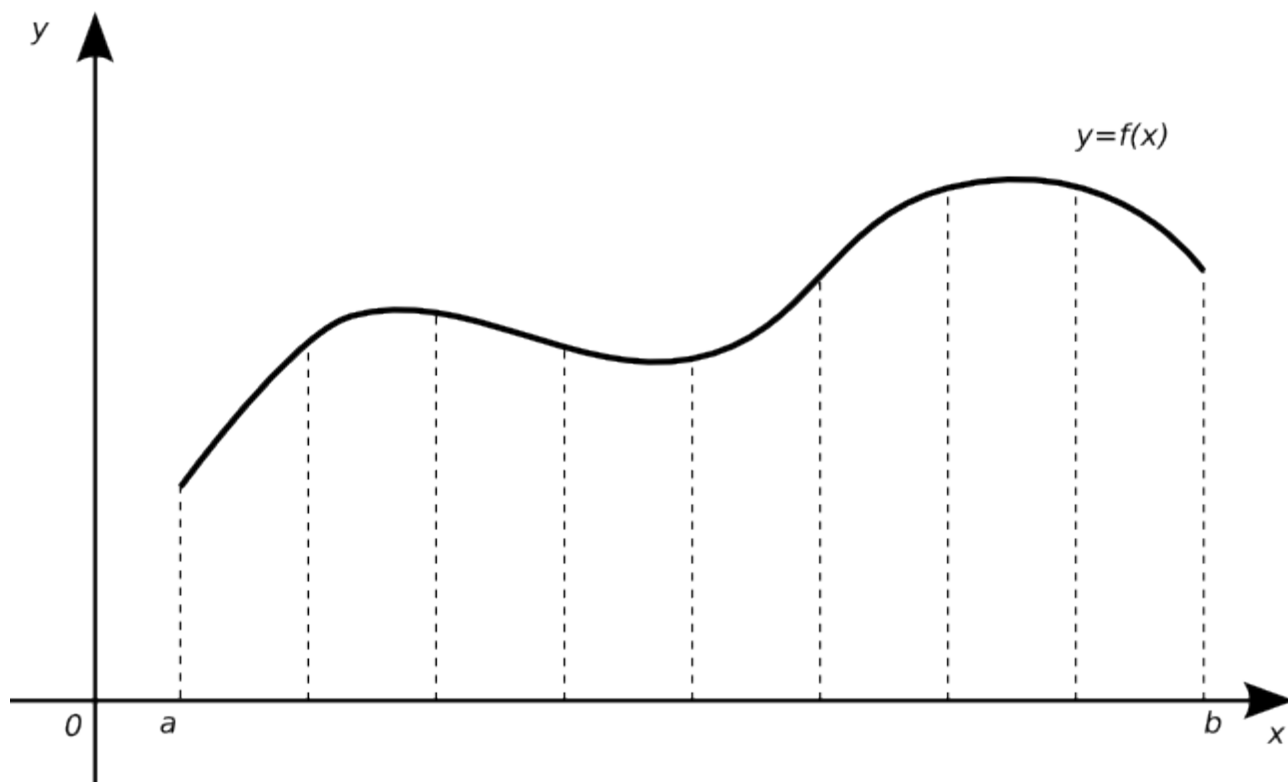


Рисунок 7.1. Численное интегрирование.

7.1. Формула левых прямоугольников

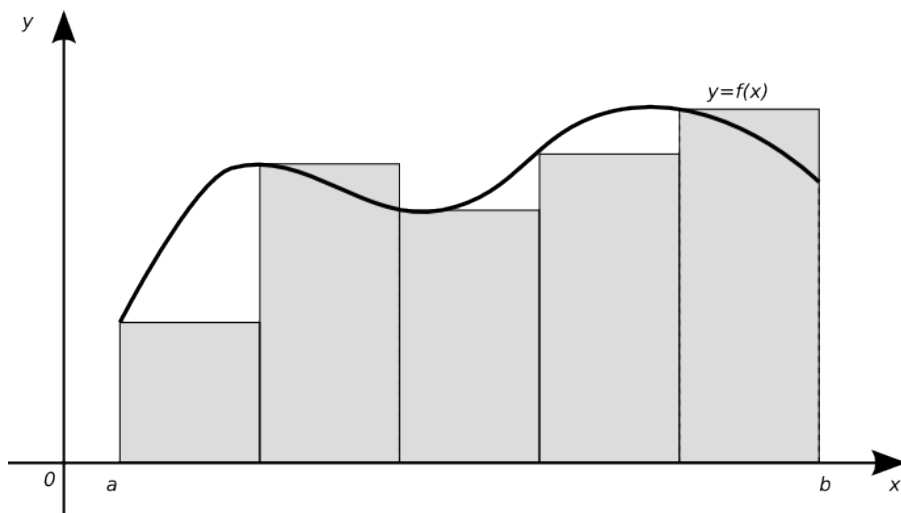


Рисунок 7.2. Формула левых прямоугольников.

$$\int_a^b f(x) dx \approx h \cdot \sum_{i=0}^{n-1} f(x_i),$$

где $h = \frac{b-a}{n}$, $x_i = a + i \cdot h$ ($i = 0, 1, \dots, n-1$).

7.2. Формула правых прямоугольников

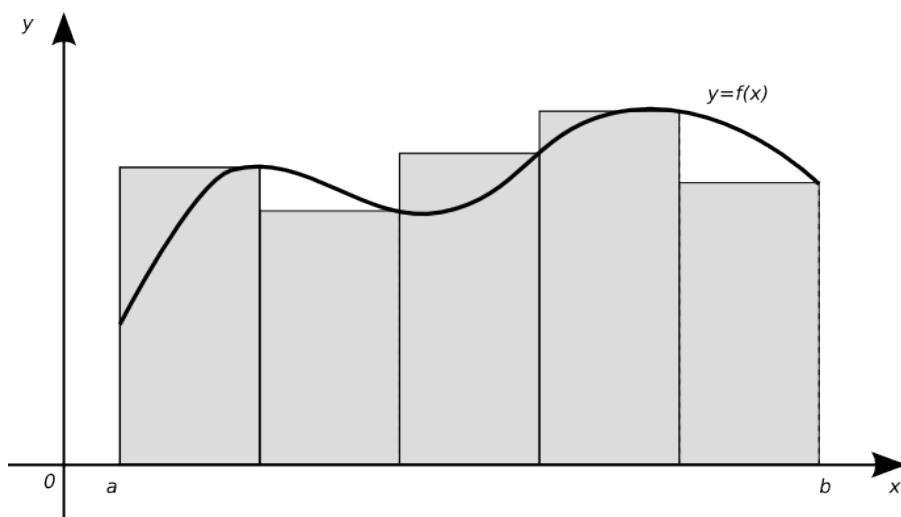


Рисунок 7.3. Формула правых прямоугольников.

$$\int_a^b f(x) dx \approx h \cdot \sum_{i=1}^n f(x_i),$$

где $h = \frac{b-a}{n}$, $x_i = a + i \cdot h$ ($i = 1, 2, \dots, n$).

7.3. Формула средних прямоугольников

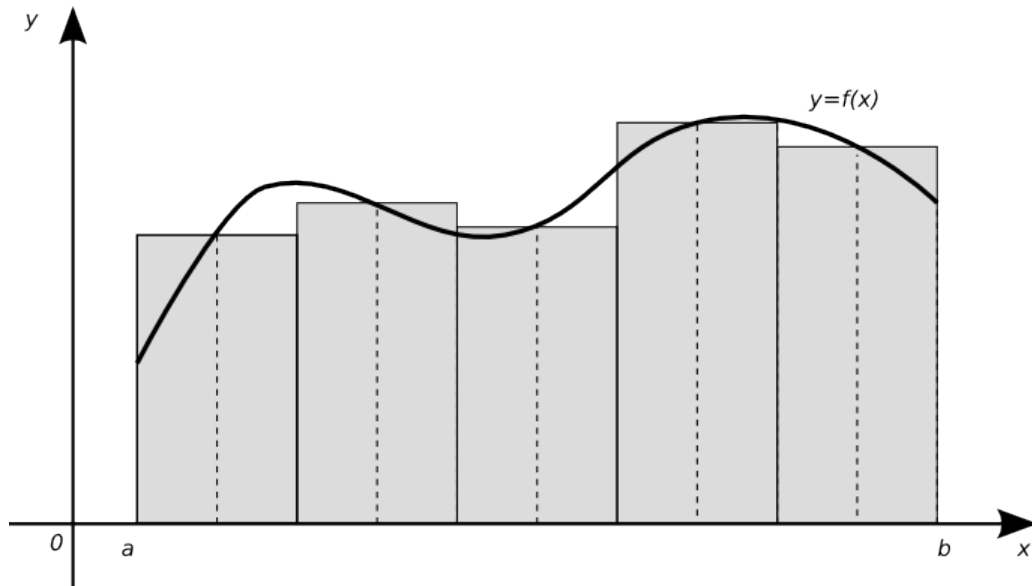


Рисунок 7.4. Формула средних прямоугольников.

$$\int_a^b f(x) dx \approx h \cdot \sum_{i=0}^{n-1} f\left(x_i + \frac{h}{2}\right),$$

где $h = \frac{b-a}{n}$, $x_i = a + i \cdot h$ ($i = 0, 1, \dots, n-1$).

7.4. Формула трапеций

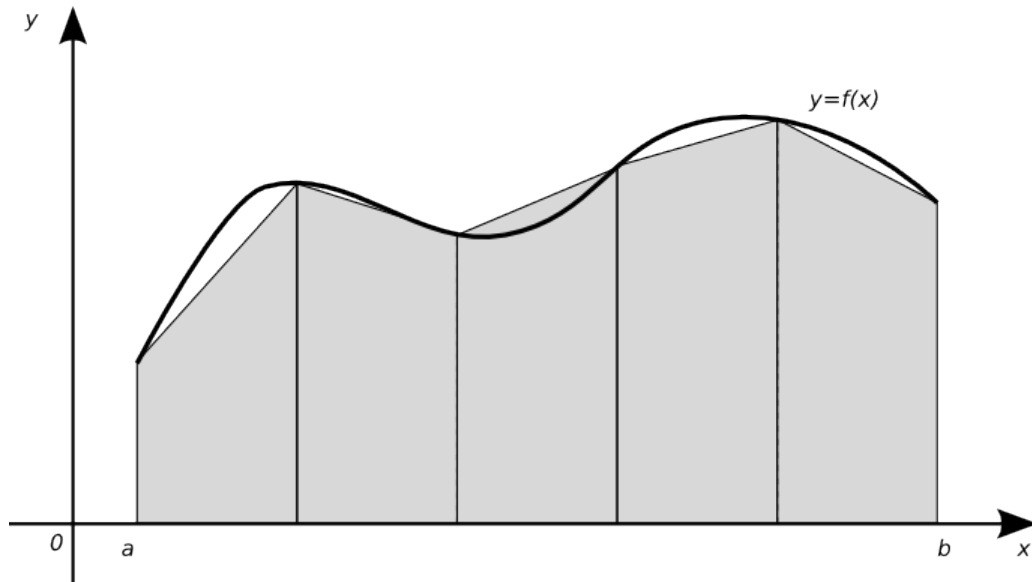


Рисунок 7.5. Формула трапеций.

$$\int_a^b f(x) dx \approx h \cdot \left(\frac{f(x_0) + f(x_n)}{2} + \sum_{i=1}^{n-1} f(x_i) \right),$$

где $h = \frac{b-a}{n}$, $x_i = a + i \cdot h$ ($i = 0, 1, \dots, n$).

7.5. Формула Симпсона

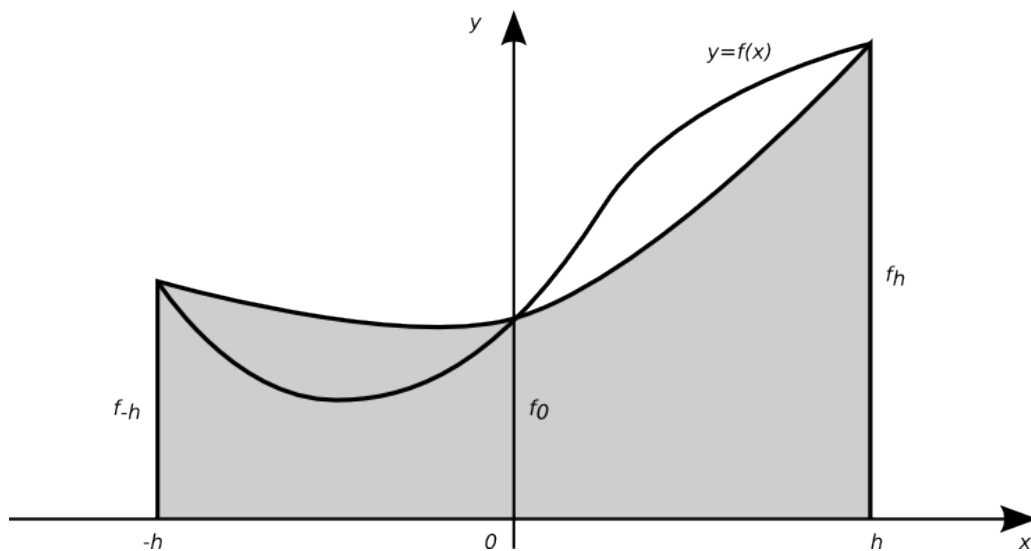


Рисунок 7.6. Формула Симпсона.

$$\int_a^b f(x) dx \approx \frac{h}{3} \cdot \left(f(x_0) + f(x_{2n}) + 4 \sum_{i=1}^n f(x_{2i-1}) + 2 \sum_{i=1}^{n-1} f(x_{2i}) \right),$$

где

$$h = \frac{b-a}{2n}, \quad x_i = a + i \cdot h \quad (i = 0, 1, \dots, 2n).$$

7.6. Метод Монте-Карло

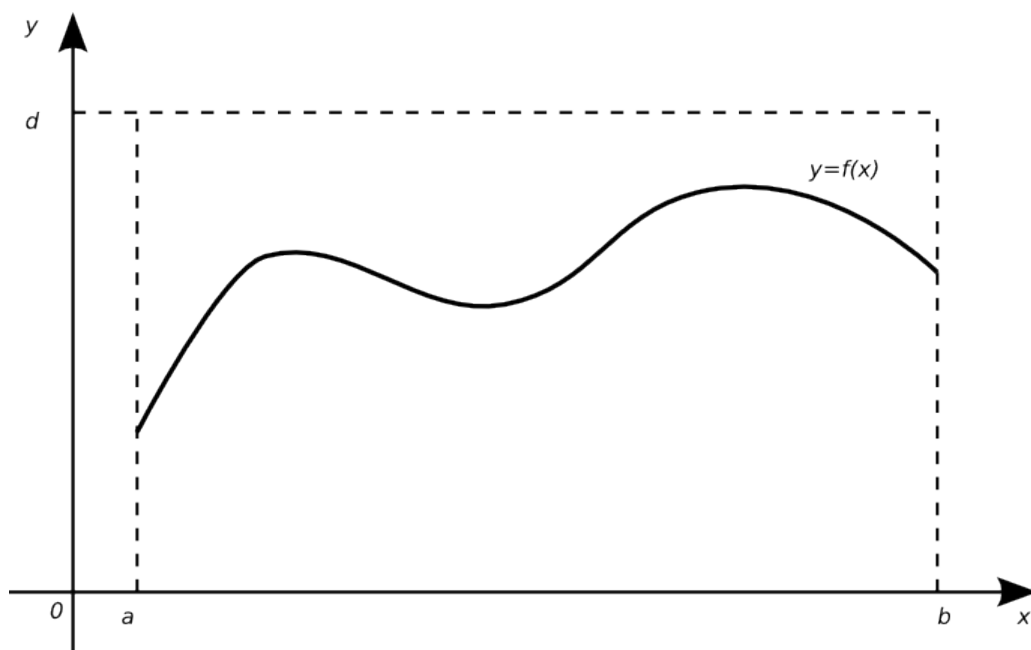


Рисунок 7.7. Метод Монте-Карло.

$$\int_a^b f(x) dx \approx D \frac{n_f}{N},$$

где $f(x) \geq 0$, (x_i, y_i) — N пар случайных чисел, равномерно распределенных в прямоугольнике $[a, b] \times [0, d]$, содержащем криволинейную трапецию, ограниченную линиями $y = f(x)$, $y = 0$, $x = a$, $x = b$, n_f — количество точек, удовлетворяющих условию $y_i \leq f(x_i)$, D — площадь прямоугольника $[a, b] \times [0, d]$.

7.7. Метод Монте-Карло для кратных интегралов

$$\int_a^b \int_c^d f(x, y) dx \approx D \frac{n_f}{N},$$

где $f(x, y) \geq 0$, (x_i, y_i, z_i) — N троек случайных чисел, равномерно распределенных в параллелепипеде $[a, b] \times [c, d] \times [0, p]$, содержащем криволинейный цилиндр, ограниченный поверхностями $z = f(x, y)$, $z = 0$, $x = a$, $x = b$, $y = c$, $y = d$, n_f — количество точек, удовлетворяющих условию $z_i \leq f(x_i, y_i)$, D — объем параллелепипеда $[a, b] \times [c, d] \times [0, p]$.

7.8. Задания для самостоятельной работы

1. Записать расчетные формулы и вычислить интегралы с применением:
 - а) формул левых, правых и средних прямоугольников;
 - б) формулы трапеций;
 - в) формулы Симпсона;
 - г) метода Монте-Карло.
2. Вычислить двойной интеграл методом Монте-Карло.

Варианты заданий

Задание №1

1. $\int_0^1 0,37e^{\sin x} dx.$
2. $\int_1^2 (0,5x + x \lg x) dx.$
3. $\int_1^2 (x + 1,9) \sin\left(\frac{x}{3}\right) dx.$

4. $\int_2^3 \frac{1}{x} \ln(x+2) dx.$
5. $\int_0^1 \frac{3\cos x}{2x+1,7} dx.$
6. $\int_2^2 (2x+0,6) \cos\left(\frac{x}{2}\right) dx.$
7. $\int_{1,2}^{2,2} (2,6x^2 \ln x) dx.$
8. $\int_{0,5}^{1,5} (x^2+1) \sin(x-0,5) dx.$
9. $\int_2^3 x^2 \cos\left(\frac{x}{4}\right) dx.$
10. $\int_3^4 \frac{\sin(0,2x-3)}{x^2+1} dx.$
11. $\int_1^2 (3x + \ln x) dx.$
12. $\int_{-1}^0 4xe^{\sin(x^2)} dx.$
13. $\int_{-0,5}^{0,5} (3x^2 + \operatorname{tg} x) dx.$
14. $\int_1^2 \frac{3x^2 + \sin x}{x^2} dx.$
15. $\int_1^2 3xe^{\cos x} dx.$

Задание №2

1. $\int_0^1 \int_0^1 (2y - 0,35e^{\sin x}) dx dy.$
2. $\int_2^2 \int_2^2 (0,7y + x \lg x) dx dy.$
3. $\int_1^2 \int_1^2 (xy + 1,5) \sin\left(\frac{x+2y}{3}\right) dx dy.$
4. $\int_2^3 \int_2^3 \frac{2}{x} \ln(x+y) dx dy.$
5. $\int_1^4 \int_1^4 \frac{5\cos(x+y)}{2x+1,5} dx dy.$
6. $\int_1^2 \int_1^2 (4x + 0,3y) \cos\left(\frac{x-y}{2}\right) dx dy.$

7. $\int_1^3 \int_1^3 (5x^2 y \ln(x+y)) \, dx dy.$
8. $\int_0^5 \int_0^5 (x^2 + y) \sin(2x - 0, 7y) \, dx dy.$
9. $\int_2^3 \int_2^3 x^2 y^2 \cos\left(\frac{x+2y}{4}\right) \, dx dy.$
10. $\int_3^4 \int_3^4 \frac{\sin(0, 3x-2y)}{x^2+y^2} \, dx dy.$
11. $\int_1^3 \int_1^3 (2x + \ln(x+y)) \, dx dy.$
12. $\int_{-1}^1 \int_{-1}^1 5xy e^{(yx^2)} \, dx dy.$
13. $\int_2^3 \int_2^3 (3(x-y)^2 + tg(2x+y)) \, dx dy.$
14. $\int_2^3 \int_2^3 \frac{5yx^2 + \sin(xy)}{2x^2} \, dx dy.$
15. $\int_1^3 \int_1^3 5xy e^{\cos(x+y)} \, dx dy.$

Вопросы к защите лабораторной работы

1. Квадратурные формулы левых, правых и средних прямоугольников.

Графическая иллюстрация.

2. Квадратурная формула трапеций. Графическая иллюстрация.
3. Квадратурная формула Симпсона. Графическая иллюстрация.
4. Метод Монте-Карло.
5. Метод Монте-Карло для кратных интегралов.

8. Численное решение дифференциальных уравнений

Будем рассматривать дифференциальное уравнение вида

$$y' = f(x, y), \quad (1)$$

где $f(x, y)$ – заданная непрерывная функция в области D .

Задача нахождения решения этого уравнения, удовлетворяющего начальному условию

$$y(x_0) = y_0, \quad (2)$$

называется задачей Коши.

Будем говорить, что функция $f(x, y)$ в области D удовлетворяет условию Липшица по переменной y , если существует такая постоянная N , что для любых точек (x, y_1) и (x, y_2) , принадлежащих области D ,

$$|f(x, y_1) - f(x, y_2)| \leq N|y_1 - y_2|.$$

В силу теоремы Лагранжа условие Липшица выполняется в области D , если область D вместе с любыми точками (x, y_1) и (x, y_2) содержит соединяющий их отрезок и частная производная функции f по y ограничена на D . В этом случае можно принять $N = \sup |f'_y(x, y)|$.

Ответ на вопрос о существовании и единственности решения задачи Коши для дифференциального уравнения (1) дает следующая

Теорема. Пусть функция $f(x, y)$ определена и непрерывна в области D , содержащей точку (x_0, y_0) , и удовлетворяет в этой области условию Липшица по y . Тогда на некотором отрезке $[x_0 - h, x_0 + h]$ существует единственное решение $y = \varphi(x)$ задачи Коши $y' = f(x, y), y(x_0) = y_0$.

8.1. Метод Эйлера

Пусть требуется найти решение задачи Коши (1)-(2) на отрезке $[a, b]$.

Разобьем отрезок $[a, b]$ на n равных частей точками $x_i = a + ih$, где $i = 0, 1, \dots, n$, $h = \frac{b-a}{n}$. Заменяя в уравнении (1) производную разностным отношением, получим

$$\frac{y_1 - y_0}{h} = f(x_0, y_0).$$

Перепишем последнее уравнение в виде

$$y_1 = y_0 + hf(x_0, y_0).$$

Повторяя этот процесс, получим приближенное решение задачи (1)-(2).

Таким образом, итерационная формула метода Эйлера имеет вид

$$x_{i+1} = a + ih, \quad y_{i+1} = y_i + hf(x_i, y_i), \quad i = 0, 1, \dots, n.$$

Достоинством метода Эйлера является его простота. Недостатком метода Эйлера является его низкая точность.

Пример. Решить методом Эйлера дифференциальное уравнение $y' = 3\sin 2y + x$ с начальным условием $y(0) = 2$ на отрезке $[0, 1]$ с шагом $h = 0, 1$.

Решение.

```
1  from math import sin
2
3  def f(x, y):
4      return 3 * sin(2 * y) + x
5
6  a = 0; b = 1; h = 0.1
7  n = int((b - a) / float(h))
8  x = []; y = []
9  x.append(a); y.append(2)
10 for i in range(n + 1):
11     x.append(a + i * h)
12     y.append(y[i] + h * f(x[i], y[i]))
13
14 print("x=", x)
15 print("y=", y)
```

Листинг 136: Метод Эйлера

Результат:

```
x= [0, 0.0, 0.1, 0.2, 0.30000000000000004, 0.4, 0.5,
0.6000000000000001, 0.7000000000000001, 0.8, 0.9, 1.0]
y= [2, 1.7729592514076216, 1.6549395342211919,
1.6146915687959411, 1.6083882413659165,
1.6158543358547546, 1.6288561067078735,
1.6440984723656926, 1.6602745627721154,
1.6768737204773871, 1.6937036616007668,
1.7106996859182662]
```

8.2. Метод Эйлера-Коши

Метод Эйлера-Коши позволяет увеличить точность метода Эйлера. Расчетные формулы этого метода имеют вид

$$\begin{aligned}x_{i+1} &= a + ih, \\y_{i+1}^* &= y_i + h \cdot f(x_i, y_i), \\y_{i+1} &= y_i + h \cdot \frac{f(x_i, y_i) + f(x_{i+1}, y_{i+1}^*)}{2}.\end{aligned}$$

8.3. Метод Рунге-Кутты

Методы Эйлера и Эйлера-Коши являются частными случаями метода Рунге-Кутты. На практике наиболее распространенным является метод Рунге-Кутты четвертого порядка. Расчетные формулы этого метода имеют вид

$$\begin{aligned}x_{i+1} &= a + ih, \\y1_{i+1} &= h \cdot f(x_i, y_i), \\y2_{i+1} &= h \cdot f\left(x_i + \frac{h}{2}, y_i + \frac{y1_{i+1}}{2}\right), \\y3_{i+1} &= h \cdot f\left(x_i + \frac{h}{2}, y_i + \frac{y2_{i+1}}{2}\right), \\y4_{i+1} &= h \cdot f(x_i + h, y_i + y3_{i+1}), \\y_{i+1} &= \frac{1}{6}(y1_{i+1} + 2 \cdot y2_{i+1} + 2 \cdot y3_{i+1} + y4_{i+1}).\end{aligned}$$

8.4. Решение краевой задачи для линейного дифференциального уравнения второго порядка методом конечных разностей

Краевая задача для линейного дифференциального уравнения

$$y'' + p(x)y' + q(x)y = f(x), \quad (3)$$

где $p(x)$, $q(x)$ и $f(x)$ — некоторые непрерывные на $[a, b]$ функции, состоит в нахождении его решения $y = y(x)$, удовлетворяющего двухточечным линей-

ным краевым условиям

$$\begin{cases} \alpha_0 y(a) + \alpha_1 y'(a) = A, \\ \beta_0 y(b) + \beta_1 y'(b) = B, \end{cases} \quad (4)$$

где $\alpha_0, \alpha_1, \beta_0, \beta_1, A, B$ – постоянные и $|\alpha_0| + |\alpha_1| \neq 0$, $|\beta_0| + |\beta_1| \neq 0$.

При решении этой задачи методом конечных разностей отрезок $[a, b]$ разбивают на равные части с шагом h , где $h = \frac{b-a}{n}$.

Точки разбиения имеют абсциссы

$$\begin{aligned} x_i &= x_0 + i \cdot h \quad (i = 0, 1, \dots, n), \\ x_0 &= a, \quad x_n = b. \end{aligned}$$

Значения в точках деления x_i искомой функции и ее производных $y' = y'(x)$, $y'' = y''(x)$ обозначим соответственно через $y_i = y(x_i)$, $y'_i = y'(x_i)$, $y''_i = y''(x_i)$.

Заменяя производные правыми односторонними конечно-разностными отношениями для внутренних точек x_i отрезка $[a, b]$, приближенно будем иметь

$$\begin{aligned} y'_i &= \frac{y_{i+1} - y_i}{h}, \\ y''_i &= \frac{y_{i+2} - 2y_{i+1} + y_i}{h^2}. \end{aligned} \quad (5)$$

Для концевых точек $x_0 = a$ и $x_n = b$ полагаем

$$y'_0 = \frac{y_1 - y_0}{h} \text{ и } y'_n = \frac{y_n - y_{n-1}}{h}. \quad (6)$$

Используя формулы (5), дифференциальное уравнение (3) при $x = x_i$ ($i = 0, 1, \dots, n-2$) приближенно можно заменить системой линейных уравнений

$$\begin{aligned} \frac{y_{i+2} - 2y_{i+1} + y_i}{h^2} + p(x_i) \frac{y_{i+1} - y_i}{h} + q(x_i) y_i &= f(x_i) \\ (i &= 0, 1, \dots, n-2). \end{aligned}$$

Кроме того, в силу формул (6) краевые условия (4) дополнительно дают еще два уравнения

$$\begin{aligned} \alpha_0 y_0 + \alpha_1 \frac{y_1 - y_0}{h} &= A, \\ \beta_0 y_n + \beta_1 \frac{y_n - y_{n-1}}{h} &= B. \end{aligned}$$

Таким образом, получена система $n+1$ линейных уравнений с $n+1$ неизвестными $y_0, y_1, \dots, y_n$, представляющими собой значения искомой функции

$$y = y(x),$$

$$\begin{cases} \frac{y_{i+2}-2y_{i+1}+y_i}{h^2} + p(x_i) \frac{y_{i+1}-y_i}{h} + q(x_i) y_i = f(x_i), \\ \alpha_0 y_0 + \alpha_1 \frac{y_1-y_0}{h} = A, \\ \beta_0 y_n + \beta_1 \frac{y_n-y_{n-1}}{h} = B. \end{cases}$$

Обозначим $p(x_i) = p_i$, $q(x_i) = q_i$, $f(x_i) = f_i$.

После алгебраических преобразований система примет вид

$$\begin{cases} (h^2 q_i - h p_i + 1) y_i + (h p_i - 2) y_{i+1} + y_{i+2} = h^2 f_i, \\ (\alpha_0 h - \alpha_1) y_0 + \alpha_1 y_1 = h A, \\ -\beta_1 y_n + (\beta_0 h + \beta_1) y_n = h B. \end{cases} \quad (7)$$

Решив систему, получим таблицу значений искомой функции $y(x)$.

Пример. Найти решение уравнения $y'' + e^x y' + 2xy = x^3$ на $[0, 1, 1, 3]$ ($n = 4$) с начальными условиями

$$2y(0, 1) - 2, 5y'(0, 1) = 0,$$

$$3y(1, 3) - 3, 4y'(1, 3) = 5.$$

Здесь

$$p(x) = e^x, \quad q(x) = 2x, \quad f(x) = x^3,$$

$$\alpha_0 = 2, \quad \alpha_1 = -2, 5, \quad A = 0,$$

$$\beta_0 = 3, \quad \beta_1 = -3, 4, \quad B = 5.$$

Построим систему (7) линейных алгебраических уравнений, где неизвестными являются $y_0, \dots, y_4$.

Коэффициенты системы рассчитываются по формулам

$$a_{i,i} = h^2 q_i - h p_i + 1,$$

$$a_{i,i+1} = h p_i - 2,$$

$$a_{i,i+2} = 1,$$

$$i = 0, 1, \dots, n-2.$$

Для последних двух уравнений (n -го и $(n+1)$ -го) используются начальные условия

$$a_{n-1,0} = \alpha_0 h - \alpha_1,$$

$$a_{n-1,1} = \alpha_1,$$

$$a_{n,n-1} = -\beta_1,$$

$$a_{n,n} = \beta_0 h + \beta_1.$$

Матрица B свободных членов формируется по формулам

$$\begin{aligned}b_i &= h^2 f_i, \\b_{n-1} &= hA, \\b_n &= hB, \\i &= 0, \dots, n-2. \\h &= \frac{1,3-0,1}{4} = 0,3.\end{aligned}$$

Остальные коэффициенты системы равны нулю.

Решая полученную систему, найдем приближенное решение дифференциального уравнения.

Реализовать решение на python можно следующим образом:

```
1  from numpy import exp, linalg, zeros
2
3  # Define differential equation
4  def p(x):
5      return exp(x)
6
7  def q(x):
8      return 2 * x
9
10 def f(x):
11     return pow(x, 3)
12
13 alpha0 = 2; alpha1 = -2.5; Ac = 0
14 beta0 = 3; beta1 = -3.4; Bc = 5
15 a0 = 0.1; b0 = 1.3
16
```

Листинг 137: Метод конечных разностей - 1

```

1  # Grid
2  n = 4
3  A = zeros((n + 1, n + 1), dtype = float)
4  B = zeros(n + 1, dtype = float)
5  h = (b0 - a0) / float(n)
6  x = [a0 + i * h for i in range(n + 1)]
7  for i in range(n - 1):
8      A[i][i] = pow(h, 2) * q(x[i]) - h * p(x[i]) + 1
9      A[i][i + 1] = h * p(x[i]) - 2
10     A[i][i + 2] = 1
11     B[i] = pow(h, 2) * f(x[i])
12 A[n - 1][0] = alpha0 * h - alpha1
13 A[n - 1][1] = alpha1
14 A[n][n - 1] = -beta1
15 A[n][n] = beta0 * h + beta1
16 B[n - 1] = h * Ac
17 B[n] = h * Bc
18 y = linalg.solve(A, B)
19 print("y =", y)

```

Листинг 138: Метод конечных разностей - 2

Результат:

$y = [0.9387045 \quad 1.16399358 \quad 1.2977811 \quad 1.29364482 \quad 1.15935696]$

8.5. Задания для самостоятельной работы

1. Решить задачу Коши для дифференциального уравнения $y' = f(x, y)$ на отрезке $[a, b]$ при заданном начальном условии $y(a) = c$ и шаге h :

- а) методом Эйлера;
- б) методом Эйлера-Коши.
- в) методом Рунге-Кутты четвертого порядка.

2. Используя метод конечных разностей, найти решение краевой задачи для обыкновенного дифференциального уравнения второго порядка с шагом $h = 0, 1$.

Варианты

Задание №1

№	$f(x, y)$	a	b	c	h
1	$xy^2 - x^2 + 1$	4	5	0,7	0,1
2	$\sqrt{5x^2 + 3} - 3y^2$	2,6	4,6	1,8	0,2
3	$\cos(4,5x - y^3) - 1,3$	-1	1	0,2	0,2
4	$2x^2 + xy + 3y^2$	2	3	1,2	0,1

№	$f(x, y)$	a	b	c	h
5	$e^{-(2y^2+3)} + 2x$	0	0,5	0,3	0,05
6	$\cos(2,5y + x)^2 + 2,4$	1	2	0,9	0,1
7	$5,1x - 2y^2 + 0,8$	0,6	2,6	3,4	0,2
8	$\frac{1}{2+3x^3y} + 2y$	1,5	2	2,1	0,05
9	$3x + 2\cos\frac{y}{\sqrt{15}}$	2,1	3,1	2,5	0,1
10	$\frac{4xy}{x+2} - 6,4$	3	5	1,7	0,2
11	$5x + 3\cos(y + 2,6)$	1	3	1,5	0,2
12	$3x + 5y^2 + 3$	1	2	0,9	0,1
13	$5 - 2\sin(x + 2y)^2$	2	3	2,3	0,1
14	$\frac{4}{x+3} + x + 5$	0,1	0,5	1,25	0,05
15	$7x + 2\cos\left(\frac{y}{3}\right)$	-2	-1	3	0,1

Задание №2

1. $y'' + \frac{y'}{x} + 3y = 2x.$

$$\begin{cases} y(0,7) = 0,51, \\ 2y(1) + 3y'(1) = 2,14. \end{cases}$$
2. $y'' + xy' + y = x + 2.$

$$\begin{cases} y(0,5) + 3y'(0,5) = 2, \\ y'(0,8) = 1,25. \end{cases}$$
3. $y'' + y' - xy = 2x^2.$

$$\begin{cases} y'(0,6) = 0,57, \\ y(0,9) - 0,95y'(0,9) = 3. \end{cases}$$
4. $y'' + 2y' + \frac{y}{x} = 5.$

$$\begin{cases} y(0,4) = 7, \\ y(0,7) - 2y'(0,7) = 3. \end{cases}$$
5. $y'' + xy' + 2y = x - 3.$

$$\begin{cases} y(0,9) - 4y'(0,9) = -1, \\ y(1,2) = 3. \end{cases}$$
6. $y'' + 3y' - \frac{2y}{x} = 5.$

$$\begin{cases} y(0,2) = 3, \\ 0,5y(0,5) + 3y'(0,5) = 7. \end{cases}$$
7. $y'' - 2y' + \frac{y}{x} = x + 1,8.$

$$\begin{cases} y(1,1) + 5.5y'(1,1) = 4.5, \\ y'(1,4) = 8. \end{cases}$$
8. $y'' - 7y' + \frac{y}{x} = x - 4.$

$$\begin{cases} y'(1,2) = 8, \\ y(1,5) - 2y'(1,5) = 7.4. \end{cases}$$
9. $y'' - 3,5y' + xy = 4,2.$

$$\begin{cases} 2y(1,3) + 3y'(1,3) = 7.5, \\ y(1,6) = 6. \end{cases}$$

- $$y'' + 4xy' + y = 4.9.$$
10.
$$\begin{cases} 2y(0, 3) - y'(0, 3) = 5, \\ y'(0, 6) = 7. \end{cases}$$
- $$y'' - \frac{3y'}{x} + 5y = 9.$$
11.
$$\begin{cases} y'(0, 8) = 1, 6, \\ 2y(1, 1) + 3y'(1, 1) = 4, 8. \end{cases}$$
- $$y'' - 5xy' + 3y = 2, 5.$$
12.
$$\begin{cases} y'(0, 7) = 3, 2, \\ 0, 5y(1) + 3y'(1) = 5, 4. \end{cases}$$
- $$y'' - \frac{2y'}{x} + 0.68y = 4x.$$
13.
$$\begin{cases} y(0, 6) - 3y'(0, 6) = 3.2, \\ y'(0, 9) = 7.4. \end{cases}$$
- $$y'' + 6\frac{y'}{2} - xy = 1.$$
14.
$$\begin{cases} 2y(0, 8) = 6, \\ 2y(1, 1) - y'(1, 1) = 3. \end{cases}$$
- $$y'' + 4y' - 5\frac{y}{x} = 3\frac{1}{x}.$$
15.
$$\begin{cases} 0, 5y(0, 9) - 4y'(0, 9) = 8, \\ 2y'(1, 2) = 2, 4. \end{cases}$$

Вопросы к защите лабораторной работы

1. Обыкновенные дифференциальные уравнения первого порядка. Задача Коши. Теорема о существовании и единственности задачи Коши.
2. Метод Эйлера.
3. Метод Эйлера-Коши.
4. Метод Рунге-Кутты.
5. Решение краевой задачи для линейного дифференциального уравнения второго порядка методом конечных разностей.

9. Численное решение уравнений в частных производных

9.1. Общие сведения из теории уравнений в частных производных

Уравнения вида

$$F \left(x_1, \dots, x_k, u, \frac{\partial u}{\partial x_1}, \dots, \frac{\partial u}{\partial x_k}, \frac{\partial^2 u}{\partial x_1^2}, \frac{\partial^2 u}{\partial x_1 \partial x_2}, \dots, \frac{\partial^n u}{\partial x_1^{p_1} \partial x_2^{p_2} \dots \partial x_k^{p_k}} \right) = 0,$$

где $p_1 + p_2 + \dots + p_k = n$, называются уравнениями в частных производных.

Наивысший порядок входящей в уравнение производной называется порядком дифференциального уравнения в частных производных.

Далее будем рассматривать классические уравнения математической физики. Это уравнения с двумя независимыми переменными: t – временем и пространственными координатами (декартовыми, цилиндрическими или сферическими).

Задача называется стационарной, если ее решение не зависит от времени, и нестационарной или эволюционной, если оно зависит от времени. Задачи с одной пространственной переменной называются одномерными, с двумя – двумерными, с тремя – трехмерными.

Каноническая форма уравнений с двумя независимыми переменными имеет следующий вид:

$$A(x, y) \frac{\partial^2 u}{\partial x^2} + 2B(x, y) \frac{\partial^2 u}{\partial x \partial y} + C(x, y) \frac{\partial^2 u}{\partial y^2} + F \left(x, y, u, \frac{\partial u}{\partial x}, \frac{\partial u}{\partial y} \right) = 0, \quad (23)$$

где коэффициенты A, B, C имеют непрерывные производные второго порядка и не обращаются одновременно в нуль.

Пусть $D = B^2 - AC$, тогда при $D = 0$ уравнение (23) – параболическое, при $D > 0$ – гиперболическое, при $D < 0$ – эллиптическое.

Классическим примером уравнения параболического типа является уравнение *теплопроводности* или *диффузии* в области $[0, l] \times [0, t]$:

$$\frac{\partial u}{\partial t} = a \frac{\partial^2 u}{\partial x^2}, \quad 0 < x < l,$$

где a – коэффициент теплопроводности (если u – температура) и массопроводности (если u – концентрация).

Классическим примером уравнения гиперболического типа является *волновое* уравнение

$$\frac{\partial^2 u}{\partial t^2} = a \frac{\partial^2 u}{\partial x^2}, \quad 0 < x < l, \quad t > 0,$$

описывающее малые продольные колебания стержня и поперечные колебания струны, причем функция $u(x, t)$ – отклонение струны от положения равновесия и a – скорость перемещения колебаний. К волновому уравнению сводятся также уравнения распространения малых акустических возмущений.

Классическим примером уравнения эллиптического типа являются уравнение Пуассона

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y), \quad x, y \in D,$$

и уравнение Лапласа

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0, \quad x, y \in D,$$

описывающее течение идеальной (без вязкости и теплопроводности) жидкости в стационарных потоках, стационарное распределение температуры, стационарное распределение напряженности электрического или магнитного полей. При этом уравнение Лапласа описывает все эти явления, когда нет источников или стоков (правая часть равна нулю), а уравнение Пуассона – с распределенными по области D источниками, задаваемыми правой частью $f(x, y)$.

9.2. Метод сеток

Решение уравнения методом сеток можно разделить на три этапа.

На первом этапе область, на которой ищется решение, заменяется дискретным множеством точек (разностной сеткой). Для этого проводятся прямые, параллельные осям координат. Расстояние между двумя параллельными прямыми называется шагом сетки. Шаги по осям x и y могут быть различными. Точки пересечения прямых (x, y) называются узлами разностной

сетки.

Если выбрать какой-нибудь начальный узел (x_0, y_0) , то любой узел можно представить в виде (x_i, y_j) , где $x_i = x_0 + ih$, $y_j = y_0 + jh$, $i, j = 0, \pm 1, \pm 2, \dots$

Множество всех узлов (x_i, y_j) , расположенных в области D или на ее границе Γ , обозначается через D_h и является сеточной областью, соответствующей области D (см. рис. 1).

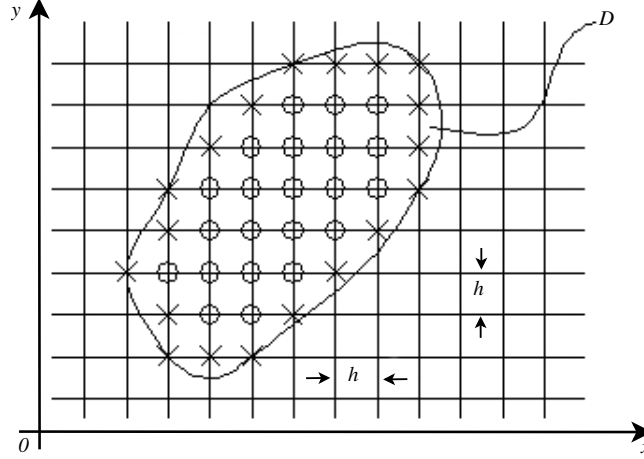


Рисунок 9.1. Сеточная область.

В сеточной области выделяют внутренние и граничные узлы. Возьмем некоторый узел $(x_i, y_j) \in D_h$. Этот узел будем считать внутренним, если он сам и четыре соседних узла (x_{i-1}, y_j) , (x_{i+1}, y_j) , (x_i, y_{j-1}) , (x_i, y_{j+1}) принадлежат сеточной области D_h . Рассматриваемые пять узлов образуют пятиточечный шаблон, который изображен на рис. 2.

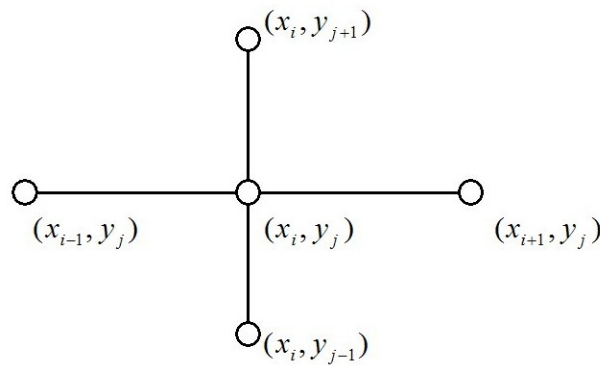


Рисунок 9.2. Пятиточечный шаблон

Множество всех внутренних узлов обозначим через D_h^0 . Узлы из сеточной области D_h , которые не принадлежат D_h^0 , являются граничными. Их совокупность обозначим через Γ_h . Таким образом, $D_h = D_h^0 \cup \Gamma_h$.

На рис. 1 внутренние узлы отмечены кружками, а граничные — крестиками.

ками. Решение ищется во внутренних узлах, а в граничных задается граничными условиями или находится с помощью аппроксимации из граничных условий исходной задачи.

На втором этапе дифференциальное уравнение и граничные условия заменяются своими разностными аналогами.

Например, рассмотрим случай с симметричным расположением узлов.

По определению, частная производная

$$\frac{\partial u(x, y)}{\partial x} = \lim_{\Delta x \rightarrow 0} \frac{u(x + \Delta x, y) - u(x, y)}{\Delta x}.$$

Если Δx достаточно мало, то справедливо равенство

$$\begin{aligned} \frac{\partial u(x, y)}{\partial x} &= \lim_{\Delta x \rightarrow 0} \frac{u(x + \Delta x, y) - u(x, y)}{\Delta x} \approx \\ &\approx \frac{u(x + \Delta x, y) - u(x, y)}{\Delta x}. \end{aligned}$$

Если рассматривать функцию только в узлах сетки, то

$$\frac{\partial u(x, y)}{\partial x} \approx \frac{u_{i+1,j} - u_{i,j}}{h},$$

где узел (i, j) соответствует точке (x, y) . Полученное выражение называется *правой конечной разностью*. Название связано с тем, что для вычисления производной в точке используются значения функции в этой точке и точке, лежащей правее.

Аналогично, если использовать точку, лежащую слева, получим

$$\frac{\partial u(x, y)}{\partial x} \approx \frac{u_{i,j} - u_{i-1,j}}{h}.$$

Такое выражение называется *левой конечной разностью*.

Найдя среднее этих выражений, получим *центральную конечную разность*

$$\frac{\partial u(x, y)}{\partial x} \approx \frac{u_{i+1,j} - u_{i-1,j}}{2h}.$$

Выражение для второй производной имеет вид

$$\frac{\partial^2 u(x, y)}{\partial x^2} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}.$$

Разностные операторы, соответствующие дифференциальному уравнению, записываются во внутренних узлах сетки. Разностные операторы, соответствующие граничным условиям, записываются в граничных узлах. В результате получается система алгебраических уравнений, число которых пропорционально числу внутренних узлов сеточной области.

На третьем этапе производится решение системы алгебраических уравнений. Как правило, это системы линейных алгебраических уравнений с разреженными матрицами.

9.3. Решение уравнений гиперболического типа

Пример. Будем решать волновое уравнение

$$\frac{\partial^2 u(x, t)}{\partial x^2} = \frac{1}{a^2} \frac{\partial^2 u(x, t)}{\partial t^2}$$

при $a = 1$ в прямоугольной области $[0, m] \times [0, n]$ ($m = 100$, $n = 200$) с шагом $h = 1$ изменения пространственной переменной, начальными условиями

$$u(x, 0) = \sin\left(\pi \cdot \frac{x}{50}\right),$$

$$\frac{\partial u(x, 0)}{\partial t} = 0$$

и граничными условиями $u(0, t) = u(m, t) = 0$ (на концах струны смещение равно 0 в любой момент времени).

Решение.

Используя разностные аналоги для частных производных

$$\frac{\partial^2 u(x, t)}{\partial x^2} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2},$$

$$\frac{\partial^2 u(x, t)}{\partial t^2} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\tau^2},$$

запишем уравнение в конечных разностях

$$\frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} = \frac{1}{a^2} \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\tau^2};$$

здесь индекс i соответствует пространственной, а j – временной координате.

Полученное уравнение позволяет выразить значение функции u в момент времени $j + 1$ через значения функции в предыдущие моменты времени

$$u_{i,j+1} = a^2 \left(\frac{\tau}{h}\right)^2 (u_{i+1,j} - 2u_{i,j} + u_{i-1,j}) + 2u_{i,j} - u_{i,j-1}.$$

Разностная схема такого вида называется явной. Она устойчива, если

$$\tau \leq \frac{h}{a}.$$

Реализовать вычисления по этой схеме на python можно следующим образом:

```

1  import matplotlib.pyplot as plt
2  from mpl_toolkits.mplot3d import axes3d
3  import numpy as np
4
5  # Grid
6  m = 100; n = 200
7  U = np.zeros((m + 1, n + 1), dtype = float)
8  h = 1; a = 1; tau = (h / float(a)) / 20
9
10 # Initial condttions
11 for i in range(m + 1):
12     U[i][0] = np.sin(np.pi * i / 50)
13     U[i][1] = U[i][0]
14
15 # Boundary conditions
16 for j in range(n + 1):
17     U[0][j] = 0
18     U[m][j] = 0
19
20 # Solution
21 for i in range(1, m):
22     for j in range(1, n):
23         U[i][j + 1] = pow(a, 2) * pow(tau / h, 2) * \
24             (U[i + 1][j] - 2 * U[i][j] + U[i - 1][j])\
25             + 2 * U[i][j] - U[i][j - 1]
26
27 # Plot surface
28 fig = plt.figure()
29 ax = fig.add_subplot(projection='3d')
30 X = range(n + 1)
31 Y = range(m + 1)
32 X, Y = np.meshgrid(X, Y)
33 ax.plot_surface(X, Y, U, rstride = 1, cstride = 1,\
34                 color = '#COCOCO')
35 plt.show()

```

Листинг 139

Результат:

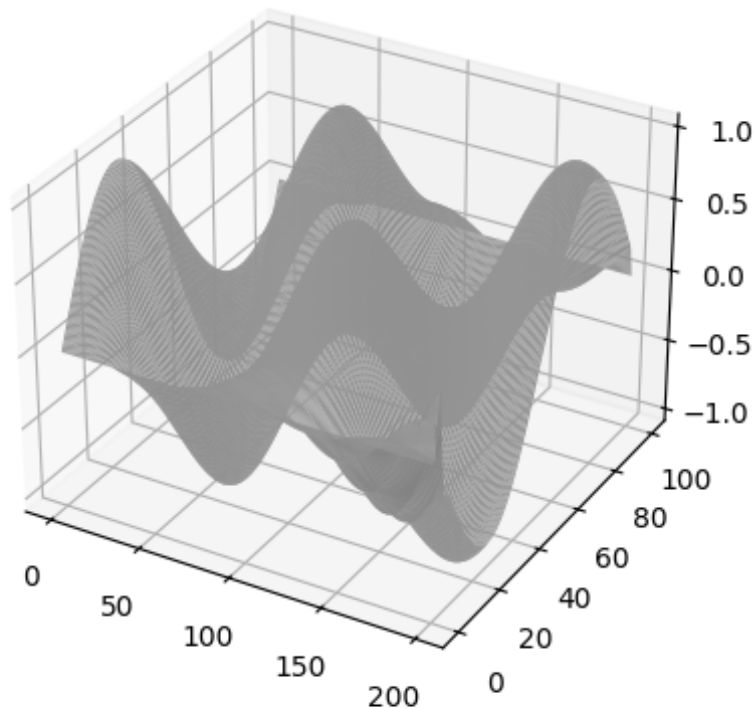


Рисунок 9.3. Решение волнового уравнения.

9.4. Решение уравнений параболического типа

Пример. Будем решать уравнение диффузии

$$\frac{\partial u(x, t)}{\partial t} = D \frac{\partial^2 u(x, t)}{\partial x^2}$$

при $D = 1$ в прямоугольной области $[0, 30] \times [0, 20]$ с шагом $h = 1$, начальным условием

$$u(x, 0) = \sin(x)$$

и граничными условиями

$$u(0, t) = t,$$

$$u(30, t) = \sin(30),$$

используя явную разностную схему.

Решение. Запишем явную разностную схему для уравнения диффузии:

$$u_{i,j+1} = D \frac{\tau}{h^2} (u_{i+1,j} - 2u_{i,j} + u_{i-1,j}) + u_{i,j}.$$

Она устойчива, если

$$\tau \leq \frac{h^2}{2D}.$$

Здесь i соответствует пространственной, а j – временной координате.

Реализовать вычисления по этой схеме на python можно следующим образом:

```
1  import matplotlib.pyplot as plt
2  from mpl_toolkits.mplot3d import axes3d
3  import numpy as np
4
5  # Grid
6  m = 30; n = 20
7  U = np.zeros((m + 1, n + 1), dtype = float)
8  h = 1; D = 1; tau = (pow(h, 2) / float(2 * D)) / 5
9
10 # Initial condititions
11 for i in xrange(m + 1):
12     U[i][0] = np.sin(i)
13
14 # Boundary conditions
15 for j in xrange(n + 1):
16     U[0][j] = j
17     U[m][j] = 1
18
19 # Solution
20 for i in xrange(1, m):
21     for j in xrange(n):
22         U[i][j + 1] = D * (tau / pow(h, 2)) * \
23             (U[i + 1][j] - 2 * U[i][j] + U[i - 1][j]) + U[i][j]
24
25 # Plot surface
26 ax = axes3d.Axes3D(plt.figure())
27 X = range(n + 1)
28 Y = range(m + 1)
29 X, Y = np.meshgrid(X,Y)
30 ax.plot_surface(X, Y, U, rstride = 1, cstride = 1,\
31                 color='#COCOCO')
32 plt.show()
```

Листинг 140

Результат:

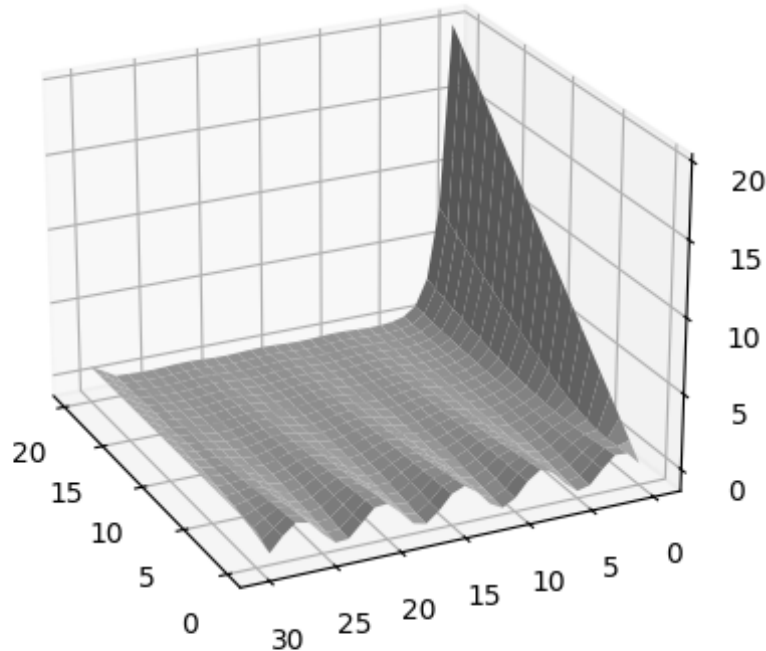


Рисунок 9.4. Решение уравнения теплопроводности.

Достоинством явных методов является простота. Недостатком является то, что накладываются жесткие ограничения на величину шага. Явные схемы устойчивы при очень малых шагах по времени.

Этого позволяют избежать неявные разностные схемы. В неявных разностных схемах значения искомой функции на очередном шаге непосредственно не выражаются через значения функции на предыдущем шаге.

Рассмотрим применение неявной схемы на примере уравнения теплопроводности.

Пример. Будем решать уравнение теплопроводности

$$\frac{\partial u(x, t)}{\partial t} = c \frac{\partial^2 u(x, t)}{\partial x^2}$$

при $c = 5$ в квадратной области $[0, n] \times [0, n]$ ($n = 30$) с шагом $h = 1$ пространственной переменной и $\tau = 1$ временной координаты, граничными условиями

$$u(0, t) = \alpha = 0, \quad u(n, t) = \beta = 0$$

и начальным условием

$$u(x, 0) = \sin\left(\pi \cdot \frac{x}{n}\right) + \frac{x\beta}{n} + \frac{(n-x)\alpha}{n}$$

(начальное распределение температуры в области), используя неявную разностную схему.

Решение. Запишем неявную разностную схему для уравнения теплопроводности:

$$\frac{u_{i,j+1} - u_{i,j}}{\tau} = \frac{c}{h^2} (u_{i+1,j+1} - 2u_{i,j+1} + u_{i-1,j+1}).$$

Здесь i соответствует пространственной, а j – временной координате. В отличие от явной схемы, для вычисления в правой части уравнения используются значения функции на том же самом временном шаге.

Пусть

$$\mu = \frac{c\tau}{h^2}.$$

Тогда

$$u_{i,j+1} - u_{i,j} = \mu (u_{i+1,j+1} - 2u_{i,j+1} + u_{i-1,j+1})$$

или

$$(1 + 2\mu) u_{i,j+1} - \mu (u_{i+1,j+1} + u_{i-1,j+1}) = u_{i,j}.$$

В матричной форме это уравнение имеет вид

$$\begin{pmatrix} 1 + 2\mu & -\mu & & & \\ -\mu & 1 + 2\mu & & & \\ & & \ddots & & \\ & & & 1 + 2\mu & -\mu \\ & & & -\mu & 1 + 2\mu \end{pmatrix} \cdot \begin{pmatrix} u_{1,j+1} \\ u_{2,j+1} \\ \vdots \\ u_{n-1,j+1} \\ u_{n,j+1} \end{pmatrix} = \begin{pmatrix} u_{1,j} + \mu\alpha \\ u_{2,j} \\ \vdots \\ u_{n-1,j} \\ u_{n,j} + \mu\beta \end{pmatrix}, \quad (1)$$

где

$$u(0, t) = \alpha, \quad u(n, t) = \beta.$$

Система (1) представляет собой последовательность систем линейных алгебраических уравнений

$$A \cdot X_j = B_j.$$

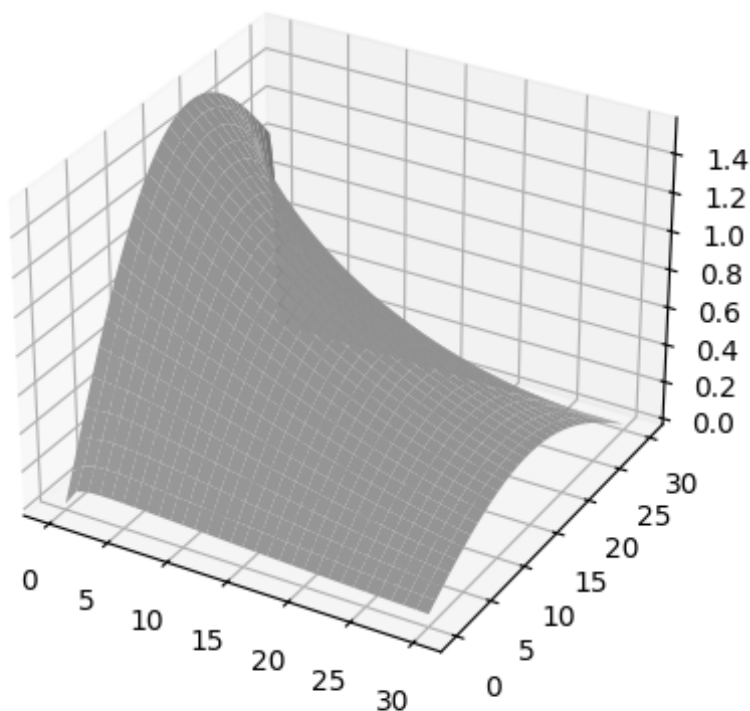
Решая последовательно эти системы, получим решение исходного уравнения. Реализовать вычисления по этой схеме на python можно следующим образом:

```

1  import matplotlib.pyplot as plt
2  from mpl_toolkits.mplot3d import axes3d
3  import numpy as np
4
5  # Grid
6  n = 30
7  U = np.zeros((n + 1, n + 1), dtype = float)
8  h = 1; tau = 1; c = 5; mu = c * tau / pow(h, 2)
9  # Initial conditions
10 for i in range(n + 1):
11     U[i][0] = np.sin(np.pi * i / n) + i / float(n)
12 # Boundary conditions
13 alpha = 0; beta = 0
14 for j in range(n + 1):
15     U[0][j] = alpha
16     U[n][j] = beta
17
18 # Solution
19 A = np.zeros((n + 1, n + 1), dtype = float)
20 B = np.zeros(n + 1, dtype = float)
21 X = np.zeros(n + 1, dtype = float)
22 for m in range(1, n):
23     A[m][m - 1] = -mu
24     A[m - 1][m] = -mu
25 for i in range(n + 1):
26     A[i][i] = 1 + 2 * mu
27 for j in range(n):
28     for i in range(n + 1):
29         B[i] = U[i][j]
30     B[0] = B[0] + alpha * mu
31     B[n] = B[n] + beta * mu
32     X = np.linalg.solve(A, B)
33     U[:, j + 1] = X[:]
34
35 # Plot surface
36 fig = plt.figure()
37 ax = fig.add_subplot(projection='3d')
38 X = range(n + 1)
39 Y = range(n + 1)
40 X, Y = np.meshgrid(X, Y)
41 ax.plot_surface(X, Y, U, rstride = 1, cstride = 1,\
42                 color = '#COCOCO')
43 plt.show()

```

Результат:



9.5. Решение уравнений эллиптического типа

Классическими примерами уравнений эллиптического типа являются уравнение Пуассона

$$\frac{\partial^2 u(x, y)}{\partial x^2} + \frac{\partial^2 u(x, y)}{\partial y^2} = f(x, y)$$

и уравнение Лапласа

$$\frac{\partial^2 u(x, y)}{\partial x^2} + \frac{\partial^2 u(x, y)}{\partial y^2} = 0.$$

Задача нахождения решения уравнения Пуассона $u(x, y)$ в плоской области D , причем на границе Γ области D искомая функция принимает заданные значения $\varphi(x, y)$ ($u(x, y)|_{\Gamma} = \varphi(x, y)$), называется задачей Дирихле.

Для численного решения этих уравнений можно использовать конечно-разностные схемы или метод релаксаций.

Уравнение Пуассона в конечных разностях имеет вид

$$u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j} = f_{i,j}.$$

Идея метода релаксации состоит в следующем. Если нет источников (уравнение Лапласа), то значение функции в данном узле на текущем шаге $k + 1$ определяется как среднее значение функции в ближайших узлах на предыдущем шаге k

$$u_{i,j}^{k+1} = \frac{1}{4} (u_{i-1,j}^k + u_{i+1,j}^k + u_{i,j-1}^k + u_{i,j+1}^k).$$

При наличии источников разностная схема имеет вид

$$u_{i,j}^{k+1} = \frac{1}{4} (u_{i-1,j}^k + u_{i+1,j}^k + u_{i,j-1}^k + u_{i,j+1}^k) - \frac{h^2}{4} f_{i,j}.$$

В методе релаксации необходимо задать начальное приближение, то есть значения функции во всех узлах области, а также граничные условия. Начальное значение можно получить, присвоив всем u_{ij} во внутренних узлах сетки значения, равные среднему значению всех граничных условий. В случае уравнения Пуассона необходимо задать значения функции $f(x, y)$ во всех узлах рассматриваемой области.

Недостатком данной схемы является низкая скорость сходимости.

Пример. Решить уравнение Пуассона

$$\frac{\partial^2 u(x, y)}{\partial x^2} + \frac{\partial^2 u(x, y)}{\partial y^2} = f(x, y)$$

в квадратной области $[0, n] \times [0, n]$ ($n = 20$) с шагом $h = 1$ изменения x и y , граничными условиями

$$\begin{aligned} u(x, n) &= -1, \\ u(0, y) &= 1 - 2\frac{y}{n}, \\ u(x, 0) &= 1, \quad u(n, y) = 1 - 2\frac{y}{n}. \end{aligned}$$

Начальное приближение — нули во всех внутренних точках области. Значения функции $f(x, y)$:

$$\begin{aligned} f_{i,j} &= 0, \\ f(6, 8) &= 10, \\ f(10, 8) &= -10. \end{aligned}$$

Реализовать решение этой задачи методом релаксации на python можно следующим образом:

```

1  from mpl_toolkits.mplot3d import axes3d
2  import matplotlib.pyplot as plt
3  from matplotlib import cm
4  import numpy as np
5
6  # Grid
7  n = 20; h = 1
8
9  # Initial conditions
10 U = np.zeros((n + 1, n + 1), dtype = float)
11 U1 = np.zeros((n + 1, n + 1), dtype = float)
12 f = np.zeros((n + 1, n + 1), dtype = float)
13 f[5][10] = 10; f[15][10] = -10
14
15 # Boundary conditions
16 for i in xrange(n + 1):
17     U[i][n] = -1
18     U[i][0] = 1
19
20 for j in xrange(n + 1):
21     U[0][j] = 1 - 2 * j / float(n)
22     U[n][j] = 1 - 2 * j / float(n)
23
24 #Solution
25 for k in xrange(100):
26     for i in xrange(1, n):
27         for j in xrange(1, n):
28             U1[i][j] = (1 / float(4)) * \
29                 (U[i - 1][j] + U[i + 1][j] + U[i][j - 1] + \
30                 U[i][j + 1]) - (pow(h, 2) / float(4)) * f[i][j]
31     for i in xrange(1, n):
32         for j in xrange(1, n):
33             U[i][j] = U1[i][j]
34
35 #Plot surface
36 ax = axes3d.Axes3D(plt.figure())
37 X = range(n + 1)
38 Y = range(n + 1)
39 X, Y = np.meshgrid(X, Y)
40 ax.plot_surface(X, Y, U, rstride = 1, cstride = 1, \
41                 color='#COCOCO')
42 plt.show()

```

Листинг 142

Результат:

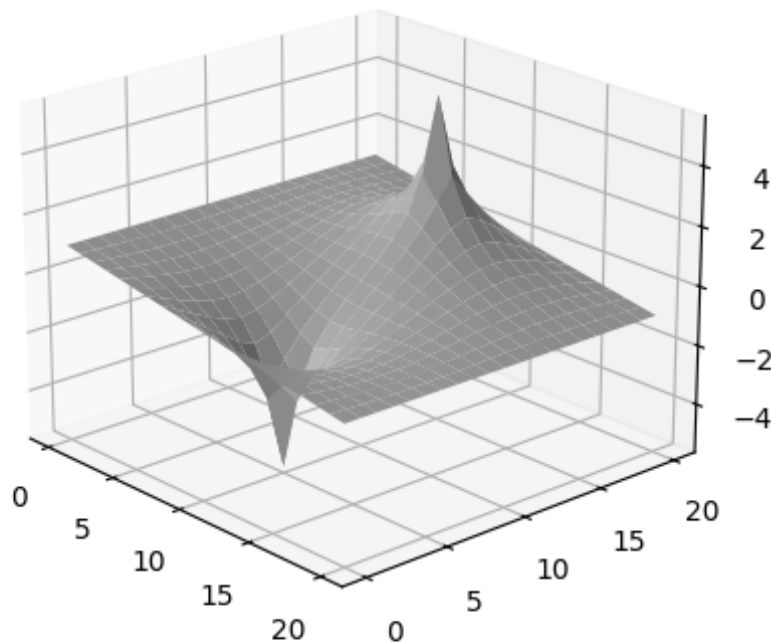


Рисунок 9.5. Решение уравнения Пуассона.

9.6. Задания для самостоятельной работы

Задание №1

Методом сеток с шагом h найти приближенное решение волнового уравнения

$$\frac{\partial^2 u(x, t)}{\partial x^2} = \frac{1}{a^2} \frac{\partial^2 u(x, t)}{\partial t^2}$$

(x – смещение, t – время) в прямоугольной области $[0, m] \times [0, n]$, используя явную разностную схему.

Граничные условия: на концах струны смещение равно 0 в любой момент времени:

$$u(0, t) = u(m, t) = 0.$$

Начальные условия:

$$\begin{aligned} u(x, 0) &= \varphi(x), \\ \frac{\partial u(x, 0)}{\partial t} &= 0. \end{aligned}$$

Так как концы струны закреплены, то $\varphi(0) = \varphi(m) = 0$.

Шаг изменения времени τ выбирается самостоятельно (так, чтобы разностная схема была устойчивой).

Результат представить в виде поверхности.

Вариант №1.

$$a = 1, m = 42, n = 30, h = 1, \\ \varphi(x) = 0, 1x \cdot \sin\left(\pi \cdot \frac{x}{21}\right).$$

Вариант №2.

$$a = 1, m = 36, n = 25, h = 1, \\ \varphi(x) = \sin\left(\pi \cdot \frac{x}{9}\right) \cdot \cos(\pi x).$$

Вариант №3.

$$a = 1, m = 39, n = 45, h = 1, \\ \varphi(x) = 0, 2 \cdot x \cdot \cos\left(\pi \cdot \frac{x}{2}\right).$$

Вариант №4.

$$a = 1, m = 28, n = 35, h = 1, \\ \varphi(x) = \sin^2(\pi x) \cdot \cos(2\pi x).$$

Вариант №5.

$$a = 1, m = 30, n = 45, h = 1, \\ \varphi(x) = 0, 3x \cdot \sin^2(\pi x).$$

Вариант №6.

$$a = 1, m = 44, n = 35, h = 1, \\ \varphi(x) = \cos(\pi x) \cdot \sin^2(3\pi x).$$

Вариант №7.

$$a = 1, m = 34, n = 20, h = 1,$$

$$\varphi(x) = 0, 2x \cdot \sin(\pi(x - 2)).$$

Вариант №8.

$$a = 1, m = 26, n = 25, h = 1,$$

$$\varphi(x) = 0, 1x \cdot \cos(\pi(x - 0, 5)).$$

Вариант №9.

$$a = 1, m = 46, n = 21, h = 1,$$

$$\varphi(x) = 0, 4x \cdot \sin\left(\pi \cdot \frac{x}{2}\right).$$

Вариант №10.

$$a = 1, m = 38, n = 27, h = 1,$$

$$\varphi(x) = \sin^2(\pi x) \cdot \cos(\pi x).$$

Вариант №11.

$$a = 1, m = 50, n = 23, h = 1,$$

$$\varphi(x) = \sin(\pi x) \cdot \cos\left(\frac{\pi x}{4}\right).$$

Вариант №12.

$$a = 1, m = 35, n = 28, h = 1,$$

$$\varphi(x) = \sin\left(\frac{\pi x}{3}\right) \cdot \cos\left(\frac{\pi x}{2}\right).$$

Вариант №13.

$$a = 1, m = 45, n = 29, h = 1,$$

$$\varphi(x) = 0, 2x \cdot \cos\left(3 \cdot \frac{x}{2}\right).$$

Вариант №14.

$$a = 1, m = 29, n = 22, h = 1,$$

$$\varphi(x) = 0, 1x \cdot \cos\left(\frac{\pi x}{2}\right).$$

Вариант №15.

$$a = 1, m = 43, n = 22, h = 1,$$
$$\varphi(x) = \sin(\pi x) \cdot \cos\left(\frac{\pi x}{2}\right).$$

Задание №2

Методом сеток с шагом h найти приближенное решение уравнения диффузии $\frac{\partial u(x,t)}{\partial t} = D \frac{\partial^2 u(x,t)}{\partial x^2}$ (x – смещение, t – время) в прямоугольной области $[0, a] \times [0, b]$, используя явную разностную схему.

Начальное условие:

$$u(x, 0) = f(x) \quad (t = 0, \quad 0 \leq x \leq a).$$

Граничные условия:

$$u(0, t) = g_1(t) \quad (x = 0, \quad 0 \leq t \leq b),$$
$$u(a, t) = g_2(t) \quad (x = a, \quad 0 \leq t \leq b).$$

Шаг изменения времени τ выбирается самостоятельно (так, чтобы разностная схема была устойчивой).

Результат представить в виде поверхности.

Вариант № 1.

$$D = 1, a = 60, b = 50, h = 1,$$

$$f(x) = \sin(x), g_1(t) = 0, g_2(t) = \sin(60).$$

Вариант № 2.

$$D = 2, a = 40, b = 60, h = 1,$$

$$f(x) = \cos(x), g_1(t) = 1, g_2(t) = \cos(40).$$

Вариант № 3.

$$D = 10, a = 30, b = 20, h = 1,$$

$$f(x) = \sin\left(\frac{x}{2}\right), g_1(t) = 0, g_2(t) = \sin(15).$$

Вариант № 4.

$$D = 5, a = 40, b = 20, h = 1,$$

$$f(x) = \cos\left(\frac{x}{2}\right), g_1(t) = 1, g_2(t) = \cos(20).$$

Вариант № 5.

$$D = 4, a = 60, b = 45, h = 1,$$

$$f(x) = x \cdot \sin(x), g_1(t) = 0, g_2(t) = 60 \cdot \sin(60).$$

Вариант № 6.

$$D = 8, a = 50, b = 35, h = 1,$$

$$f(x) = x \cdot \cos(x), g_1(t) = 0, g_2(t) = 50 \cdot \cos(50).$$

Вариант № 7.

$$D = 9, a = 55, b = 55, h = 1,$$

$$f(x) = 1, g_1(t) = 1, g_2(t) = 1.$$

Вариант № 8.

$$D = 11, a = 50, b = 70, h = 1,$$

$$f(x) = 4, g_1(t) = 4, g_2(t) = 4.$$

Вариант № 9.

$$D = 0.1, a = 52, b = 61, h = 1,$$

$$f(x) = 2 + \sin(x), g_1(t) = 2, g_2(t) = 2 + \sin(52).$$

Вариант № 10.

$$D = 0.21, a = 60, b = 47, h = 1,$$

$$f(x) = \cos(x) + 1, g_1(t) = 2, g_2(t) = \cos(60) + 1.$$

Вариант № 11.

$$D = 0.31, a = 49, b = 63, h = 1,$$

$$f(x) = \sin(x) - 1, g_1(t) = -1, g_2(t) = \sin(49) - 1.$$

Вариант № 12.

$$D = 1.5, a = 56, b = 72, h = 1,$$

$$f(x) = 2 + \cos\left(\frac{x}{2}\right), g_1(t) = 3, g_2(t) = 2 + \cos(28).$$

Вариант № 13.

$$D = 2.4, a = 54, b = 56, h = 1,$$

$$f(x) = x \cdot \sin(x - 1), g_1(t) = 0, g_2(t) = 54 \cdot \sin(53).$$

Вариант № 14.

$$D = 3.1, a = 117, b = 58, h = 1,$$

$$f(x) = x \cdot \cos(x + 1), g_1(t) = 0, g_2(t) = 117 \cdot \cos(118).$$

Вариант № 15.

$$D = 4, a = 48, b = 61, h = 1,$$

$$f(x) = \frac{(x - 15)^2}{100}, g_1(t) = 2.25, g_2(t) = 10.89.$$

Задание №3

Методом сеток с шагом $h = 1$ пространственной переменной и $\tau = 1$ временной координаты найти приближенное решение уравнения теплопроводности

$$\frac{\partial u(x, t)}{\partial t} = c \frac{\partial^2 u(x, t)}{\partial x^2}$$

(x — смещение, t — время) в квадрате $[0, n] \times [0, n]$, используя неявную разностную схему.

Начальное условие:

$$u(x, 0) = \sin\left(\frac{\pi x}{n}\right) + \frac{x}{n}\beta + \frac{(n - x)}{n}\alpha \quad (t = 0, 0 \leq x \leq n).$$

Граничные условия:

$$\begin{aligned}u(0, t) &= g_1(t) = \alpha & (x = 0, \quad 0 \leq t \leq n), \\u(n, t) &= g_2(t) = \beta & (x = n, \quad 0 \leq t \leq n).\end{aligned}$$

Результат представить в виде поверхности.

Вариант № 1.

$$n = 30, c = 5, \alpha = -1, \beta = 1.$$

Вариант № 2.

$$n = 50, c = 3, \alpha = 2, \beta = 3.$$

Вариант № 3.

$$n = 10, c = 4, \alpha = -1, \beta = 0.$$

Вариант № 4.

$$n = 40, c = 20, \alpha = -5, \beta = 5.$$

Вариант № 5.

$$n = 35, c = 15, \alpha = -2, \beta = 0.$$

Вариант № 6.

$$n = 32, c = 10, \alpha = 2, \beta = 5.$$

Вариант № 7.

$$n = 36, c = 3, \alpha = 0, \beta = 5.$$

Вариант № 8.

$$n = 42, c = 10, \alpha = 0, \beta = 0.5.$$

Вариант № 9.

$$n = 30, c = 6, \alpha = -1, \beta = 2.$$

Вариант № 10.

$$n = 50, c = 7, \alpha = -2, \beta = 3.$$

Вариант № 11.

$$n = 44, c = 8, \alpha = -2, \beta = 2.$$

Вариант № 12.

$$n = 52, c = 9, \alpha = 0, \beta = 0.5.$$

Вариант № 13.

$$n = 36, c = 11, \alpha = 0, \beta = 0.4.$$

Вариант № 14.

$$n = 48, c = 12, \alpha = 0.1, \beta = 0.2.$$

Вариант № 15.

$$n = 18, c = 5, \alpha = 0.5, \beta = 0.6.$$

Задание №4

Используя метод релаксации с шагом $h = 1$, найти приближенное решение уравнения Пуассона

$$\frac{\partial^2 u(x, y)}{\partial x^2} + \frac{\partial^2 u(x, y)}{\partial y^2} = f(x, y)$$

в квадрате K с вершинами $A(0, 0)$, $B(0, 10)$, $C(10, 10)$, $D(10, 0)$, удовлетворяющее на границе ∂K квадрата краевому условию

$$u(x, y)|_{\partial K} = \varphi(x, y).$$

Решение представить в виде поверхности.

Вариант № 1.

$$f(x, y) = 8x + 6y + 4,$$

$$\varphi(x, y) = x^2 + y^2.$$

Вариант № 2.

$$f(x, y) = 12x + 4,$$

$$\varphi(x, y) = x^2 - y^2.$$

Вариант № 3.

$$f(x, y) = 4x + 6y + 2,$$

$$\varphi(x, y) = xy.$$

Вариант № 4.

$$f(x, y) = 8y + 4,$$

$$\varphi(x, y) = x^2 + y^2 - x.$$

Вариант № 5.

$$\begin{aligned}f(x, y) &= 10x + 2, \\ \varphi(x, y) &= x^2 - y^2 - y.\end{aligned}$$

Вариант № 6.

$$\begin{aligned}f(x, y) &= 2y - 4, \\ \varphi(x, y) &= xy - x.\end{aligned}$$

Вариант № 7.

$$\begin{aligned}f(x, y) &= 4 - 4x, \\ \varphi(x, y) &= xy - x^2.\end{aligned}$$

Вариант № 8.

$$\begin{aligned}f(x, y) &= 2x, \\ \varphi(x, y) &= x^2 + y^2 - xy.\end{aligned}$$

Вариант № 9.

$$\begin{aligned}f(x, y) &= 2 - 2x, \\ \varphi(x, y) &= x^2 + y^2 + xy.\end{aligned}$$

Вариант № 10.

$$\begin{aligned}f(x, y) &= 6y + 2, \\ \varphi(x, y) &= xy - y.\end{aligned}$$

Вариант № 11.

$$\begin{aligned}f(x, y) &= 2y, \\ \varphi(x, y) &= xy - y^2.\end{aligned}$$

Вариант № 12.

$$\begin{aligned}f(x, y) &= 2x - 6y, \\ \varphi(x, y) &= xy - x^2 - y.\end{aligned}$$

Вариант № 13.

$$\begin{aligned}f(x, y) &= -2x, \\ \varphi(x, y) &= x^2 + y^2 - x - y.\end{aligned}$$

Вариант № 14.

$$f(x, y) = 2x + 6y + 4,$$

$$\varphi(x, y) = x^2 - xy.$$

Вариант № 15.

$$f(x, y) = 4y,$$

$$\varphi(x, y) = y^2 - xy.$$

Вопросы к защите лабораторной работы

1. Типы дифференциальных уравнений в частных производных с двумя переменными.
2. Основные этапы метода сеток.
3. Задача Дирихле для уравнений Пуассона и Лапласа.
4. Разностная аппроксимация первой и второй производной функции двух переменных.
5. Явные и неявные разностные схемы.

10. Численное решение интегральных уравнений

Интегральными уравнениями называются функциональные уравнения, которые содержат неизвестную функцию под знаком интеграла.

Одномерные линейные интегральные уравнения можно представить в виде

$$g(t)x(t) = \lambda \int_a^b k(t, s)x(s)ds + f(t), \quad (1)$$

где $k(t, s)$, $g(t)$ и $f(t)$ — заданные функции, $x(s)$ — неизвестная функция, $t \in [a, b]$. Если в этом уравнении пределы интегрирования являются постоянными, то уравнение (1) называется интегральным уравнением Фредгольма. Если верхний предел интегрирования является переменным, то уравнение (1) называется интегральным уравнением Вольтерра. Уравнение Вольтерра является частным случаем уравнения Фредгольма.

При $g(t) \equiv 0$ уравнение (1) называется уравнением I рода, при $g(t) \neq 0$ уравнение (1) называется уравнением II рода. Если $g(t) = 0$ для некоторых, но не всех $t \in [a, b]$, то уравнение (1) называется уравнением III рода.

Если $g(t) \equiv 1$ и $f(t) \equiv 0$, то уравнение (1) называется однородным, если $f(t) \neq 0$, то — неоднородным.

10.1. Численное решение интегральных уравнений Фредгольма

Рассмотрим решение уравнения вида

$$x(t) = \int_a^b k(t, s)x(s)ds + f(t) \quad (2)$$

методом квадратур.

Разобьем отрезок $[a, b]$ на равные части точками $t_i = a + i \cdot h$ ($h = \frac{b-a}{n}$, $i = 0, 1, \dots, n$). Получим систему уравнений

$$x(t_i) = \int_a^b k(t_i, s)x(s)ds + f(t_i). \quad (3)$$

Заменяя в последней системе интеграл с помощью какой-либо квадратурной формулы

$$\int_a^b k(t_i, s)x(s)ds \approx \sum_{j=0}^n A_j k(t_i, s_j)x(s_j),$$

получим систему линейных алгебраических уравнений

[illegible]

Учитывая, что разбиения по t и s совпадают, и вводя обозначения

$$x(t_i) = x(s_i) = x_i, k(t_i, s_j) = k_{ij}, f(t_i) = f_i (i = 0, 1 \dots, n),$$

систему (4) можно переписать в виде

[illegible]

Решая эту систему, получим численное решение уравнения (2).

Пример. Решить интегральное уравнение Фредгольма методом квадратур, используя формулу прямоугольников

$$x(t) = \int_0^1 tsx(s)ds + 1 - \frac{t}{2}.$$

Решение.

```

1 from numpy import sin, linalg
2
3 def k(t, s):
4     return t * s
5
6 def f(t):
7     return 1 - t / 2
8
9 a = 0; b = 1; n = 9
10 h = (b - a) / float(n);
11 t = [a + i * h for i in range(n + 1)]
12 s = [a + j * h for j in range(n + 1)]
13 A = []
14 for i in range(n + 1):
15     number = [i for i in range(n + 1)]
16     A.append(number)
17     for j in range(n + 1):
18         A[i][j] = -h * k(t[i], s[j])
19     if i == j:
20         A[i][j] = 1 - h * k(t[i], s[j])
21 B = [f(t[i]) for i in range(n + 1)]
22 print(linalg.solve(A, B))

```

Листинг 143

Результат:

[1. 1.01013514 1.02027027 1.03040541
1.04054054 1.05067568
1.06081081 1.07094595 1.08108108 1.09121622]

10.2. Численное решение интегральных уравнений Вольтерра

Рассмотрим решение уравнения

$$x(t) = \int_a^t k(t, s)x(s)ds + f(t) \quad (6)$$

методом квадратур.

Разобьем отрезок $[a, b]$ на равные части точками $t_i = a + i \cdot h$ ($h = \frac{b-a}{n}$, $i = 0, 1, \dots, n$). Получим систему уравнений

$$x(t_i) = \int_a^{t_i} k(t_i, s)x(s)ds + f(t_i). \quad (7)$$

Заменяя в последней системе интеграл с помощью какой-либо квадратурной формулы

$$\int_a^{t_i} k(t_i, s)x(s)ds \approx \sum_{j=0}^i A_j k(t_i, s_j)x(s_j),$$

получим систему линейных алгебраических уравнений

$$\begin{cases} x(t_0) = A_0 k(t_0, s_0)x(s_0) + f(t_0), \\ x(t_1) = A_0 k(t_1, s_0)x(s_0) + A_1 k(t_1, s_1)x(s_1) + f(t_1), \\ \dots \\ x(t_n) = A_0 k(t_n, s_0)x(s_0) + \dots + A_n k(t_n, s_n)x(s_n) + f(t_n). \end{cases} \quad (8)$$

Учитывая, что разбиения по t и s совпадают, и вводя обозначения $x(t_i) = x(s_i) = x_i$, $k(t_i, s_j) = k_{ij}$, $f(t_i) = f_i$ ($i = 0, 1, \dots, n$), систему (8) можно переписать в виде

$$\begin{cases} (1 - A_0 k_{00})x_0 = f_0, \\ -A_0 k_{10}x_0 + (1 - A_1 k_{11})x_1 = f_1, \\ \dots \\ -A_0 k_{n0}x_0 - A_1 k_{n1}x_1 - \dots + (1 - A_n k_{nn})x_n = f_n. \end{cases} \quad (9)$$

Решая эту систему, получим численное решение уравнения (6).

Пример. Решить интегральное уравнение Вольтерра методом квадратур,

используя формулу прямоугольников:

$$x(t) = \int_0^t 2tsx(s)ds + 1 - t^3, \quad t \in [0, 1].$$

Решение.

```
1  from math import sin
2
3  def k(t, s):
4      return 2 * t * s
5
6  def f(t):
7      return 1 - pow(t, 3)
8
9  a = 0; b = 1; n = 19
10 h = (b - a) / float(n);
11 t = [a + i * h for i in range(n + 1)]
12 s = [a + j * h for j in range(n + 1)]
13 number = [i for i in range(n + 1)]
14 X = number
15 X[0] = f(t[0]) / (1 - h * k(t[1], s[1]))
16 for i in range(1, n + 1):
17     b = f(t[i])
18     for j in range(0, i):
19         b = b + h * k(t[i], s[j]) * X[j]
20     X[i] = b / (1 - h * k(t[i], s[i]))
21 print(X)
```

Листинг 144

Результат:

```
[1.000291672743182, 1.000145836371591, 1.0005839415188273,
1.0013167493519155, 1.002349803975351, 1.003693146105693,
1.0053628329109678, 1.0073826775268524, 1.009786325132191,
1.012619822994163, 1.015944897618422, 1.019843230775462,
1.0244221391329222, 1.029822226653188, 1.0362278209679794,
1.04388136504793, 1.053103476979361, 1.0643212140406029,
1.0781083440942627, 1.0952434007813117]
```

10.3. Численное решение уравнений

Вольтерра-Фредгольма с частными интегралами

К интегральным уравнениям Вольтерра-Фредгольма с частными интегралами приводятся различные задачи механики сплошных сред [1, 34]. Явное решение таких уравнений в общем случае невозможно. Поэтому важное значение имеют приближенные и численные методы их решения.

В пространстве $C(D)$ непрерывных функций будем рассматривать интегральное уравнение Вольтерра-Фредгольма с частными интегралами

$$x(t, s) = \int_0^t l(t, s, \tau)x(\tau, s)d\tau + \int_0^1 m(t, s, \sigma)x(t, \sigma)d\sigma + f(t, s); \quad (10)$$

здесь $(t, s) \in D = [0, 1] \times [0, 1]$, а l, m, f – заданные непрерывные функции. Пусть

$$(M(t)x)(s) = \int_0^1 m(t, s, \sigma)x(\sigma)d\sigma, \quad t \in [0, 1].$$

Рассмотрим решение обратимого уравнения (10). В [2] установлена

Теорема. *Равносильны утверждения: уравнение (10) обратимо; 1 не принадлежит спектру оператора $M(t)$ при каждом $t \in [0, 1]$.*

Опишем простую и эффективную схему построения приближенного решения обратимого уравнения (10). Пусть $x \in C(D)$ — решение уравнения (10). Тогда

$$\forall \epsilon > 0 \exists \delta > 0 (|t - t'| < \delta \Rightarrow |x(t, s) - x(t', s)| < \epsilon).$$

Отрезок $[0, 1]$ разобьем точками $0 = t_0 < t_1 < \dots < t_n = 1$ на равные части с длиной меньше δ . Функция

$$x_n(t, s) = \begin{cases} x(t_{i-1}, s), & (t, s) \in [t_{i-1}, t_i] \times [0, 1] \quad (i = 1, \dots, n-1), \\ x(t_{n-1}, s), & (t, s) \in [t_{n-1}, 1] \times [0, 1] \quad (i = n), \end{cases}$$

достаточно хорошо аппроксимирует решение уравнения (10). Функции $x(t_i, s)$ удовлетворяют системе, которая получается из (10) подстановкой $t = t_i$, и аппроксимируется системой интегральных уравнений

$$x(t_i, s) = \frac{1}{n} \sum_{k=0}^{i-1} l(t_i, s, t_k)x(t_k, s) + \int_0^1 m(t_i, s, \sigma)x(t_i, \sigma)d\sigma + f(t_i, s) \quad (11)$$

($i = 0, 1, \dots, n-1$), где сумма равна 0 при $i = 0$. Решая последовательно по

$i = n$ обратимых уравнений (11), найдем $x(t_i, s)$. Тогда

$$x_n(t, s) \approx y_n(t, s) = \begin{cases} x_{i-1}(s), (t, s) \in [t_{i-1}, t_i) \times [0, 1], \\ i = 1, \dots, n-1, \\ x_{n-1}(s), (t, s) \in [t_{n-1}, 1] \times [0, 1], \\ i = n. \end{cases}$$

где $(t, s) \in [t_{i-1}, t_i) \times [0, 1]$ при $i = 1, \dots, n-1$ и $(t, s) \in [t_{n-1}, 1] \times [0, 1]$ при $i = n$. Таким образом, решение x уравнения (10) достаточно хорошо аппроксимируется ограниченной вектор-функцией $t \rightarrow y_n(t, s)$ со значениями в $C([0, 1])$, которая непрерывна во всех точках, кроме, быть может, точек $t_1, \dots, t_{n-1}$.

При численном решении уравнение (10) заменяется системой

$$x_{ij} = \frac{1}{n} \sum_{p=0}^{i-1} l(t_i, s_j, t_p) x_{pj} + \frac{1}{m} \sum_{q=0}^{m-1} m(t_i, s_j, s_q) x_{iq} + f_{ij} \quad (12)$$

($i = 0, 1, \dots, n$; $j = 0, 1, \dots, m$) линейных алгебраических уравнений, где точки t_i и точки s_j разбивают отрезок $[0, 1]$ на n и m равных частей соответственно. Полученная система фиксированием $i = 0, 1, \dots, n$ разбивается на $n+1$ систем, которые решаются последовательно, начиная с $i = 0$. Решение (x_{0q}) ($q = 0, 1, \dots, m$) системы с $i = 0$ подставляется в систему с $i = 1$ и находится x_{1q} ($q = 0, 1, \dots, m$). Найденные x_{0q} и x_{1q} ($q = 0, 1, \dots, m$) подставляются в систему с $i = 2$. Решая эту систему, находим x_{2q} ($q = 0, 1, \dots, m$). Продолжая этот процесс далее, получаем x_{ij} ($i = 0, 1, \dots, n$; $j = 0, 1, \dots, m$). Производя в заключение линейную интерполяцию, находим приближённое непрерывное решение уравнения (10).

Данный алгоритм легко реализуется с помощью языка программирования Python.

Пример. Найти численное решение уравнения

$$x(t, s) = \int_0^t (t+s+\tau^2) x(\tau, s) d\tau + \int_0^1 s \cdot \sigma x(t, \sigma) d\sigma + 1 - t^2 - ts - \frac{t^3}{3} - \frac{s}{2},$$

где $(t, s) \in D = [0, 1] \times [0, 1]$. Точное решение этого уравнения $x(t, s) = 1$.

В результате численных расчетов по приведенной схеме при $n=4$ получим

```

[[ 1.          1.          1.          1.          ]
 [ 0.99869792  0.99847629  0.99825465  0.99803302]
 [ 0.9971873   0.99652286  0.99583073  0.99511089]
 [ 0.99500177  0.99336605  0.9916082   0.98972304]]

```

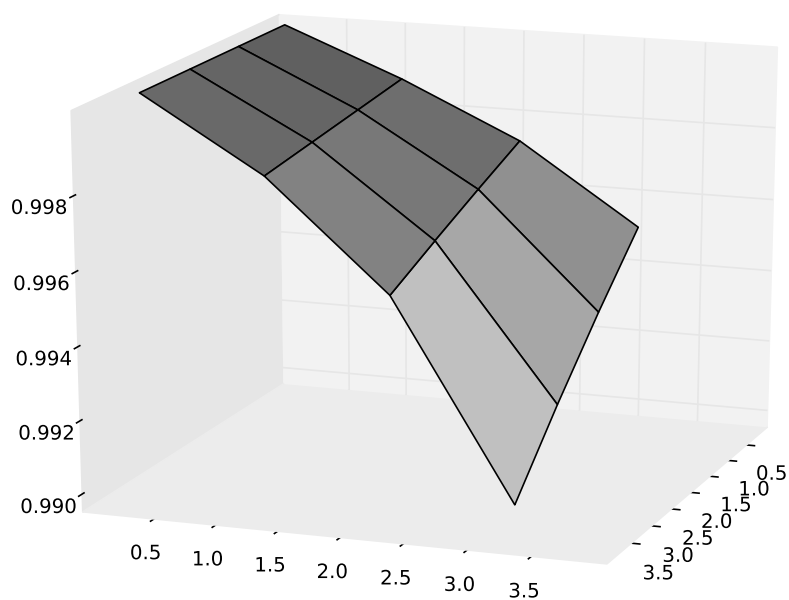


Рисунок 10.1. Графическое представление решения ($n=4$).

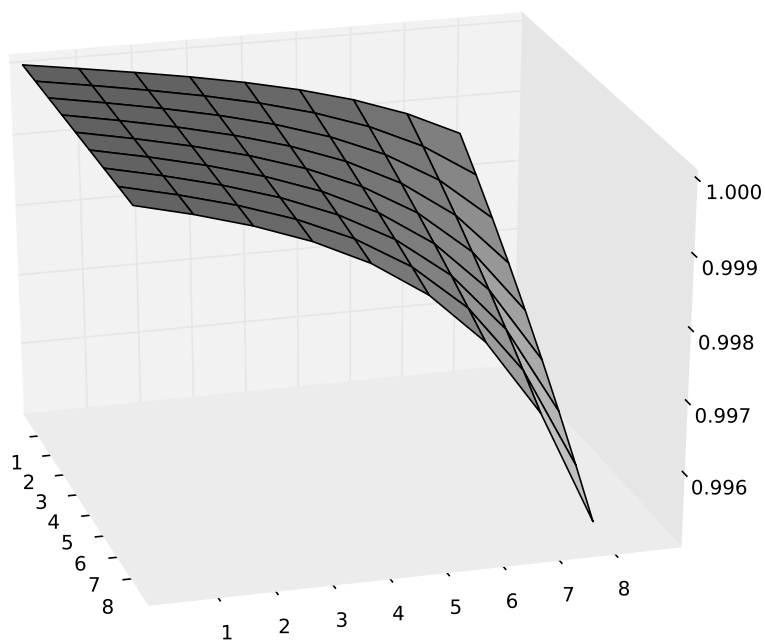


Рисунок 10.2. Графическое представление решения ($n=9$).

Пример. Найти численное решение уравнения

$$x(t, s) = \int_0^t \tau x(\tau, s) d\tau + \int_0^1 \sigma x(t, \sigma) d\sigma + s^2 - \frac{t^4}{4} - \frac{t^2 s^2}{2} - \frac{1}{4} + \frac{t^2}{2},$$

где

$$(t, s) \in D = [0, 1] \times [0, 1].$$

Точное решение этого уравнения

$$x(t, s) = t^2 + s^2.$$

В результате численных расчетов по приведенной схеме при $n=4$ получим

```
[ [ 0.3359375  0.3984375  0.5859375  0.8984375 ]
  [ 0.41748047 0.47998047 0.66748047 0.97998047]
  [ 0.65396118 0.71646118 0.90396118 1.21646118]
  [ 1.04387093 1.10637093 1.29387093 1.60637093] ]
```

Графическое представление решения ($n=4$)

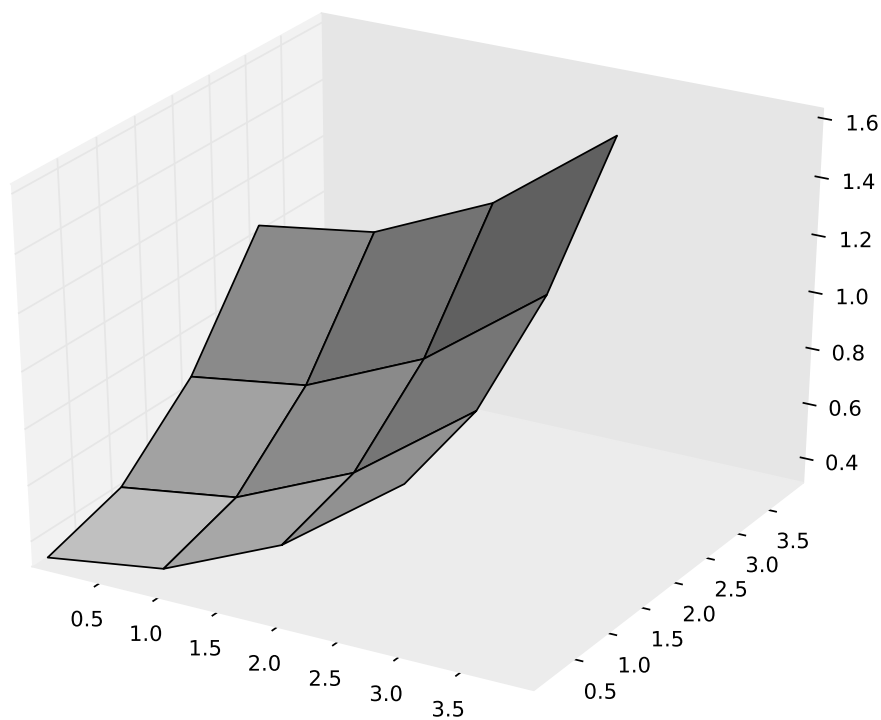


Рисунок 10.3. Графическое представление решения ($n=4$).

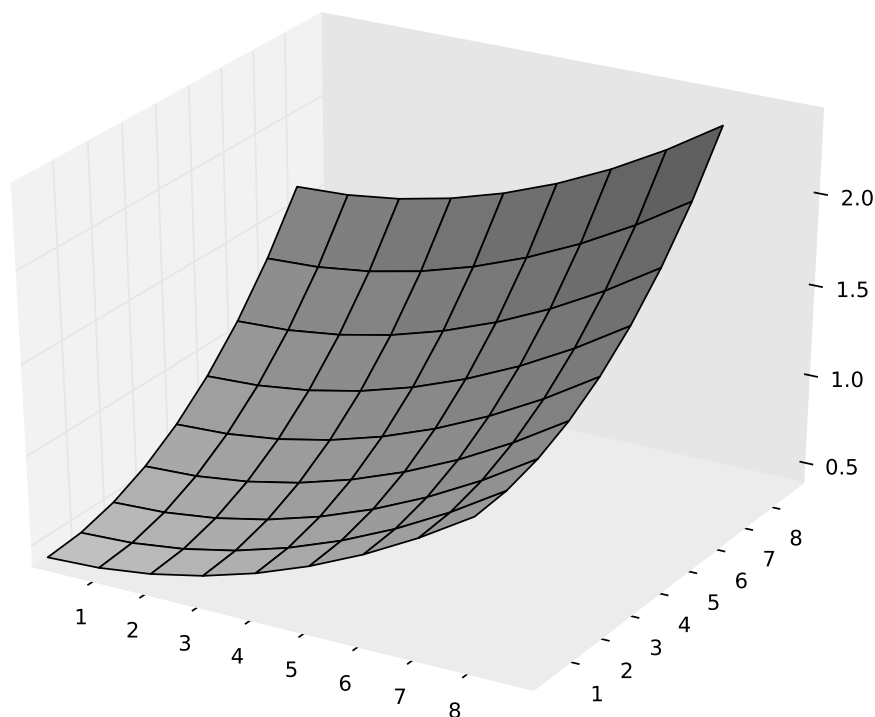


Рисунок 10.4. Графическое представление решения ($n=9$).

10.4. Задания для самостоятельной работы

1. Решить интегральное уравнение Фредгольма второго рода методом квадратур с использованием формулы:

- а) прямоугольников;
- б) трапеций;
- в) Симпсона.

Варианты заданий

$$1. x(t) = \int_0^1 (t + 2s)x(s)ds - 1.$$

$$2. x(t) = \int_0^1 (t + 2s)x(s)ds - t.$$

$$3. x(t) = \int_0^1 (2ts)x(s)ds + t.$$

$$4. x(t) = \int_0^1 (ts)x(s)ds + 2t.$$

$$5. x(t) = \int_0^1 (ts)x(s)ds - 1 - \frac{t}{2}.$$

$$6. x(t) = \int_0^2 (t-s)x(s)ds + 2 - 3t.$$

$$7. x(t) = \int_0^2 (t+s)x(s)ds - 2t.$$

$$8. x(t) = \int_0^1 (t^2 + s)x(s)ds + \frac{1}{2} + t - t^2.$$

$$9. x(t) = \int_0^1 (t^2 - s)x(s)ds + \frac{1}{2}.$$

$$10. x(t) = \int_0^1 (t^2 + s^2)x(s)ds - \frac{4}{3} + t - t^2.$$

$$11. x(t) = \int_0^1 (t+s)^2 x(s)ds - t - \frac{1}{3}.$$

$$12. x(t) = \int_0^1 (t+2s)x(s)ds + 2 - t^2.$$

$$13. x(t) = \int_0^1 (2ts + t^2)x(s)ds + 1 - t^2.$$

$$14. x(t) = \int_0^1 (-t^2 - s)x(s)ds + t^2 - \frac{3}{2}.$$

$$15. x(t) = \int_0^1 (s-t)x(s)ds - \frac{1}{2} - t.$$

2. Решить интегральное уравнение Вольтерра второго рода методом квадратур с применением формулы:

а) прямоугольников;

б) трапеций;

в) Симпсона.

$$1. x(t) = \int_0^t (ts)x(s)ds + 1 - \frac{t^3}{2}, t \in [0, 1].$$

$$2. x(t) = \int_0^t (t+s)x(s)ds - \frac{t^2}{2}, t \in [0, 1].$$

$$3. x(t) = \int_0^t (t-s)x(s)ds + \frac{t^2}{2}, t \in [0, 1].$$

$$4. x(t) = \int_0^t (t^2+s)x(s)ds + \frac{t^2}{2} - t^3, t \in [0, 1].$$

$$5. x(t) = \int_0^t (t^2+s)x(s)ds - \frac{t^2}{2}, t \in [0, 1].$$

$$6. x(t) = \int_0^t (t^2+s)x(s)ds - t^3, t \in [0, 1].$$

$$7. x(t) = \int_0^t (t^2+s^2)x(s)ds - \frac{t^3}{3}, t \in [0, 1].$$

$$8. x(t) = \int_0^t (t^2+s^2)x(s)ds - t^3, t \in [0, 1].$$

$$9. x(t) = \int_0^t (2ts)x(s)ds + 2 - t^3, t \in [0, 1].$$

$$10. x(t) = \int_0^t (t^2-s^2)x(s)ds + 5 - \frac{2t^3}{3}, t \in [0, 1].$$

$$11. x(t) = \int_0^t (2t+3s)x(s)ds + 4 - \frac{7t^2}{2}, t \in [0, 1].$$

$$12. x(t) = \int_0^t (t-s)x(s)ds - 1 - \frac{t^2}{2}, t \in [0, 1].$$

$$13. x(t) = \int_0^t (2t+3s)x(s)ds - 3t^2, t \in [0, 1].$$

$$14. x(t) = \int_0^t (t-s)x(s)ds + \frac{t^2}{2}, t \in [0, 1].$$

$$15. x(t) = \int_0^t (t^2 + s)x(s)ds + \frac{t^2}{2}, t \in [0, 1].$$

3. Найти численное решение интегрального уравнения Вольтерра-Фредгольма с частными интегралами $(t, s \in [0, 1] \times [0, 1])$:

$$1. x(t, s) = \int_0^t (t+2s+\tau^2)x(\tau, s)d\tau + \int_0^1 s\sigma x(t, \sigma)d\sigma + 2 - 2t^2 - 4ts - \frac{t^3}{3} - s,$$

$$2. x(t, s) = \int_0^t (2t+s+\tau^2)x(\tau, s)d\tau + \int_0^1 2t\sigma x(t, \sigma)d\sigma + s + \frac{t}{3} - t^2 - t^3 - \frac{5}{2}t^2s - s^2t - \frac{1}{4}t^4 - \frac{1}{3}t^3s,$$

$$3. x(t, s) = \int_0^t (t+s+2\tau)x(\tau, s)d\tau + \int_0^1 ts\sigma x(t, \sigma)d\sigma + 5 - 10t^2 - 5ts - \frac{5}{2}ts,$$

$$4. x(t, s) = \int_0^t (t+s+\tau)x(\tau, s)d\tau + \int_0^1 ts\sigma x(t, \sigma)d\sigma + 3 - \frac{9}{2}t^2 - \frac{9}{2}ts,$$

$$5. x(t, s) = \int_0^t (ts\tau)x(\tau, s)d\tau + \int_0^1 (s+\sigma)x(t, \sigma)d\sigma + 2 - 2t^3s + 4s,$$

$$6. x(t, s) = \int_0^t (ts\tau)x(\tau, s)d\tau + \int_0^1 ts\sigma x(t, \sigma)d\sigma + 5 - \frac{5}{2}t^3s + \frac{5}{2}ts,$$

$$7. x(t, s) = \int_0^t (ts\tau)x(\tau, s)d\tau + \int_0^1 (t+\sigma)x(t, \sigma)d\sigma + \frac{7}{2} - \frac{7}{2}t^3s - 7t,$$

$$8. x(t, s) = \int_0^t (ts\tau)x(\tau, s)d\tau + \int_0^1 (t+s+\sigma)x(t, \sigma)d\sigma + 3 - 3t^2s - 6t - 6s,$$

$$9. x(t, s) = \int_0^t t(s+\tau)x(\tau, s)d\tau + \int_0^1 (s+\sigma)x(t, \sigma)d\sigma + 4 - 8t^2s - 4t^3 - 8s,$$

$$10. x(t, s) = \int_0^t (ts\tau^2)x(\tau, s)d\tau + \int_0^1 (t+\sigma^2)x(t, \sigma)d\sigma + 8 - 4t^4s - 12t,$$

$$11. x(t, s) = \int_0^t (ts^2\tau^2)x(\tau, s)d\tau + \int_0^1 (s^2 + \sigma^2)x(t, \sigma)d\sigma + 6 - 3t^4s - 9s^2,$$

$$12. x(t, s) = \int_0^t (ts\tau)x(\tau, s)d\tau + \int_0^1 (t - \sigma^2)x(t, \sigma)d\sigma + 20 - \frac{15}{2}t^3s - 15t,$$

$$13. x(t, s) = \int_0^t (ts + \tau)x(\tau, s)d\tau + \int_0^1 (ts - \sigma)x(t, \sigma)d\sigma + 15 - 10t^2s - 5t^2 - 10ts,$$

$$14. x(t, s) = \int_0^t (ts - \tau)x(\tau, s)d\tau + \int_0^1 (ts + \sigma)x(t, \sigma)d\sigma + 8 - 16t^2s - 16ts - 8t^2,$$

$$15. x(t, s) = \int_0^t (t + s\tau)x(\tau, s)d\tau + \int_0^1 (ts + \sigma^2)x(t, \sigma)d\sigma + 12 - 18t^2 - 27ts.$$

Вопросы к защите лабораторной работы

1. Понятие интегрального уравнения. Типы интегральных уравнений.
2. Численное решение интегральных уравнений Фредгольма.
3. Численное решение интегральных уравнений Вольтерра.
4. Численное решение уравнений Вольтерра - Фредгольма с частными интегралами.

Литература

1. Appell, J.M. Partial Integral Operators and Integro-Differential Equations [Текст] / J.M. Appell, A.S. Kalitvin, P.P. Zabrejko. – New York: Marcel Dekker, 2000. – 560 p.
2. Александров, В.М. Задачи механики сплошных сред со смешанными граничными условиями / В.М. Александров, Е.В. Коваленко. – М.: Наука, 1986. – 326 с.
3. Арушанян, О.Б. Практикум на ЭВМ по вычислительным методам. Решение задачи Коши для обыкновенных дифференциальных уравнений одношаговыми разностными методами / О.Б. Арушанян. – М.: МГУ, 2002. – 51 с.
4. Арушанян, О.Б. Практикум на ЭВМ по вычислительным методам. Численное решение нелинейных уравнений / О.Б. Арушанян. – М.: МГУ, 2003. – 37 с.
5. Бабичева, Л.А. Методические указания и задания к выполнению лабораторной работы по теме «Решение задачи Дирихле методом сеток» / Л.А. Бабичева, В.М. Тюрин, В.Я. Ярославцева. – Липецк: ЛипПИ, 1983. – 28 с.
6. Бахвалов, Н.С. Численные методы / Н.С. Бахвалов. – М.: Наука, 1973. – 631 с.
7. Бахвалов, Н.С. Численные методы: учебное пособие / Н.С. Бахвалов, Н.П. Жидков, Г.М. Кобельков. – М.: Наука, 1987. – 600 с.
8. Березин, И.С. Методы вычислений / И.С. Березин, Н.П. Жидков. – М.: Физматгиз, 1962. – Ч. 1, 2. – 680 с.
9. Бизли, Дэвид. Python. Исчерпывающее руководство / Дэвид Бизли. – СПб.: Питер, 2023. – 368 с.
10. Бейдер, Д. Чистый Python. Тонкости программирования для профи / Д. Бейдер. – СПб.: Питер, 2019. – 288 с.
11. Буслов, В.А. Численные методы I. Исследование функций: Курс лекций / В.А. Буслов, С.Л. Яковлев. – Санкт-Петербург: Санкт-Петербургский государственный университет, 2001. – 59 с.
12. Буслов, В.А. Численные методы II. Решение уравнений: Курс лекций / В.А. Буслов, С.Л. Яковлев. – Санкт-Петербург: Санкт-Петербургский государственный университет, 2001. – 44 с.

13. Васильев, Алексей. Программирование на Python в примерах и задачах / Алексей Васильев. – Москва : Эксмо, 2021. – 616 с.
14. Васильев, А.Н. Python на примерах. Практический курс по программированию / А.Н. Васильев. – СПб.: Наука и Техника, 2016. – 432 с.
15. Вержбицкий, В.М. Численные методы. Линейная алгебра и нелинейные уравнения / В.М. Вержбицкий. – М.: Высшая школа, 2000. – 266 с.
16. Верлань, А.Ф. Интегральные уравнения. Методы, алгоритмы, программы: справочное пособие / А.Ф. Верлань, В.С. Сизиков. – Киев: Наукова Думка, 1986. – 543 с.
17. Волков, Е.А. Численные методы / Е.А. Волков – СПб.: Лань, 2004. – 256 с.
18. Гэддис, Т. Начинаем программировать на Python / Т. Гэддис. – 5-е изд. – СПб.: БХВ-Петербург, 2022. – 880 с.
19. Дауни, Аллен Б. Основы Python. Научитесь думать как программист / Аллен Б. Дауни. – Москва: Манн, Иванов и Фербер, 2021. – 304 с.
20. Доусон, М. Програмируемо на Python / М. Доусон. – СПб.: Питер, 2014. – 416 с.
21. Даугавет, И.К. Теория приближенных методов. Линейные уравнения / И.К. Даугавет. – СПб.: БХВ-Петербург, 2006. – 288 с.
22. Заварыкин, В.М. Техника вычислений и алгоритмизация. Вводный курс / В.М. Заварыкин, В.Г. Житомирский, М.П. Лапчик. – М.: Просвещение, 1987. – 160 с.
23. Заварыкин, В.М. Численные методы: учеб. пособие / В.М. Заварыкин, В.Г. Житомирский, М.П. Лапчик. – М.: Просвещение, 1990. – 176 с.
24. Златопольский, Д.М. Основы программирования на языке Python / Д.М. Златопольский. 2-е изд. – М.: ДМК Пресс, 2018. – 396 с.
25. Калитвин, А.С. Интегральные уравнения Вольтерра и Вольтерра-Фредгольма с частными интегралами / А.С. Калитвин, В.А. Калитвин. – Липецк: ЛГПУ, 2006. – 177 с.
26. Калитвин, А.С. Курс лекций по обыкновенным дифференциальным уравнениям: учебное пособие / А.С. Калитвин. – Липецк: ЛГПУ, 2007. – 340 с.
27. Калитвин, В.А. Теорема Банаха и ее применения. Примеры решения задач в пакетах Mathcad и Scilab: учебное пособие / В.А. Калитвин. – Липецк: ЛГПУ, 2006. – 30 с.
28. Калитвин, В.А. Численные методы. Использование C++: учебное пособие / В.А. Калитвин. – Липецк: ЛГПУ имени П.П. Семенова-Тян-Шанского,

2019. – 143 с.

29. Калитвин, В.А. Численное решение дифференциальных уравнений с использованием Python: учебное пособие / В.А. Калитвин. – Липецк: ЛГПУ, 2010. – 42 с.
30. Калитвин, В.А. Численное решение интегральных уравнений с использованием Python: учебное пособие / В.А. Калитвин. – Липецк: ЛГПУ, 2010. – 30 с.
31. Калитвин, В.А. Численное решение уравнений и систем с использованием Python: учебное пособие / В.А. Калитвин. – Липецк: ЛГПУ, 2010. – 41 с.
32. Калитвин, В.А. Численные методы. Использование Mathcad: учебное пособие / В.А. Калитвин. – Липецк: ЛГПУ, 2006. – 126 с.
33. Калитвин, В.А. Численные методы. Использование Scilab: учебное пособие / В.А. Калитвин. – Липецк: ЛГПУ, 2009. – 179 с.
34. Калитвин, В.А. Введение в программирование на Python: учебное пособие / В.А. Калитвин. – Липецк: ЛГПУ имени П.П. Семенова-Тян-Шанского, 2023. – 84 с.
35. Калитвин, В.А. Об оценке времени работы приложения для численного решения уравнений Вольтерра-Фредгольма с частными интегралами. Актуальные проблемы естественных, математических, технических наук и их преподавания: сборник научных трудов. – Липецк: ЛГПУ имени П.П. Семенова-Тян-Шанского, 2023. - С. 84-90.
36. Калитвин, В.А. Численное решение уравнений Вольтерра - Фредгольма с частными интегралами. Свидетельство о государственной регистрации программы для ЭВМ №2008611447/ В.А. Калитвин. – Москва: ФИПС, 2008.
37. Калитвин, В.А. Численное решение линейных интегро-дифференциальных уравнений Барбашина. Версия 1.1. (IDUB-LINEAR-1.1). Свидетельство о регистрации программы для ЭВМ RU 2023689242, 27.12.2023. Заявка от 11.12.2023.
38. Калитвин, В.А. Численное решение линейных уравнений Вольтерра с частными интегралами. Версия 1.2. (INTEQ-LVPI-1.2). Свидетельство о регистрации программы для ЭВМ RU 2023689159, 26.12.2023. Заявка от 15.12.2023.
39. Калитвин, В.А. Численное решение уравнений с частными интегралами

- с переменными и постоянными пределами интегрирования. Версия 1.1. (INTEQ-VCLIMITS-1.1). Свидетельство о регистрации программы для ЭВМ RU 2023688116, 20.12.2023. Заявка от 10.12.2023.
40. Калитвин В.А. О методах численного решения интегральных уравнений Романовского с частными интегралами. // Международная конференция по дифференциальным уравнениям и динамическим системам. Тезисы докладов. Суздаль, 30 июня – 5 июля 2022. – Владимир: ООО «Аркаим». С. 131 - 134. - ISBN 978-5-93767-460-9.
 41. Калиткин, Н.Н. Численные методы / Н.Н. Калиткин. – М.: Наука, 1978. – 512 с.
 42. Каханер, Д. Численные методы и программное обеспечение / Д. Каханер, К. Моулер, С. Нэш. – М.: Мир, 1998. – 575 с.
 43. Кольцов, Д.М. Python. Полное руководство / Д.М. Кольцов. – СПб.: Наука и техника, 2022. – 480 с.
 44. Костомаров, Д.П. Вводные лекции по численным методам: учебное пособие / Д.П. Костомаров, А.П. Фаворский. – М.: Логос, 2004. – 184с.
 45. Лутц, М. Программирование на Python / М. Лутц. – 4-е издание. – СПб.: Символ-Плюс, 2011. – Т. 1. – 992 с.
 46. Лутц, М. Изучаем Python / М. Лутц; пер. с англ. — изд. 3-е. — СПб.: Символ-Плюс, 2009. – 848 с.
 47. Любанович, Билл. Простой Python. Современный стиль программирования / Билл Любанович. – 2-е изд. – СПб.: Питер, 2021. – 592 с.
 48. Мэтьюз, Джон, Г. Численные методы. Использование MATLAB / Джон Г. Мэтьюз, Куртис Д. Финк. – М.: Издательский дом «Вильямс», 2001. – 720 с.
 49. Мэтиз Эрик. Изучаем Python. Программирование игр, визуализация данных, веб-приложения. – СПб.: Питер, 2017. – 496 с.
 50. Пирумов, У.Г. Численные методы / У.Г. Пирумов. – М.: Изд-во МАИ, 1998. – 188 с.
 51. Поршневу, С.В. Вычислительная математика: Курс лекций / С.В. Поршневу. – СПб.: БХВ-Петербург, 2004. – 320 с.
 52. Прохоренку, Н.А. Python 3 и PyQt 6. Разработка приложений / Н.А. Прохоренку, В.А. Дронов. – СПб.: БХВ-Петербург, 2023. – 832 с.
 53. Самарский, А.А. Численные методы / А.А. Самарский, А.В. Гулин. – М.: Наука, 1989. – 432 с.

54. Саммерфилд, М. Программирование на Python 3. Подробное руководство / М. Саммерфилд. – СПб.: Символ-Плюс, 2009. – 608 с.
55. Седер, Наоми. Python. Экспресс-курс / Наоми Седер. – 3-е изд. – СПб.: Питер, 2019. – 480 с.
56. Седжвик, Роберт. Программирование на языке Python: учебный курс / Роберт Седжвик, Кевин Уэйн, Роберт Дондеро; Пер. с англ. – СПб.: ООО «Альфа-книга», 2017. – 736 с.
57. Сурнова, В.П. Решение краевой задачи для линейного дифференциального уравнения второго порядка с использованием комплекса «ИСКРА-1256»: методические указания и задания к выполнению лабораторной работы / В.П. Сурнова, Н.М. Цыганкова, С.Ф. Василевский. – Липецк: ЛипПИ, 1988. – 15 с.
58. Яворски, Михал. Python. Лучшие практики и инструменты / Михал Яворски, , Тарек Зиаде. – СПб.: Питер, 2021. – 560 с.
59. Kalitvin, V.A.. On Algorithm for the Numerical Solution of Linear Volterra Equations with Partial Integrals and Its Implementation. 2022 4th International Conference on Control Systems, Mathematical Modeling, Automation and Energy Efficiency (SUMMA), 2022, pp. 164-166, doi: 10.1109/SUMMA57301.2022.9973436.
60. Devpractice Team. Python. Визуализация данных. Matplotlib. Seaborn. Mayavi [Электронный ресурс]. – URL: devpractice.ru, 2020. – 412 с.
61. Devpractice Team. Линейная алгебра на Python [Электронный ресурс]. – URL: devpractice.ru. 2019. – 114 с.
62. Python. Официальный сайт [Электронный ресурс]. – URL: <http://python.org>.

Учебное издание

Калитвин Владимир Анатольевич

ЧИСЛЕННЫЕ МЕТОДЫ. ИСПОЛЬЗОВАНИЕ PYTHON.

Учебное пособие

Редактор В.И. Буланова

Компьютерная вёрстка и форматирование
В.А. Калитвин

Текст пособия набран в LATEX 2_ε.

Подписано в печать 11.06.2024 г.

Формат 60 × 84 1/16. Бумага для копировальной техники.

Усл. печ.л. 12. Тираж 300 экз.

Заказ № .

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Липецкий государственный педагогический университет

имени П.П. Семенова-Тян-Шанского»

398020, г. Липецк, ул. Ленина, 42

Отпечатано в отделе редакционно-печатной деятельности

ЛГПУ имени П.П. Семенова-Тян-Шанского

398020, г. Липецк, ул. Ленина, 42